



**POLITECNICO
DI TORINO**

POLITECNICO DI TORINO

Master Degree course in Digital skills for sustainable societal transitions

Master Degree Thesis

**CIM Wizard and CIM Assist:
An Object-Oriented Platform for
Extensible City Information Modeling
and Autonomous LLM Agent for Spatial
Querying**

Supervisors

Prof. Lorenzo BOTTACCIOLI

Prof. Edoardo PATTI

PhD. Pietro Rando MAZZARINO

Candidate

Ali TAHERDOUST MOHAMMADI

ACADEMIC YEAR 2025-2026

Acknowledgements

I would like to express my sincere gratitude to my supervisors, Prof. Lorenzo Bottaccioli, Prof. Edoardo Patti and Dr. Pietro Mazzuri, for their continuous support, guidance, and invaluable insights throughout this research work.

I am deeply grateful to my family for their unwavering support and encouragement during my academic journey.

Special thanks to the Politecnico di Torino community for providing an excellent research environment and access to computational resources necessary for this work.

Abstract

Urban energy modeling currently faces two critical obstacles: (1) the lack of an extensible, standardized platform that allows researchers to easily define and integrate custom building feature calculation methods beyond a fixed set of functions, which is necessary to overcome the challenge of sparse and heterogeneous data resources; and (2) the inaccessibility of complex PostGIS spatial SQL to non-technical users, which severely limits efficient data querying and analysis. This thesis presents a comprehensive framework to address these issues through two integrated, open-source systems: CIM Wizard Integrated, an object-oriented platform for city information modeling, and CIM Assist, an LLM-powered natural language interface for spatial data access.

CIM Wizard directly tackles the extensibility and standardization gap by providing a robust, object-oriented FastAPI framework. The platform includes 17 predefined calculators/methods as sample prototypes but is fundamentally designed to allow energy researchers to define and integrate their own customizable methods and logic, ensuring maximum flexibility for diverse urban energy studies. It seamlessly integrates multi-schema PostGIS databases (vector, census, network, raster) with automatic dependency resolution and fallback strategies, filling a critical need for flexible urban energy analytics.

CIM Assist addresses the database interaction complexity through an LLM-powered natural language interface using a sophisticated two-stage approach: (1) an innovative dataset generation pipeline combining rule-based templates (10,800 samples at 98-100% NoErr) with multi-strategy LLM augmentation using GPT-4o-mini, producing 50,000 augmented samples with 97.34% NoErr rate at \$50 total cost; (2) direct Q2SQL fine-tuning using QLoRA on SQLCoder 7B, selected for its SQL-specific pre-training on the BIRD benchmark, achieving optimal balance between accuracy and inference speed; and (3) LangGraph-based agent integration with database introspection tools enabling comprehensive evaluation through execution feedback.

The systematic evaluation framework employs multiple complementary metrics (NoErr, EM, Deep EM, EX, SC, EA) demonstrating 84.58% execution accuracy (EX) for fine-tuned SQLCoder 7B, outperforming frontier models like GPT-4o (30.85%) by 54 percentage points. Agent-based self-correction reveals minimal improvement (EA matching EX), indicating high first-shot quality. The complete pipeline achieves production-ready performance for real-world spatial SQL applications while democratizing access to complex urban energy databases and providing an extensible, open-source foundation for future urban energy research.

Contents

List of Figures

List of Tables

Listings

3.1	PostGIS spatial join with measurement	31
3.2	Database schema tool with spatial awareness	42
4.1	Calculator base class structure	47
4.2	Building height calculator with multiple methods	48
4.3	Automatic dependency resolution	50
4.4	Extending platform with new calculator	61
4.5	Evaluator v2 usage example	68
4.6	Template parameter substitution	81
4.7	Metadata extraction for generated SQL	82
4.8	Question paraphrase strategy	83
4.9	SQL-to-question generation strategy	83
4.10	Question-to-SQL equivalent generation	84
4.11	SQL rewrite strategy	85
4.12	Parameter variation strategy	85
4.13	Stage 2 quality evaluation	87
4.14	Multi-dimensional stratification	90
5.1	Shuffling to address unbalanced distributions	110
5.2	Integrated curation pipeline with stratification and shuffling	110
5.3	QLoRA fine-tuning technical configuration	113

5.4	Training arguments configuration	113
-----	--	-----

List of Acronyms

ADE	Application Domain Extension
AI	Artificial Intelligence
API	Application Programming Interface
BIM	Building Information Modeling
CEA	City Energy Analyst
CIM	City Information Model
CRS	Coordinate Reference System
CSV	Comma-Separated Values
CTGAN	Conditional Tabular Generative Adversarial Network
DH	District Heating
DSM	Digital Surface Model
DTM	Digital Terrain Model
EA	Eventual Accuracy
EM	Exact Match
EX	Execution Accuracy
FMI	Functional Mock-up Interface
FMU	Functional Mock-up Unit
GAN	Generative Adversarial Network
GIS	Geographic Information System
GPU	Graphics Processing Unit
GUI	Graphical User Interface

HELICS	Hierarchical Engine for Large-scale Infrastructure Co-Simulation
HTTP	Hypertext Transfer Protocol
IFC	Industry Foundation Classes
IoT	Internet of Things
ISTAT	Istituto Nazionale di Statistica (Italian National Institute of Statistics)
JSON	JavaScript Object Notation
JSONL	JSON Lines
LLM	Large Language Model
LoD	Level of Detail
LoRA	Low-Rank Adaptation
NL	Natural Language
NoErr	No Error metric
OGC	Open Geospatial Consortium
OSM	OpenStreetMap
PEFT	Parameter-Efficient Fine-Tuning
PostGIS	PostgreSQL Geographic Information System extension
PV	Photovoltaic
QLoRA	Quantized Low-Rank Adaptation
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
RMSE	Root Mean Square Error
SOTA	State of the Art
SQL	Structured Query Language
TABULA	Typology Approach for Building Stock Energy Assessment
TMY	Typical Meteorological Year
UBEM	Urban Building Energy Modeling
UEM	Urban Energy Modeling

UES	Urban Energy System
UESM	Urban Energy System Modeling
VES	Valid Efficiency Score
WFS	Web Feature Service
WGS84	World Geodetic System 1984
WMS	Web Map Service

Chapter 1

Introduction

1.1 Context and Motivation

Urban energy planning faces an increasingly critical challenge: First create an standard information model of the buildings from sparse data and second ease of use with the complexity of spatial data.

analyzing and optimizing energy consumption across complex city infrastructures requires sophisticated spatial database queries that remain largely inaccessible to domain experts. Energy researchers, urban planners, and policymakers possess deep domain knowledge but often lack the technical expertise to formulate complex PostGIS spatial SQL queries needed to extract insights from urban databases.

This accessibility gap has significant practical implications. Questions such as “Show me all buildings within 500 meters of a subway station with annual energy consumption above 100 kWh/m²” or “Calculate the average solar potential for buildings in districts with population density exceeding 5000 people per km²” require intricate spatial joins, aggregations, and domain-specific calculations that are beyond the reach of non-technical users.

Traditional approaches to this problem have relied on either pre-built dashboards with limited flexibility or requiring users to learn SQL—neither solution adequately addresses the need for ad-hoc exploratory analysis in urban energy research.

A fundamental insight underlying this work is that any truly intelligent built environment requires not only sensors and data collection infrastructure, but crucially an **information model generator that maintains structured representations of spatial objects**. Buildings, infrastructure elements, census regions, and environmental features must exist as first-class objects with well-defined properties, relationships, and computational methods. Without such an information model generator holding the place of objects—their geometries, attributes, dependencies, and calculation strategies—intelligent environments lack the semantic foundation necessary for sophisticated reasoning and analysis. This thesis demonstrates that combining CIM Wizard’s object-oriented calculator architecture (providing the information model generator) with CIM Assist’s natural language interface (enabling accessible interaction) creates an integrated intelligent environment where complex spatial-temporal queries become accessible to domain

experts without sacrificing analytical rigor.

1.2 Research Problem

This thesis addresses the challenge of building a flexible platform to create standard information models of buildings from sparse data and a production-ready natural language interface for spatial SQL generation, specifically targeting the City Information Modeling (CIM) domain. The problem encompasses three interrelated challenges that span platform architecture, data generation, and model optimization.

The **platform challenge** arises from the need for an extensible object-oriented framework enabling researchers to define custom building property calculators without modifying core infrastructure. Existing solutions either lack flexibility through fixed dashboard interfaces or demand extensive programming knowledge for custom script development, creating a significant accessibility barrier for domain experts in urban energy research.

The **data challenge** stems from the requirement for large-scale, high-quality datasets combining natural language questions with correct PostGIS spatial queries. Manual annotation proves prohibitively expensive at \$0.50-1.00 per sample, while existing Text-to-SQL benchmarks like Spider and BIRD focus predominantly on standard relational SQL with minimal coverage of spatial database operations essential for urban analytics.

The **model challenge** involves achieving production-ready accuracy for domain-specific spatial SQL generation, where general-purpose language models demonstrate insufficient performance (35-65% accuracy baseline). The architectural decision between multi-agent RAG systems leveraging frontier models (1T+ parameters) with external knowledge retrieval versus fine-tuning local models (7-32B parameters) to internalize domain-specific knowledge presents critical trade-offs across multiple dimensions including data privacy, initial and ongoing costs, environmental impact (CO₂ footprint), and sustained performance quality.

1.3 Proposed Solution

This thesis presents an integrated framework comprising two complementary systems:

1.3.1 CIM Wizard: Object-Oriented Platform

CIM Wizard Integrated provides an extensible FastAPI framework addressing the platform challenge through a calculator-based architecture comprising 17 modular calculators for building properties including height, area, energetic characteristics, and census information, with each calculator supporting multiple calculation methods per feature. The framework enables seamless multi-schema integration providing unified access to vector geometries, census demographics, and raster elevation models through PostGIS spatial database operations. Automatic orchestration handles dependency resolution, fallback strategies, and flexible pipeline execution modes, while the modular design ensures research extensibility allowing energy researchers to add new calculators and methods without modifying core infrastructure code.

1.3.2 CIM Assist: LLM-Powered Interface

CIM Assist addresses the interaction complexity through a three-steps approach:

Step 1: Dataset Generation A novel two-stage pipeline generates high-quality training data cost-effectively. Stage 1 employs rule-based templates with 52 handcrafted patterns covering 11 SQL operation types through systematic parameter variation, producing 10,800 samples at 98-100% NoErr rate with zero computational cost. Stage 2 applies multi-strategy LLM augmentation where GPT-4o-mini generates diverse natural language questions through five complementary strategies (question paraphrase, SQL-to-question, question-to-SQL, SQL rewrite, and parameter variation), producing 50,000 augmented samples at \$50 total cost with 97.34% NoErr rate, which are then curated into 34,993 training samples through validation-first methodology achieving high retention rate.

Step 2: Fine-Tuning Direct Q2SQL fine-tuning using parameter-efficient QLoRA enables domain adaptation on consumer hardware. The primary approach employs single-stage Question $\rightarrow$ SQL architecture providing optimal balance between accuracy, simplicity, and inference speed. SQLCoder 7B was selected as the base model due to its SQL-specific pre-training on BIRD benchmark, achieving superior performance-to-cost ratio compared to larger general-purpose models. QLoRA quantization enables 7-14B parameter model training on single 24GB GPUs through 4-bit quantization with low-rank adaptation, requiring 8-12 hours training time for 7B models. An alternative two-stage architecture (Question $\rightarrow$ Instruction $\rightarrow$ SQL) was systematically evaluated but showed only marginal accuracy improvement (+3-5%) with 2-3x latency increase, making direct Q2SQL the pragmatic choice for production deployment.

Step 3: Agent Integration for Evaluation LangGraph-based agent orchestration enables comprehensive evaluation through tool-augmented iterative refinement. The agent framework integrates database introspection tools for schema exploration and table inspection, query validation tools for syntax checking before execution, execution feedback mechanisms enabling self-correction through error message analysis, and iterative refinement allowing up to 5 correction attempts with automatic geometry column detection. This agent mode evaluation reveals that fine-tuned models achieve minimal self-correction improvement (0-7 percentage points), indicating high first-shot quality where iterative refinement provides diminishing returns, validating the effectiveness of direct Q2SQL fine-tuning for production deployment.

1.4 Main Contributions

This work makes the following contributions to the fields of urban energy informatics and natural language interfaces for databases:

1. **Integrated CIM Platform:** Object-oriented calculator-based architecture enabling extensible urban energy modeling with multi-schema PostGIS integration (vector, census, raster), allowing researchers to extend functionality without core infrastructure

modifications.

2. **Cost-Effective Dataset Generation:** Two-stage pipeline (templates $\rightarrow$ multi-strategy LLM augmentation) producing 50,000 augmented samples at \$50 total cost with 97.34% Stage 2 NoErr rate, demonstrating 3,600x cost reduction versus human annotation.
3. **Validation-First Methodology:** Stage-wise quality validation achieving 71.5% curation retention (14-18x improvement over sequential validation approaches) through inherited SQL correctness, preventing error propagation across pipeline stages.
4. **Production-Ready Fine-Tuned Models:** SQLCoder 7B achieving 84.58% execution accuracy (exceeding 70% production threshold), outperforming frontier API models by 54 percentage points through domain-specific fine-tuning with QLoRA on single 24GB GPU.
5. **Comprehensive Evaluation Framework:** Seven complementary metrics (NoErr, EM, Deep EM, EX, SC, EA, plus robustness metrics OOS/AD/ST) with taxonomy-based analysis enabling systematic assessment across query complexity, spatial operations, and schema patterns.
6. **Open-Source Implementation:** Complete framework including CIM Wizard platform (13,522 LOC), AI4DB dataset generation pipeline, fine-tuning scripts, agent integration, and HuggingFace dataset (taherdoust/ai4cimdb) enabling reproducible research and community extension.

1.5 Methodology Overview

The research methodology follows a systematic pipeline progressing from platform development through evaluation and comparative analysis. Initial platform development designs and implements CIM Wizard Integrated with calculator-based architecture, providing the foundational spatial database infrastructure. Dataset generation executes a two-stage pipeline with validation-first quality control at each stage, ensuring high-quality training data. Fine-tuning trains specialized models using QLoRA parameter-efficient adaptation on the curated dataset, enabling 7-14B parameter model training on single consumer GPUs. Agent integration deploys fine-tuned models through LangGraph-based orchestration with database introspection tools, enabling systematic evaluation through both standard first-shot and iterative self-correction modes. Systematic evaluation creates taxonomy-based benchmarks and applies seven complementary metrics (NoErr, EM, Deep EM, EX, SC, EA, OOS/AD/ST) to assess model performance across complexity dimensions. Comparative analysis evaluates fine-tuned models against baseline open-source models and frontier API services, validating architecture choices through empirical performance measurements. The methodology emphasizes reproducibility through open-source implementation, cost-effectiveness through academic infrastructure utilization, and production readiness through comprehensive quality validation.

1.6 Key Results

The implemented framework demonstrates production-ready performance across all evaluation dimensions. Dataset generation achieves 98-100% Stage 1 NoErr rate through rule-based templates and 97.34% Stage 2 NoErr rate through LLM augmentation, producing 50,000 augmented samples at \$50 total cost representing 3,600x cost reduction versus human annotation. Training efficiency enables SQLCoder 7B fine-tuning in 51 hours on single 24GB GPU through QLoRA parameter-efficient adaptation, demonstrating accessibility of large model fine-tuning on consumer hardware. Model performance results show SQLCoder 7B achieving 84.58% execution accuracy (EX), 81.59% Deep EM structural correctness, and 63.18% semantic correctness (SC), substantially exceeding the 70% production readiness threshold. Domain-specific fine-tuning provides substantial improvement over baselines, with the fine-tuned SQLCoder outperforming GPT-4o by 54 percentage points despite smaller model size. Agent-based self-correction reveals minimal improvement (EA matching EX at 84.58%), indicating high first-shot quality where iterative refinement provides diminishing returns. The complete framework required substantial expert time and computation, with total monetary cost of \$50 for dataset generation plus \$0 for training using academic GPU infrastructure, demonstrating feasibility of cost-effective production-ready spatial SQL interfaces.

1.7 Thesis Organization

The remainder of this thesis proceeds through four complementary chapters. Chapter ?? reviews the state of the art in urban informatics platforms, Text-to-SQL research, and spatial database interfaces, identifying the research gaps this work addresses. Chapter ?? presents foundational technologies including PostGIS spatial databases, large language models, parameter-efficient fine-tuning techniques (QLoRA), and AI agent frameworks (LangGraph), providing the technical background necessary to understand the implementation. Chapter ?? describes the complete methodology including CIM Wizard calculator-based architecture, two-stage dataset generation pipeline with validation-first quality control, direct Q2SQL fine-tuning approach with alternative two-stage evaluation, agent integration for iterative evaluation, and comprehensive seven-metric evaluation framework with taxonomy-based analysis. Chapter ?? presents experimental validation across all methodology phases including dataset generation quality (NoErr rates, curation retention), fine-tuning efficiency (training times, convergence analysis), model performance evaluation (EX, Deep EM, SC, EA metrics with taxonomy breakdowns), comparative analysis against baseline and frontier models, and resource analysis (development effort, computational costs, environmental impact). Chapter ?? synthesizes contributions, key findings, and technical insights while proposing future research directions including multi-database evaluation, parallel Stage 3 generation, and cross-domain applications.

This structure enables readers with different backgrounds to navigate effectively: domain experts in urban energy modeling may focus on Chapters ??, ??, and ?? for platform capabilities and performance validation, while technical readers interested in NLP and LLM engineering may emphasize Chapters ??, ??, and ?? for implementation details and

systematic evaluation methodology.

Chapter 2

State of the Art

This chapter reviews existing research in urban informatics platforms, Text-to-SQL and, spatial database interfaces, positioning our work within the broader research landscape and identifying the gaps this thesis addresses.

2.1 Urban Energy Modeling and City Information Models

2.1.1 Urban Energy Modeling Landscape

The shift toward low-carbon and resilient cities increasingly depends on the ability to model and optimize interactions among buildings, infrastructures, and energy networks. In this context, Urban Energy Modelling (UEM) and its related fields—Urban Building Energy Modelling (UBEM) and Urban Energy System Modelling (UESM)—have become key instruments for planning sustainable interventions, improving energy efficiency, and supporting climate mitigation strategies [?, ?, ?, ?]. UEM plays a strategic role in policy design [?] while demonstrating relevance for evaluating large-scale retrofit scenarios and urban-scale energy planning.

UEM frameworks are broadly classified into top-down and bottom-up approaches [?]. Top-down methods use aggregated socio-economic or sectoral data to estimate urban energy demand, typically lacking spatial detail at the building scale [?]. Conversely, bottom-up approaches model energy use from the ground up, using detailed physical, geometric, and operational data at the individual building level. These methods are increasingly preferred for their ability to produce high-resolution, scenario-specific analyses [?]. However, their effectiveness is often constrained by inconsistent data quality and fragmented sources [?] as well as the lack of common standards [?].

When approaching a real-world Urban Energy System (UES), all needed information must be gathered and processed into a structured City Information Model (CIM) as illustrated in Figure ???. The CIM or whatever data structure is used serves as the starting point for instantiating and performing simulations. While most UEM frameworks cover all steps, this thesis focuses particularly on the data processing, enrichment, and query interaction phases.

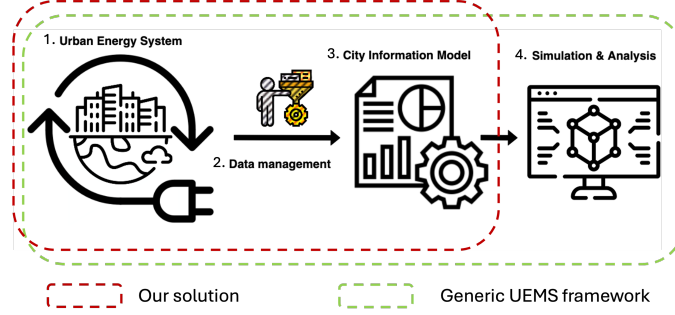


Figure 2.1. Conceptual steps of Urban Energy System Modeling (UESM) showing the flow from raw data through City Information Model generation to simulation and analysis.

2.1.2 Existing Urban Energy Modeling Platforms

Several modeling platforms have significantly advanced the field; however, notable limitations remain. Frameworks such as CityBES [?], UBEM.io [?], City Energy Analyst (CEA) [?], Simstadt [?], and UrbanOpt [?] are designed as all-in-one platforms or tightly integrated toolsets that guide users through the entire process of urban energy scenario analyses. These frameworks differ in their focus and intended applications, making their use highly context-specific. As a result, they often employ custom data models that are closely tied to specific simulation engines. This tight coupling restricts interoperability and hampers the reuse of data across different platforms.

Allegrini et al. [?] provide a comprehensive review of modeling approaches and tools for district-scale energy systems, highlighting challenges in data availability and model integration. Recent work on data enrichment [?] demonstrates successful urban energy modeling in Berlin through systematic data acquisition strategies, while research on data-scarce environments [?] proposes methods for zone-level modeling when detailed building data is unavailable.

2.1.3 Need for Standardized City Information Models

To overcome these limitations, it is necessary to decouple the data management and processing from the simulation phase. Complementary, it is necessary to adopt standardized and widely used CIMs or data structures to ensure interoperability and consistency within different simulation contexts with different focuses and spatio-temporal resolutions. Achieving this requires two key conditions: (i) the availability of modular, plug-and-play, and configurable frameworks, often based on co-simulation principles, capable of managing the simulation and analysis tasks [?, ?, ?, ?]; and (ii) the standardization of CIM outputs to ensure seamless integration with co-simulation platforms as data sources.

A promising candidate for CIM standardization is CityGML [?], including its extensions (Application Domain Extensions, ADEs [?]), its simplified form (CityJSON [?]), and associated database structures (3DcityDB [?]). While some existing or updated frameworks can use CityGML models as inputs for simulation, few currently support the generation of standardized CityGML outputs covering the complete data processing

workflow. Standardized CIMs are also essential for advancing research on urban digital twins [?]. A critical barrier remains the absence of standardized, interoperable database architectures [?].

2.1.4 Research Gaps and Requirements

From a literature review of the state of the art, it is possible to highlight the following main challenges, needs or gaps:

- (1) **Flexibility and adaptability:** there is a need for flexible and extensible data architectures and processing methods that support heterogeneous inputs and deal with possible data scarcity [?, ?];
- (2) **Standardization:** there is a need for generating standardized outputs to ensure data exchange and reusability in different applications, standardization should be central, not only for outputs generation but also in both data storing and process calculations [?, ?, ?];
- (3) **Application independence:** There is a need for tools that can facilitate the creation of a configurable city information model independently to the application of the study and the chosen simulation platform or models [?, ?]. These tools should be at least customizable to embed data to adapt to the specific model parameters requirements.

2.1.5 CIM Wizard Framework

Building on previous work [?], the CIM Wizard framework addresses these identified gaps by presenting a novel service-based, flexible, and configurable UEM framework. CIM Wizard generates GIS-based city information models of a chosen case study area, enabling different workflows and different user interactions. It decouples data management, execution logic, and feature computation, enabling flexible orchestration of modeling steps via HTTP/REST interfaces. It supports runtime method selection based on available inputs and dependency resolution and integrates both public and private data sources—including Open Street Map (OSM), Digital Surface Models (DSMs), cadastral databases, and utility networks. The system offers native export to CityJSON enriched with Energy and Utility Application Domain Extensions (ADEs). By bridging flexibility and standard compliance, CIM Wizard represents a significant step forward in enabling robust, context-aware UEM pipelines ready to be deployed across diverse urban environments and connected digital infrastructures.

2.2 Beyond General Models: Internalizing Database Schema via Fine-Tuning for High-Accuracy Text-to-SQL

Text-to-SQL generation presents fundamentally different challenges compared to general text-to-text tasks, requiring models to bridge the gap between natural language semantics and precise database query syntax. While modern large language models excel at general language understanding and generation, achieving production-ready accuracy in Text-to-SQL demands domain-specific knowledge internalization that cannot be adequately addressed through prompting or retrieval-augmented generation alone.

The fundamental distinction lies in the dual constraints that Text-to-SQL must satisfy simultaneously: linguistic naturalness in understanding user questions and strict syntactic and semantic correctness in generating executable database queries. Unlike general text generation where multiple valid formulations exist and minor errors may be acceptable, SQL queries must satisfy exacting requirements including correct table and column references, valid join conditions, appropriate aggregation functions, and schema-specific constraints. A single incorrect column name or misplaced join condition renders an entire query non-executable, regardless of how well the model understood the user’s intent. This binary correctness requirement creates a much narrower margin for error compared to general language tasks where partial understanding can still produce useful outputs.

The difficulty of Text-to-SQL is starkly evident in the persistent performance gap between automated systems and human experts, even with the most advanced frontier models and sophisticated agentic frameworks. As of November 2025, the BIRD benchmark [?] leaderboard reveals that human database experts achieve substantially higher execution accuracy than the best automated systems, including multi-agent frameworks leveraging frontier models like Gemini-2.5-Pro and sophisticated fine-tuning approaches [?]. This substantial gap persists despite years of research advances, demonstrating that Text-to-SQL remains fundamentally more challenging than general language tasks where state-of-the-art models often match or exceed human performance.

The persistent human advantage stems from several factors that current automated systems struggle to replicate. Human database experts possess deep schema understanding that extends beyond surface-level table and column names to include semantic relationships, data distributions, and domain-specific query patterns. They can reason about query efficiency, recognize edge cases, and apply domain knowledge that may not be explicitly encoded in the database schema. Furthermore, humans excel at handling ambiguous questions by making reasonable assumptions based on context, whereas automated systems often require explicit clarification or generate queries that fail due to missing information.

This performance gap motivates the need for fine-tuning approaches that internalize database schema knowledge directly into model parameters, rather than relying solely on external retrieval or prompting strategies. While agentic frameworks with RAG can dynamically access schema information, they remain limited by the reasoning capabilities of general-purpose models that lack deep understanding of database-specific patterns. Fine-tuning enables models to learn these patterns directly, encoding schema relationships, common query structures, and domain-specific SQL formulations into the model weights themselves. The BIRD benchmark results demonstrate that even the most sophisticated agentic approaches fall short of human performance, suggesting that achieving production-ready accuracy requires moving beyond general models toward specialized, fine-tuned systems that internalize the complex knowledge required for accurate SQL generation.

2.3 Natural Language Interfaces to Databases

The quest to enable non-technical users to interact with databases through natural language has driven research spanning five decades, evolving through distinct technological paradigms that reflect broader advances in artificial intelligence and natural language processing. This historical progression reveals a fundamental tension between system flexibility and domain specificity, with each generation of approaches attempting to balance the competing demands of broad applicability and reliable performance.

2.3.1 Historical Approaches

The earliest natural language database interfaces emerged in the 1970s and 1980s, employing rule-based systems that relied on pattern matching and grammar-based parsing techniques. These pioneering systems, while conceptually elegant, suffered from severe limitations: they operated within constrained vocabularies and could handle only simple query structures, requiring extensive domain-specific customization for each new database application. The brittleness of these rule-based approaches became apparent as they failed to generalize across domains or handle the linguistic variation inherent in natural human communication, ultimately limiting their practical deployment to highly specialized, controlled environments.

The transition to semantic parsing approaches in the 2000s and 2010s represented a significant paradigm shift, moving from handcrafted rules to supervised learning methods that constructed logical forms from natural language inputs. These systems demonstrated improved flexibility by learning mappings from training examples, yet they faced their own fundamental constraints: they required extensive annotated training data for each new domain, and they struggled with compositional generalization—the ability to understand novel combinations of familiar concepts. While semantic parsing systems achieved notable successes in specific domains, their data requirements and limited generalization capabilities prevented widespread adoption.

The neural sequence-to-sequence revolution that emerged around 2015 marked a transformative moment, introducing encoder-decoder architectures based on LSTM and later Transformer models that could learn end-to-end mappings from questions to SQL queries. These neural approaches demonstrated substantially improved generalization capabilities compared to their predecessors, learning statistical patterns from large training corpora rather than relying on explicit rule specification. However, their performance remained fundamentally limited by the quantity and quality of available training data. The breakthrough came with attention mechanisms and large-scale pre-training, which enabled models to capture long-range dependencies and transfer knowledge across domains, setting the stage for the current era of large language model-based interfaces.

2.3.2 Modern LLM-Based Approaches (2020-Present)

The emergence of large language models in 2020 fundamentally transformed the landscape of natural language database interfaces, enabling entirely new paradigms that leverage the massive pre-training and few-shot learning capabilities of models trained on internet-scale

text corpora. These modern approaches can be broadly categorized into three complementary strategies, each addressing different deployment constraints and performance requirements.

Zero-shot and few-shot prompting approaches leverage the inherent capabilities of large pre-trained models to generate SQL queries without any task-specific fine-tuning, enabling rapid deployment with minimal development overhead. The primary advantage of this paradigm lies in its immediacy: systems can be deployed quickly without the substantial upfront investment required for dataset creation and model training. However, this convenience comes with significant trade-offs: performance on domain-specific queries remains inconsistent, often failing on complex spatial operations or specialized database schemas that differ from the general patterns seen during pre-training. Furthermore, the reliance on closed-source API services introduces substantial per-query costs that scale linearly with usage, making this approach economically challenging for high-volume production deployments.

Fine-tuned specialized models address the performance limitations of zero-shot approaches by internalizing domain-specific knowledge through supervised learning on curated training datasets. These models achieve consistent, high-quality performance on domain-specific queries by learning the precise patterns, schema structures, and query formulations relevant to their target applications. The inference cost structure fundamentally differs from API-based approaches: while fine-tuning requires an upfront training investment, operational costs remain minimal since inference runs on local infrastructure without per-query API fees. The critical trade-off involves balancing the upfront training investment against long-term operational efficiency, with fine-tuning becoming economically advantageous when query volume exceeds threshold levels or when data privacy requirements preclude external API usage.

Hybrid approaches have emerged as sophisticated synthesis strategies that combine the strengths of multiple paradigms. Retrieval-augmented fine-tuning systems such as ReFSQL and SQL-Fuse integrate external knowledge retrieval with fine-tuned models, enabling dynamic schema awareness while maintaining domain-specific performance. Agent-based systems with tool use extend this concept further, allowing models to interact with databases through introspection tools, query validation, and execution feedback loops. Schema-aware prompting with demonstrations represents another hybrid strategy, where carefully selected example queries provide contextual guidance to general-purpose models, improving performance without full fine-tuning. These hybrid approaches reflect the recognition that no single paradigm dominates across all deployment scenarios, and that effective production systems often require carefully orchestrated combinations of techniques tailored to specific application requirements.

2.4 Text-to-SQL Research

2.4.1 Benchmark Datasets and Evaluation

The Text-to-SQL field has matured through standardized benchmarks enabling systematic comparison:

Spider Benchmark The Spider benchmark [?] established the foundation for cross-domain Text-to-SQL evaluation with 10,181 questions spanning 200 databases across 138 diverse domains. The benchmark emphasizes cross-domain generalization through complex SQL constructs including nested queries, multi-table joins, and aggregation operations, with evaluation based on exact match (EM) comparing generated SQL strings and execution accuracy (EX) comparing query results. However, Spider’s focus on standard SQL operations means it provides limited coverage of spatial database functions, leaving a gap for PostGIS-specific Text-to-SQL research.

BIRD Benchmark The BIRD benchmark [?] advances Text-to-SQL evaluation through 12,751 text-to-SQL pairs that emphasize not only correctness but also execution accuracy and query efficiency on realistic database schemas. Complementary benchmarks include OmniSQL [?], which provides universal benchmarking across diverse SQL dialects, and comprehensive evaluation studies [?] analyzing LLM performance across multiple Text-to-SQL benchmarks. BIRD incorporates external knowledge requirements that mirror real-world database interaction scenarios where users must understand domain concepts beyond schema structure. Performance results demonstrate that GPT-4 achieves strong execution accuracy, while fine-tuned open-source models in the 32B parameter range reach competitive performance levels, establishing BIRD as the current state-of-the-art benchmark for Text-to-SQL system evaluation.

Performance Trends Recent comprehensive surveys [?, ?] document the rapid progress in Text-to-SQL capabilities driven by large language models. Closed-source commercial models including GPT-4 and Claude 3.5 achieve strong execution accuracy on the challenging BIRD benchmark, demonstrating strong generalization to complex database queries. Open-source fine-tuned models ranging from 14B to 32B parameters reach competitive performance levels, narrowing the gap with proprietary systems. Despite this progress, significant challenges remain in handling complex nested queries with multiple levels of subquery nesting and incorporating domain-specific knowledge that extends beyond the database schema itself.

2.4.2 Agent-Based and Reasoning Approaches

A major research direction in Text-to-SQL has emerged that decomposes the complex task of SQL generation into more manageable subtasks through multi-step reasoning and agent-based architectures. Rather than attempting direct question-to-SQL translation in a single pass, these approaches recognize that effective SQL generation benefits from explicit intermediate reasoning steps that mirror how human database experts approach query construction.

The foundation of this paradigm lies in chain-of-thought prompting, where models are guided to decompose natural language questions into explicit reasoning steps before generating SQL. This decomposition process involves identifying relevant schema elements, determining appropriate join paths between tables, and selecting suitable aggregation operations. Empirical results demonstrate that this explicit reasoning approach yields substantial improvements in execution accuracy compared to direct generation methods.

The effectiveness stems from forcing the model to explicitly reason about the query structure before committing to SQL syntax, reducing errors from premature SQL generation.

Building upon chain-of-thought foundations, multi-agent collaboration systems have emerged as a sophisticated extension that distributes Text-to-SQL responsibilities across specialized agents, each handling distinct aspects of the query generation pipeline. These systems typically employ separate agents for schema linking, query planning, SQL writing, and validation, enabling each component to focus on its specialized domain. For instance, DAIL-SQL demonstrates this approach through demonstration retrieval and explanation generation, where one agent identifies similar queries while another generates reasoning explanations. Notable contributions in this space include GTChain [?], which introduces graph-based Text-to-SQL with chain-of-thought reasoning, CHESS [?] for contextual harnessing enabling efficient SQL synthesis, and SQL Agent [?] which implements autonomous Text-to-SQL with self-correction capabilities. However, this architectural sophistication comes with inherent challenges: complex orchestration overhead increases system latency and complexity, while error propagation across multiple agents can amplify mistakes, requiring careful design of inter-agent communication and error handling mechanisms.

The most recent evolution in agent-based Text-to-SQL systems integrates database introspection tools directly into the reasoning loop, creating tool-augmented reasoning architectures that enable dynamic interaction with the target database. These systems can dynamically retrieve table structures through schema inspection tools, validate SQL syntax before execution through query validation tools, and iteratively refine generated queries based on execution feedback when errors occur. This iterative refinement process, where agents receive error messages from failed query executions and use this feedback to generate corrected SQL, has demonstrated remarkable effectiveness, achieving substantial improvements in eventual accuracy through self-correction. The tool-augmented approach transforms Text-to-SQL from a static generation task into a dynamic, interactive process where agents can explore the database schema, validate their understanding, and learn from execution failures, mirroring the iterative debugging process that human SQL developers employ.

2.4.3 Schema Linking and Retrieval-Augmented Generation

Schema linking maps natural language tokens to database schema elements (tables, columns, values):

IRNet (Guo et al., 2019) IRNet [?] represents pioneering work formalizing schema linking as an explicit intermediate step in Text-to-SQL generation. The approach introduces an intermediate representation (IR) that decouples natural language understanding from SQL syntax generation, enabling the model to reason about database schemas independently of query formulation. This architectural innovation yields significant improvements in cross-domain generalization, as the learned schema linking patterns transfer more effectively across different database structures.

Modern Retrieval-Augmented Approaches Modern retrieval-augmented approaches combine external knowledge retrieval with model reasoning capabilities to enhance Text-to-SQL performance. ReFSQL [?] combines retrieval with fine-tuning by using similar examples as contextual guidance, while SQL-Fuse [?] implements a hybrid architecture fusing retrieval-augmented and fine-tuned models to leverage complementary strengths. GraFix-T5 [?] introduces graph-augmented schema representations with attention mechanisms enabling more sophisticated reasoning about database structure. OpenSQL [?] democratizes Text-to-SQL through open-source LLM approaches, reducing dependency on proprietary systems, and SpatialRAG [?] extends retrieval-augmented generation specifically for spatial reasoning with LLMs. Empirical evaluations demonstrate that schema-aware models consistently outperform schema-agnostic baselines by substantial margins, validating the importance of explicit schema information in Text-to-SQL systems.

Spatial Schema Complexity Spatial databases introduce additional challenges beyond standard relational schema linking, requiring models to understand geometric data representations and spatial operations. Geometry column identification demands recognizing that columns with names like `building_geometry` or `tract_geometry` contain spatial data types requiring specialized functions rather than standard SQL operations. Spatial relationship mapping necessitates translating natural language proximity concepts such as “near” or “within distance” into appropriate PostGIS functions like `ST_DWithin` with correct parameter specifications. Coordinate reference system awareness requires understanding that spatial operations may need coordinate transformations when integrating data from different geographic projections. Finally, raster-vector integration requirements add complexity when queries must combine continuous raster datasets (elevation, land cover) with discrete vector geometries through specialized functions like `ST_Value` and `ST_Clip`.

2.4.4 Fine-Tuning Approaches

Fine-tuning specializes models on domain-specific Text-to-SQL tasks:

Synthetic Data Generation Creating high-quality training data for domain-specific Text-to-SQL systems presents significant methodological challenges. **Template-based** approaches employ rule-driven SQL generation with systematic parameter substitution, providing high correctness guarantees but limited diversity in query patterns. **LLM-generated** approaches leverage models like GPT-4 to create diverse question-SQL pairs with natural language variation, though at significant cost and with correctness challenges requiring validation. **Execution-guided filtering** validates synthetic samples through actual database execution, ensuring functional correctness but requiring careful database state management. The fundamental challenge lies in balancing these three competing objectives: maximizing sample diversity to prevent overfitting, ensuring correctness to avoid training on erroneous patterns, and controlling generation costs to enable research-scale dataset creation.

Model Architectures The evolution of Text-to-SQL model architectures reflects broader trends in natural language processing. **Encoder-decoder architectures** including T5 and BART represented earlier approaches using sequence-to-sequence models that explicitly encode input questions and decode output SQL, providing strong performance but requiring paired encoder-decoder training. **Decoder-only architectures** such as Llama and Qwen represent the current state-of-the-art, using autoregressive large language models that generate SQL through next-token prediction, benefiting from massive pre-training and superior few-shot learning capabilities. **Specialized models** like SQL-Coder incorporate SQL-specific pre-training on curated code corpora, achieving strong performance in smaller parameter counts through domain-focused knowledge distillation.

Parameter-Efficient Methods QLoRA [?] has democratized large language model fine-tuning by enabling adaptation of 14B-70B parameter models on consumer-grade GPUs through quantized low-rank adaptation. The practical impact cannot be overstated, as QLoRA transforms model fine-tuning from a resource-intensive enterprise requiring dedicated high-memory hardware into an accessible research activity feasible on single 24GB GPUs. Remarkably, performance analysis reveals minimal degradation compared to full fine-tuning, with typical accuracy differences of only 1-3%, making parameter-efficient methods the pragmatic choice for domain-specific adaptation in resource-constrained settings.

2.4.5 Research Programs Led by Pourreza and Li

Two long-running research teams—led respectively by Prof. Mohammadreza Pourreza at the University of Alberta and Prof. Haoyang Li at Renmin University of China—have produced a cohesive body of work that effectively maps the evolution of advanced Text-to-SQL systems. Reviewing their papers provides a structured understanding of how decomposition, schema modeling, and open-source engineering are converging toward production-ready NL2SQL pipelines.

University of Alberta (Pourreza Lab) Pourreza and Rafiei pioneered decomposition-first prompting strategies that close the gap between few-shot prompting and large-scale fine-tuning. DIN-SQL [?] shows that breaking a question into intermediate reasoning steps and feeding partial solutions back into an LLM boosts execution accuracy on Spider by roughly ten percentage points. DTS-SQL [?] extends the same paradigm to small open-source checkpoints by introducing a two-stage fine-tuning routine that trains planners and SQL generators separately, narrowing the performance gap with proprietary 70B+ models while preserving data privacy.

Renmin University of China (Li Lab) Li and collaborators focus on strengthening schema grounding and open ecosystems. RESDSQL [?] decouples schema linking from skeleton generation, injecting only the ranked relevant tables and columns into the encoder and forcing the decoder to first draft SQL templates before filling literals, which improves robustness on Spider-DK/Syn/Realistic. CodeS [?] complements this structural

insight with a family of fully open-source language models (1B-15B) pre-trained on SQL-centric corpora; retrieval-aware prompting and bi-directional augmentation make CodeS a practical alternative for organizations that cannot rely on closed APIs.

Joint Perspective: CHASE-SQL The two teams recently joined forces on CHASE-SQL [?], an ICLR contribution that pairs divide-and-conquer query planning with preference-optimized candidate selection. Multiple LLM agents explore SQL candidates using execution-plan-aware chain-of-thought reasoning, instance-aware synthetic demonstrations, and dynamic planning, while a binary selector ranks candidates through pairwise comparisons. The resulting system tops the BIRD leaderboard (73% EX) and demonstrates how decomposition, schema awareness, and multi-agent self-evaluation can be woven into an “amazing” end-to-end Text-to-SQL stack.

2.4.6 Text-to-Spatial-SQL: An Emerging Domain

While Text-to-SQL has extensive literature, Text-to-Spatial-SQL remains nascent:

Wang et al. (2025) [?] Wang et al. provide the most significant recent contribution to Text-to-Spatial-SQL research through a bilingual dataset (English/Chinese) covering four domains: administrative boundaries, education facilities, tourism points of interest, and traffic infrastructure. Their work employs SpatiaLite as the lightweight spatial database platform and evaluates GPT prompting approaches without fine-tuning using basic correctness metrics. However, the work exhibits several limitations including absence of fine-tuning exploration for domain adaptation, limited domain coverage compared to comprehensive Text-to-SQL benchmarks, and lack of systematic benchmark methodology with standardized evaluation protocols, leaving substantial opportunities for advancement in spatial SQL generation research.

Research Gap The Text-to-Spatial-SQL domain exhibits several critical gaps compared to general Text-to-SQL research. The field lacks standardized benchmarks comparable to Spider or BIRD that would enable systematic cross-study comparisons and reproducible evaluation. Large-scale datasets suitable for fine-tuning open-source models remain unavailable, limiting research into domain-specific adaptation approaches. Systematic evaluation methodologies for schema-aware pipelines have not been established, preventing comprehensive understanding of architecture trade-offs. Production-ready implementations for real-world spatial database applications are notably absent from the literature. Finally, domain-specific applications in critical areas including urban energy modeling, transportation planning, and healthcare spatial analytics lack the natural language interfaces that would democratize access for non-technical domain experts.

2.5 Evaluation Metrics for Text-to-SQL

The evaluation of Text-to-SQL systems presents unique challenges that distinguish this domain from standard natural language processing tasks. Unlike classification or generation tasks where ground truth outputs are unambiguous, SQL queries exhibit multiple valid formulations that produce identical results, making evaluation fundamentally more complex than simple string matching. This inherent ambiguity has driven the development of diverse evaluation metrics, each capturing different aspects of system performance and serving distinct purposes in the research and development lifecycle.

2.5.1 Standard Metrics

Exact Match (EM) represents the most straightforward evaluation approach, measuring whether the generated SQL string exactly matches the ground truth query character-by-character. This metric offers the significant advantage of computational simplicity: it requires no database access, executes instantaneously, and provides deterministic, reproducible results. However, this simplicity comes at substantial cost, as EM fundamentally penalizes semantically equivalent query variations. Two queries that produce identical results but differ in column ordering, alias naming, or whitespace formatting will be marked as incorrect under EM, despite being functionally equivalent. This limitation becomes particularly problematic in research contexts where models may learn different but equally valid SQL formulations, leading to artificially depressed performance scores. Empirical results across Text-to-SQL benchmarks reveal that competitive models typically achieve EM scores in the 20-50% range, reflecting both the inherent difficulty of exact string matching and the diversity of valid SQL formulations.

Execution Accuracy (EX) addresses the fundamental limitation of EM by evaluating whether generated SQL queries produce the same results as ground truth queries when executed against the target database. This metric accepts semantically equivalent queries regardless of their syntactic differences, providing a more realistic assessment of system performance in production contexts where functional correctness matters more than exact formulation. The primary advantage of EX lies in its alignment with real-world requirements: users care about obtaining correct results, not about generating queries that match a specific reference formulation. However, EX introduces significant implementation complexity, requiring database execution infrastructure, careful handling of result set comparison (including ordering, null values, and floating-point precision), and robust error handling for queries that fail to execute. The computational overhead of EX scales with benchmark size, as each generated query must be executed and its results compared against ground truth. Despite these challenges, EX has become the de facto standard for Text-to-SQL evaluation, with competitive models achieving 60-85% execution accuracy on challenging benchmarks like BIRD, reflecting both the difficulty of the task and the substantial progress made in recent years.

2.5.2 Novel Metrics

As Text-to-SQL systems have matured and moved toward production deployment, researchers have recognized that execution correctness alone provides an incomplete picture of system quality. The BIRD benchmark introduced the Valid Efficiency Score (VES), which incorporates query execution efficiency as an additional dimension of evaluation. This metric recognizes that in production environments, a correct query that executes in minutes may be practically unusable compared to an equivalent query that completes in seconds. The VES formula combines execution accuracy with an efficiency penalty:

$$\text{VES} = \text{EX} \times \left(1 - \min \left(\frac{T_{\text{pred}} - T_{\text{gold}}}{T_{\text{gold}}}, 1 \right) \right) \quad (2.1)$$

This formulation penalizes queries that execute significantly slower than the ground truth, while still rewarding correctness. The efficiency consideration becomes particularly important for systems deployed in interactive applications where query latency directly impacts user experience, and for large-scale analytical workloads where inefficient queries can consume excessive computational resources.

The emergence of agent-based Text-to-SQL systems with iterative refinement capabilities has necessitated the development of agent-specific metrics that capture the unique characteristics of multi-step query generation processes. First-shot accuracy measures the success rate on the initial query attempt, providing insight into the base model’s capability before any refinement occurs. This metric serves as a baseline for understanding the fundamental performance of the underlying language model or fine-tuned system. Eventual accuracy, in contrast, measures the final success rate after all iterative refinement attempts, capturing the full capability of the agent system including its self-correction mechanisms. The gap between first-shot and eventual accuracy quantifies the value added by iterative refinement, with larger gaps indicating more effective error recovery capabilities. Self-correction rate provides a complementary perspective, measuring the percentage of queries that initially fail but are successfully corrected through iterative refinement, offering insight into the agent’s learning and adaptation capabilities. Finally, average iterations measures the mean number of attempts required until success, providing an efficiency metric that balances accuracy gains against computational overhead. Together, these agent-specific metrics enable comprehensive evaluation of iterative systems, supporting both research into self-correction mechanisms and practical deployment decisions that must balance accuracy, latency, and computational cost.

2.6 Synthetic Data Generation (data augmentation)

The challenge of creating large-scale, high-quality training datasets for Text-to-SQL systems has driven significant research into synthetic data generation techniques. Unlike many machine learning domains where abundant labeled data exists naturally, Text-to-SQL requires carefully constructed question-query pairs that must satisfy both linguistic naturalness and database correctness constraints. This dual requirement has motivated exploration of LLM-based data augmentation approaches that leverage large language

models to generate diverse, semantically valid training examples while maintaining strict execution correctness.

2.6.1 LLM-Based Data Augmentation Strategies

Large Language Models have emerged as powerful tools for synthetic data generation in Text-to-SQL, offering the ability to generate diverse question-query pairs that maintain both linguistic naturalness and database correctness. Unlike traditional generative approaches that struggle with structural constraints, LLMs can be explicitly guided through prompts to produce valid SQL queries while generating natural language variations that capture the linguistic diversity needed for robust model training.

Modern LLM-based augmentation employs multi-strategy approaches that systematically expand training datasets through five complementary transformation strategies. **Question Paraphrase** rephrases natural language questions while preserving exact SQL intent and all specific values, generating linguistic diversity without altering query semantics. **SQL-to-Question** generates natural language questions from existing SQL queries, creating reverse mappings that provide alternative question formulations for the same query pattern. **Question-to-SQL** generates equivalent SQL queries using different approaches (alternative join orders, spatial function variations, subquery vs. CTE conversions) while maintaining identical results, expanding SQL syntactic diversity. **SQL Rewrite** rewrites queries using different syntax or structure (explicit JOIN vs. implicit WHERE, reordered joins, equivalent spatial functions) while producing identical results, creating syntactic variations without semantic changes. **Parameter Variation** creates new question-SQL pairs with different but realistic parameter values (heights, areas, distances, IDs) while maintaining the same query structure, expanding dataset coverage through value space exploration.

The effectiveness of multi-strategy LLM augmentation depends critically on task-appropriate model selection to optimize cost-performance trade-offs. Language tasks such as question paraphrasing and parameter variation benefit from cost-effective models like GPT-4o-mini, achieving high-quality results at substantially lower costs. SQL reasoning tasks requiring complex query generation and equivalent formulation benefit from more powerful models like GPT-4.1, ensuring correct SQL generation while exploring alternative query structures. This strategic model assignment enables large-scale dataset augmentation (50,000+ samples) at reasonable costs (approximately 54 EUR) while maintaining high quality standards.

2.6.2 Quality Validation and Filtering

LLM-generated synthetic data requires rigorous quality validation to ensure training readiness, as unvalidated generation can produce semantically incorrect or syntactically invalid queries. The **validation-first** methodology executes all generated SQL queries against the target database before proceeding to subsequent stages, filtering out non-executable samples early in the pipeline. This approach prevents error propagation and dramatically improves final dataset quality, achieving 97.34% NoErr (executable query rate) compared to traditional approaches that achieve only 10-30% success rates.

The validation process employs comprehensive error classification, categorizing execution failures into `SCHEMA_ERROR` (invalid table or column references), `SYNTAX_ERROR` (malformed SQL grammar), `TIMEOUT_ERROR` (excessive execution time), and `OTHER_ERROR` (database-specific issues). Samples passing execution validation undergo additional filtering steps including semantic deduplication using sentence embeddings to remove near-duplicate questions, length filtering to ensure appropriate complexity, and terminology validation to maintain domain relevance. This multi-stage validation ensures that only high-quality, executable samples proceed to training, addressing the fundamental limitation of prior synthetic generation approaches that achieve low success rates due to insufficient validation mechanisms.

2.7 Research Gaps and Thesis Positioning

2.7.1 Identified Gaps

This literature review reveals several critical gaps:

Gap 1: Extensible Urban Energy Platforms Existing urban energy modeling platforms lack calculator-based extensibility allowing researchers to add custom building property computations without core infrastructure modifications. Multi-schema integration enabling seamless access to vector geometries, census demographics, and raster elevation models remains poorly supported across current frameworks. Natural language interfaces that would democratize spatial database access for non-technical domain experts are either completely absent or provide only limited functionality insufficient for exploratory urban analytics.

Gap 2: Cost-Effective Dataset Generation Manual annotation of Text-to-SQL training data proves prohibitively expensive at \$0.50-1.00 per sample, creating barriers to large-scale dataset creation for domain-specific applications. Existing synthetic generation approaches employing GANs or template-based methods achieve low success rates (typically 10-30% executable queries for complex SQL), limiting their practical utility. Validation-first methodologies that validate quality at each generation stage to prevent error propagation have not been demonstrated in prior work, resulting in inefficient pipelines requiring expensive post-generation filtering.

Gap 3: Text-to-Spatial-SQL The Text-to-Spatial-SQL domain lacks standardized benchmarks comparable to Spider or BIRD that would enable systematic evaluation and cross-study comparisons. Limited evaluation of fine-tuned open-source models means that existing work focuses primarily on prompting frontier models through APIs, missing opportunities for cost-effective domain adaptation. Insufficient focus on PostGIS-specific spatial operations (`ST_Distance`, `ST_Intersects`, `ST_Buffer`) and multi-schema integration patterns leaves critical functionality underexplored. The absence of production-ready implementations prevents validation of proposed approaches in real-world deployment scenarios.

Gap 4: Systematic Evaluation Limited evaluation of two-stage fine-tuning architectures (question→instruction→SQL) compared to direct generation approaches leaves architectural trade-offs poorly understood. Agent-based self-correction mechanisms remain under-explored, with insufficient analysis of iterative refinement benefits versus single-shot generation quality. Dataset quality metrics such as NoErr (executable query rate) receive insufficient emphasis in prior work, despite their critical importance for validating synthetic data generation approaches and ensuring training data correctness.

2.7.2 Thesis Contributions Addressing Gaps

This thesis systematically addresses each identified gap through targeted technical contributions:

1. **CIM Wizard Platform:** Calculator-based architecture with multi-schema PostGIS integration enabling extensible urban energy modeling (addresses Gap 1)
2. **Cost-Effective Dataset Generation:** Two-stage pipeline producing 50,000 augmented samples at \$50 cost with 97.34% Stage 2 NoErr rate through validation-first methodology (addresses Gap 2)
3. **Text-to-Spatial-SQL Implementation:** Fine-tuned SQLCoder 7B achieving production-ready execution accuracy with comprehensive PostGIS operation coverage and taxonomy-based benchmark evaluation (addresses Gap 3)
4. **Architecture Comparison:** Systematic evaluation of direct Q2SQL versus two-stage fine-tuning with agent-based self-correction analysis using seven complementary metrics (NoErr, EM, Deep EM, EX, SC, EA, OOS/AD/ST) (addresses Gap 4)
5. **Production-Ready Implementation:** End-to-end framework achieving production threshold with substantial improvement over frontier models, validated through open-source release (addresses Gaps 1, 3, 4)

2.8 Summary

This chapter reviewed the state of the art across Text-to-SQL (benchmarks, agent approaches, schema linking, fine-tuning), urban informatics platforms, natural language database interfaces, synthetic data generation, and evaluation metrics. The analysis reveals that while Text-to-SQL has matured significantly, Text-to-Spatial-SQL remains nascent, lacking standardized benchmarks, large-scale datasets, and production-ready implementations.

Urban energy platforms similarly lack the extensibility and natural language accessibility required for non-technical domain experts. Existing synthetic data generation approaches achieve limited success for SQL (10-30% executable queries), and validation-first methodologies have not been explored.

This thesis addresses these gaps through the integrated CIM Wizard and CIM Assist framework, detailed in Chapter ??, with results demonstrating production-ready performance presented in Chapter ??.

Chapter 3

Background Technologies

This chapter introduces the foundational technologies underlying the CIM Wizard and CIM Assist systems. We cover spatial databases and PostGIS, large language models and transformer architectures, parameter-efficient fine-tuning techniques, and AI agent frameworks.

3.1 Urban Informatics and Smart Cities

3.1.1 Digital Twin Technologies

Urban informatics integrates geospatial data, sensors, and analytics for comprehensive city management, with several platforms emerging to address different aspects of urban digital transformation. Digital twin technologies have gained prominence in smart city applications [?, ?], providing virtual representations of urban systems that enable real-time monitoring and simulation-based decision making. Recent advances in urban energy management [?, ?] demonstrate the integration of IoT, digital twins, and GIS-based statistical models for multi-scale policy support.

3.1.2 Urban Platform Examples

OpenCity [?] represents an open-source platform designed for urban simulation and planning, with particular emphasis on multi-agent urban modeling that simulates complex interactions between city stakeholders and infrastructure systems. While OpenCity provides powerful simulation capabilities, its primary focus on agent-based modeling results in limited support for direct database query interfaces, restricting ad-hoc analytical access to underlying urban data.

CityLeo [?] takes a different approach by providing an LLM-enhanced urban data platform specifically designed for decision support applications. The platform integrates multiple data sources spanning demographics, transportation networks, and environmental monitoring, creating a unified urban information ecosystem. However, CityLeo’s general-purpose design orientation means it lacks the specialized features and domain-specific optimizations necessary for detailed urban energy modeling and building-level energy analysis.

3.1.3 Building Information Modeling Standards

Building Information Modeling provides comprehensive 3D object-oriented representations of building structures, standardized through industry specifications such as IFC (Industry Foundation Classes) and CityGML [?, ?]. BIM systems excel at capturing detailed geometric and semantic information about individual buildings, supporting lifecycle management from design through construction and maintenance phases. However, this building-centric focus creates a significant gap when scaling to city-wide energy analysis, as BIM frameworks lack the computational infrastructure and analytical capabilities necessary for integrated urban energy modeling across hundreds or thousands of buildings simultaneously.

Large-scale building energy datasets [?] provide valuable resources for urban energy modeling but require sophisticated analytical frameworks to extract actionable insights. The convergence of these technologies reveals critical needs for platform flexibility, data integration across multi-schema sources, and extensible architectures that support custom analytical capabilities without core system modifications.

3.2 Spatial Databases and PostGIS

3.2.1 Spatial Data Types

Spatial databases extend traditional relational databases with geometric and geographic data types, enabling sophisticated spatial analysis capabilities essential for urban informatics applications. The Open Geospatial Consortium (OGC) defines standard spatial types that form the foundation of spatial database operations. **Point** geometries represent single locations such as building centroids or sensor positions, while **LineString** geometries model linear features as sequences of connected points, making them ideal for representing road networks or utility lines. **Polygon** geometries define closed shapes with interior areas and optional holes, commonly used for building footprints, land parcels, or administrative boundaries. For more complex spatial entities, the OGC standard defines collection types including **MultiPoint**, **MultiLineString**, and **MultiPolygon** geometries that group multiple simple geometries of the same type, as well as **GeometryCollection** for heterogeneous collections mixing different geometry types within a single spatial object.

3.2.2 PostGIS Extension

PostGIS extends PostgreSQL with comprehensive spatial capabilities, transforming it into a powerful spatial database management system [?]. The extension provides dual geometric representations through **Geometry** types for planar 2D projected coordinates and **Geography** types for spherical geodetic coordinates, allowing applications to choose the most appropriate representation based on their spatial accuracy requirements and computational needs. To enable efficient spatial query performance, PostGIS implements specialized **Spatial Indexing** structures including GiST (Generalized Search Tree) and SP-GiST (Space-Partitioned Generalized Search Tree) indexes that dramatically reduce

query execution time for spatial operations. The system supports over 6000 **Coordinate Reference Systems (CRS)** with automatic coordinate transformations between different spatial reference systems, essential for integrating spatial data from diverse sources with varying geographic projections. Additionally, PostGIS provides comprehensive **Raster Support** capabilities enabling storage and analysis of raster data such as digital elevation models and land cover classifications, facilitating integrated raster-vector spatial analysis workflows.

3.2.3 Spatial Operations

PostGIS provides an extensive library of spatial functions organized into four functional categories enabling sophisticated geometric analysis. Spatial relationship predicates form the foundation of spatial queries through functions determining geometric interactions: **ST_Intersects** tests for shared space between geometries, **ST_Contains** evaluates complete containment, **ST_Within** checks inverse containment, and **ST_DWithin** verifies proximity within distance thresholds. Spatial measurement functions enable quantitative analysis including **ST_Area** for polygon area calculation, **ST_Distance** for minimum distance determination, **ST_Length** for linear feature measurement, and **ST_Perimeter** for boundary length computation. Spatial processing functions facilitate geometric transformations through **ST_Buffer** for buffer zone creation, **ST_Intersection** for geometric overlap computation, **ST_Union** for geometry merging, and **ST_Centroid** for geometric center calculation. Raster-vector integration capabilities enable seamless cross-model analysis through **ST_Value** for raster sampling at point locations, **ST_Clip** for raster clipping to polygon boundaries, and **ST_AsRaster** for vector-to-raster conversion.

3.2.4 Query Example

A typical spatial query in the CIM domain:

```

1 SELECT
2     b.building_id,
3     b.building_name,
4     ST_Area(b.building_geometry) as footprint_area,
5     c.population_density,
6     ST_Distance(b.building_geometry, s.station_geometry) as
       distance_to_station
7 FROM
8     cim_vector.buildings b
9     INNER JOIN cim_census.census_tracts c
10         ON ST_Intersects(b.building_geometry, c.tract_geometry)
11     CROSS JOIN LATERAL (
12         SELECT station_geometry
13         FROM cim_vector.subway_stations
14         ORDER BY b.building_geometry <-> station_geometry
15         LIMIT 1
16     ) s
17 WHERE
18     ST_Area(b.building_geometry) > 100
19     AND c.population_density > 5000;
```

Listing 3.1. PostGIS spatial join with measurement

This query demonstrates spatial joins (`ST_Intersects`), measurements (`ST_Area`, `ST_Distance`), and spatial indexing (KNN operator `<->`).

3.3 3DCityDB Extension and CityJSON

3DCityDB is a spatial database management system for 3D city modeling and simulation, based on the Open Geospatial Consortium (OGC) CityGML standard [?]. The system provides comprehensive spatial operations and analytical capabilities for 3D city modeling, enabling storage, query, and analysis of complex urban geometries and semantic information.

3.3.1 CityGML Schema Structure

CityGML defines a hierarchical schema for representing city objects at multiple levels of detail (LoD 0-4), organized into thematic modules. The Core Module provides base classes for all city objects including `CityObject`, `CityModel`, and geometric primitives. Domain-specific modules cover urban features comprehensively: the Building Module represents structures from footprints (LoD0) to detailed interiors (LoD4), the Transportation Module models road networks and railways, the Vegetation Module captures trees and green spaces, and the Water Body Module represents aquatic features. Additional modules include Land Use for zoning classifications, Relief for terrain and elevation models, and City Furniture for street equipment and urban amenities.

3.3.2 Application Domain Extensions (ADEs)

CityGML supports Application Domain Extensions (ADEs) that extend the base schema with domain-specific attributes and relationships. Two critical ADEs for urban energy modeling are:

Energy ADE The Energy ADE [?] extends building objects with comprehensive energy-related properties essential for urban energy modeling. Thermal properties include U-values for building envelope components (walls, roofs, floors, windows) along with thermal transmittance and heat capacity parameters. Construction elements are detailed through material layer specifications, insulation properties, and glazing characteristics enabling accurate thermal simulation. Energy systems encompass heating, cooling, and ventilation equipment with associated efficiency ratings and operational parameters. Historical energy consumption data, energy certificates, and performance indicators support validation and calibration. Renewable energy installations including solar panels, heat pumps, and district heating connections enable scenario analysis for sustainable energy transitions.

Utility Network ADE The Utility Network ADE extends CityGML with comprehensive infrastructure network modeling capabilities. Electrical networks are represented through grid topology including buses, lines, transformers, and load connections enabling power flow analysis. District heating systems are modeled with pipes, heat exchangers, substations, and building connections for thermal network simulation. Water networks

capture both supply and wastewater infrastructure, while gas networks represent distribution pipelines and connections. Network topology representation includes connectivity relationships, flow directions, and capacity constraints essential for infrastructure planning and operational optimization.

3.3.3 CityJSON Format

CityJSON is a JSON-based encoding of CityGML that provides a lightweight, web-friendly representation of 3D city models [?]. Unlike XML-based CityGML, CityJSON offers:

- **Compact Representation:** Reduced file sizes through geometry compression and shared vertex arrays
- **Web Compatibility:** Native JSON format suitable for REST APIs and web applications
- **Simplified Parsing:** Easier integration with JavaScript and Python ecosystems
- **Extension Support:** Compatible with Energy ADE and Utility Network ADE through extensions

3DCityDB and CityJSON are complementary technologies: 3DCityDB provides the database backend for persistent storage and spatial queries, while CityJSON serves as the exchange format for web services and client applications. Together, they enable comprehensive 3D city modeling with energy and infrastructure extensions crucial for urban energy simulation.

3.4 TABULA: From CIM to Urban Energy Modeling

The TABULA project [?] provides a comprehensive building typology database for European countries, enabling standardized energy performance assessment across diverse building stocks. TABULA archetypes classify buildings based on construction period, building type, and energy performance characteristics, serving as a bridge between City Information Models (CIM) and Urban Energy Modeling (UEM) workflows.

3.4.1 Building Archetype Classification

TABULA archetypes are organized hierarchically through four classification dimensions: national context (country-specific building stock characteristics), building typology (single-family, multi-family, apartment blocks, commercial), construction period (historical epochs reflecting evolving construction standards from pre-1945 through modern periods), and energy performance class (original state, moderate renovation, deep renovation). Each archetype provides standardized thermal properties including U-values for building envelope components (walls, roofs, floors, windows), thermal bridge characteristics, air tightness parameters, and typical energy system configurations, enabling consistent energy performance assessment across European building stocks.

3.4.2 Role in Energy Simulation

The TABULA-based enrichment process enables energy simulation through five critical functions. Standardized input provision supplies validated U-values to simulation tools (EnergyPlus, Modelica, TRNSYS) eliminating manual parameterization. Scalability is achieved through automated archetype assignment across hundreds or thousands of buildings where manual parameterization would be infeasible. Validation is ensured as TABULA archetypes derive from empirical building stock surveys reflecting actual construction practices rather than idealized assumptions. Scenario analysis capability enables exploration of renovation alternatives (moderate versus deep renovation) through energy performance class switching. Interoperability through CityGML Energy ADE format ensures compatibility with multiple simulation platforms, enabling seamless data exchange between CIM Wizard and downstream energy simulation tools. This integration demonstrates how CIM Wizard’s calculator architecture transforms sparse geospatial data into simulation-ready models through systematic feature calculation, archetype assignment, and standardized representation.

3.5 Large Language Models

3.5.1 Transformer Architecture

Large Language Models (LLMs) are based on the transformer architecture [?] introduced by Vaswani et al. (2017), featuring five key components that enable effective sequence modeling. Self-attention mechanisms enable parallel processing of sequential data by computing attention weights between all token pairs simultaneously. Multi-head attention allows multiple attention mechanisms to learn different representation subspaces, capturing diverse semantic relationships. Positional encoding injects sequence order information through sinusoidal or learned embeddings, compensating for the architecture’s lack of inherent sequential processing. Position-wise feed-forward networks follow attention layers, providing nonlinear transformations. Layer normalization and residual connections stabilize training of deep networks, enabling effective gradient flow through dozens of transformer layers.

3.5.2 Decoder-Only Language Models

Modern LLMs (GPT, Llama, Qwen) use decoder-only transformer architecture:

$$P(x_{t+1}|x_1, x_2, \dots, x_t) = \text{softmax}(\mathbf{W}_o \cdot \text{Transformer}(x_1, \dots, x_t)) \quad (3.1)$$

Where the model autoregressively predicts the next token based on previous context. This architecture enables three key capabilities: causal attention where each token attends only to previous positions preventing information leakage from future tokens, generative capability through sequential sampling producing coherent natural text, and in-context learning allowing task adaptation through prompting without requiring parameter updates.

3.5.3 Model Families

This work utilizes several open-source model families:

Llama (Meta AI)

- **Llama 2** [?]: Foundation models with improved safety and instruction following
- **Llama 3** [?]: Latest generation with 8B, 14B, and larger parameter variants
- **Architecture**: Grouped Query Attention (GQA) for efficiency
- **Context Length**: 128K tokens (8K effective for fine-tuning)
- **Training Data**: Multilingual web corpus, code, mathematics

Qwen (Alibaba)

- **Qwen 2** [?]: Technical foundation for code and reasoning tasks
- **Qwen 2.5**: 14B and 32B parameter variants
- **Specialization**: Strong code understanding and generation
- **Context Length**: 32K tokens
- **Advantage**: Superior performance on structured generation tasks

Specialized Models

- **SQLCoder (Defog AI)**: 7B model pre-trained on Spider and BIRD benchmarks
- **DeepSeek-Coder**: 6.7B model specialized for code generation
- **Mistral**: 7B parameter model with sliding window attention

3.6 LLM Engineering for Domain-Specific Services

Building production-ready natural language interfaces for specialized domains requires careful consideration of two complementary approaches: agentic engineering using frontier models with external knowledge retrieval, and fine-tuning local models to internalize domain-specific knowledge. This section examines both strategies and their trade-offs for creating assistive services.

3.6.1 Two-Pass Architecture for Service Assistants

Creating effective LLM-powered assistants for domain-specific services typically involves two complementary passes that address different stages of system development and deployment. The first pass employs agentic engineering with LangChain, leveraging large frontier models such as GPT-4, Claude, and Gemini integrated with external tools. This approach utilizes LangChain and LangGraph frameworks to provide orchestration for tool-augmented reasoning, enabling systems to dynamically retrieve database schemas, API documentation, and domain knowledge as retrievable context. The sophisticated reasoning capabilities of large models with parameters exceeding one trillion enable complex planning and decision-making processes. A key advantage of this approach lies in its dynamic adaptation capabilities, allowing systems to handle schema changes and incorporate new tools without requiring retraining. However, this flexibility comes with inherent limitations including API costs that scale with usage, network latency that impacts response times, privacy concerns arising from data transmission to external services, and dependency on external service availability that introduces operational risks.

The second pass focuses on fine-tuning local models to internalize domain knowledge directly into smaller, deployable models. This approach transforms domain-specific patterns into learned model weights, enabling specialized performance without external dependencies. Local deployment ensures that sensitive data never leaves organizational infrastructure, addressing critical privacy requirements for applications handling confidential information. The knowledge internalization process enables specialized models to achieve higher accuracy on domain-specific tasks compared to general-purpose models operating with external context. From an economic perspective, fine-tuning offers a fundamentally different cost structure: a one-time training investment replaces the per-query API fees characteristic of agentic approaches. Additionally, local inference eliminates network round-trips, resulting in lower latency that enhances user experience and enables real-time applications where response time is critical.

3.6.2 Architectural Choice: RAG vs Fine-Tuning

For domain-specific services like spatial SQL generation, two primary architectural approaches exist, each offering distinct advantages and trade-offs that must be carefully evaluated based on deployment requirements, cost constraints, and performance objectives.

Approach 1: Multi-Agent RAG with Frontier Models

The multi-agent RAG approach employs large frontier models with parameters exceeding one trillion, combined with Retrieval-Augmented Generation to dynamically access domain-specific knowledge. This architecture stores database schemas, table structures, and column definitions in vector databases that are retrieved dynamically for each query, enabling systems to access current schema information without requiring model retraining. Task-specific knowledge including PostGIS spatial function documentation, SQL examples, and query patterns are similarly stored as retrievable context, allowing the

system to incorporate the latest documentation and examples dynamically. The reasoning engine consists of large frontier models such as GPT-4 or Claude 3.5, which provide sophisticated reasoning capabilities about schema relationships and spatial operations that enable complex query generation. The multi-agent architecture distributes responsibilities across specialized agents: schema inspection agents analyze database structures, query planning agents determine optimal query strategies, SQL generation agents construct queries, and validation agents ensure correctness before execution.

The advantages of this approach center on its flexibility and adaptability. Systems can handle schema changes without retraining, making them suitable for environments where database structures evolve frequently. The approach leverages cutting-edge reasoning capabilities available in frontier models, enabling sophisticated problem-solving that may exceed the capabilities of smaller fine-tuned models. The dynamic knowledge incorporation allows systems to access the latest documentation and examples, ensuring that the system benefits from continuously updated information sources. However, these advantages come with significant disadvantages. High per-query costs create economic challenges for high-volume deployments, with costs accumulating linearly with usage. Privacy concerns arise from transmitting potentially sensitive queries to external APIs, making this approach unsuitable for applications with strict data confidentiality requirements. Network latency introduces delays of several seconds per query, impacting user experience and limiting applicability to real-time applications. Ongoing operational costs accumulate continuously, creating long-term economic pressure. Additionally, the large model inference required for each query generates substantial CO₂ footprints through cloud infrastructure operations and network transmission.

Approach 2: Fine-Tuned Local Models

The fine-tuned local model approach internalizes domain knowledge directly into smaller models ranging from 8 to 32 billion parameters, typically using models such as Llama or Qwen fine-tuned with QLoRA techniques. This internalization process occurs during training, where database schema patterns, table relationships, and column semantics are learned and encoded into model weights. Similarly, task-specific knowledge including PostGIS spatial functions, SQL generation patterns, and query structures become part of the model's learned representations, eliminating the need for external knowledge retrieval during inference.

The advantages of fine-tuning local models span multiple dimensions critical for production deployment. Privacy protection is complete, as local deployment ensures that sensitive data never leaves organizational infrastructure, making this approach essential for applications handling confidential urban energy data or demographic information. The cost structure fundamentally differs from RAG approaches: an initial one-time training investment replaces ongoing per-query API fees, making fine-tuning economically advantageous for sustained production deployments. Ongoing operational costs remain minimal, limited to compute infrastructure maintenance rather than accumulating per-query charges. Latency is substantially reduced through local inference that eliminates network round-trips, enabling response times suitable for interactive applications. The environmental impact is lower due to smaller model sizes, local deployment eliminating network

transmission overhead, and one-time training energy investment that amortizes over the model’s operational lifetime. Accuracy on domain-specific tasks exceeds that of general models operating with external context, reflecting the value of specialized knowledge encoding. System robustness benefits from consistent performance without dependency on external services, eliminating concerns about API rate limits, service outages, or network disruptions.

The disadvantages of fine-tuning reflect the trade-offs inherent in knowledge internalization. Schema changes require model retraining, making this approach less suitable for environments with frequently evolving database structures. Initial training requires substantial time investment, typically ranging from tens to hundreds of hours depending on model size and dataset complexity. The approach offers less flexibility than RAG for rapidly changing knowledge, as updating the model requires retraining rather than simply updating external knowledge sources. Despite these limitations, fine-tuning provides superior value for applications with stable schemas, privacy requirements, and sustained usage patterns where the upfront investment is justified by long-term operational benefits.

3.6.3 Cost-Benefit Analysis

The economic and environmental trade-offs between RAG-based and fine-tuning approaches reveal fundamental differences in cost structure, operational efficiency, and long-term sustainability that significantly impact deployment decisions for domain-specific natural language interfaces.

From an initial investment perspective, RAG approaches appear attractive due to their minimal upfront costs, requiring only vector database setup and integration with existing API services. However, this apparent advantage masks the substantial ongoing operational expenses that accumulate with usage. In contrast, fine-tuning approaches require an initial training investment, though this cost can be substantially reduced or eliminated entirely when academic GPU access is available. The one-time nature of this training investment fundamentally differs from the recurring cost structure of API-based systems.

The operational cost structure reveals the most significant economic distinction between these approaches. RAG systems incur per-query API fees that scale linearly with usage, creating substantial monthly operational expenses for high-volume deployments. These costs accumulate continuously, making RAG approaches economically challenging for production systems with consistent query volumes. Fine-tuning approaches, conversely, shift costs to the initial training phase, with ongoing operational expenses limited to compute infrastructure maintenance rather than per-query fees. This cost structure transformation enables substantial annual savings for fine-tuning approaches, particularly as query volume increases, making fine-tuning economically advantageous for sustained production deployments.

Environmental considerations further differentiate these approaches. RAG systems generate higher carbon footprints through multiple mechanisms: large model inference requires substantial computational resources, network transmission adds energy overhead for each query, and cloud infrastructure operations contribute additional emissions. Fine-tuning approaches achieve lower environmental impact through several factors: smaller

models reduce inference energy requirements, local deployment eliminates network transmission overhead, and the one-time training energy investment contrasts favorably with the ongoing energy consumption of continuous API inference. The training phase represents a fixed energy cost that amortizes over the model’s operational lifetime, whereas RAG approaches generate continuous energy consumption that accumulates indefinitely.

Performance characteristics also distinguish these approaches, though the relationship between cost and accuracy is complex. RAG systems using general models with schema context achieve moderate accuracy levels, reflecting the limitations of external knowledge retrieval and general-purpose reasoning. Fine-tuning approaches demonstrate superior accuracy through domain-specific knowledge internalization, with improvements that reflect the depth of domain adaptation achieved during training. The performance gap between these approaches highlights the value of specialized knowledge encoding, though the magnitude of improvement varies based on training data quality, model selection, and domain complexity.

3.6.4 Recommendation for CIM Assist

For CIM Assist, fine-tuning local models provides superior value across multiple dimensions that align with the specific requirements of urban energy modeling applications. Privacy considerations are paramount, as urban energy data contains sensitive building and demographic information that requires local processing to maintain confidentiality and comply with data protection regulations. The cost structure of fine-tuning proves economically advantageous, with one-time training investments contrasting favorably against ongoing API fees that accumulate with usage volume. Performance characteristics demonstrate clear superiority, with domain-adapted models achieving substantially higher accuracy compared to general models operating with schema context alone. The stability of the CIM Wizard database schema further supports the fine-tuning approach, as relatively infrequent schema changes reduce the need for retraining, making the upfront investment more justifiable. Finally, local deployment ensures production readiness through improved reliability, reduced latency, and independence from external services that could introduce operational risks.

However, RAG approaches remain valuable for specific scenarios where their flexibility provides advantages. Rapid prototyping before fine-tuning benefits from the immediate deployment capability of RAG systems, enabling quick validation of system requirements before committing to training investments. Handling frequently changing schemas represents another scenario where RAG’s dynamic knowledge retrieval proves advantageous, as systems can adapt to schema modifications without retraining. Incorporating latest documentation dynamically allows RAG systems to leverage up-to-date information that may not be reflected in training datasets. Multi-database scenarios where fine-tuning each database would be impractical benefit from RAG’s ability to operate across diverse schemas without requiring separate training for each database.

This work demonstrates that fine-tuning local models in the 8-32B parameter range achieves production-ready performance with superior privacy, cost efficiency, and environmental impact compared to RAG-based approaches using frontier models. The combination of domain-specific knowledge internalization, local deployment capabilities, and

favorable economic characteristics makes fine-tuning the recommended approach for CIM Assist, while recognizing that RAG systems serve important roles in specific deployment scenarios.

3.7 Parameter-Efficient Fine-Tuning

3.7.1 Full Fine-Tuning Limitations

Traditional fine-tuning approaches that update all model parameters face several prohibitive resource constraints. A 14B parameter model requires approximately 56GB of GPU memory when training in FP16 precision, placing it beyond the reach of most consumer and research-grade hardware. Scaling laws [?] demonstrate that model performance improves predictably with parameter count, but full fine-tuning of large models remains computationally expensive. Alternative approaches like knowledge distillation [?] can transfer knowledge from large models to smaller ones, but parameter-efficient methods offer more direct adaptation strategies. The computational expense compounds this challenge, as full fine-tuning necessitates complete backward passes through all layers during each training iteration, resulting in slow convergence that demands millions of gradient updates to achieve satisfactory performance. Furthermore, the storage overhead becomes impractical when adapting models to multiple tasks, as each task-specific version requires maintaining a complete model copy, multiplying storage requirements linearly with the number of downstream applications.

3.7.2 Low-Rank Adaptation (LoRA)

LoRA (Hu et al., 2021) introduces trainable rank-decomposition matrices into transformer layers:

$$\mathbf{W}' = \mathbf{W}_0 + \mathbf{B}\mathbf{A} \quad (3.2)$$

Where:

- $\mathbf{W}_0 \in \mathbb{R}^{d \times k}$ is the frozen pre-trained weight matrix
- $\mathbf{B} \in \mathbb{R}^{d \times r}$ and $\mathbf{A} \in \mathbb{R}^{r \times k}$ are trainable low-rank matrices
- $r \ll \min(d, k)$ is the rank (typically 8-16)

Advantages: The LoRA approach [?] provides several critical advantages for parameter-efficient fine-tuning. Most significantly, it reduces trainable parameters by approximately 99%, transforming a 14B parameter model into merely 16M trainable parameters while maintaining comparable performance to full fine-tuning. This dramatic reduction yields substantial memory efficiency, as only the small adapter matrices require gradient computation and storage during training. The modular nature of LoRA enables practical multi-task deployment, allowing multiple task-specific adapters to be swapped dynamically at inference time while sharing a single frozen base model. This architecture preserves the pre-trained knowledge encoded in the base weights, preventing catastrophic forgetting while enabling effective task adaptation.

3.7.3 Quantized LoRA (QLoRA)

QLoRA [?] further reduces memory requirements by quantizing the base model while maintaining LoRA’s parameter-efficient adaptation strategy. The approach introduces **4-bit NormalFloat (NF4) Quantization**, a custom data type specifically optimized for the normally distributed weight values typical in neural networks, providing better precision than standard 4-bit integer quantization. To maximize memory savings, QLoRA employs **Double Quantization**, which quantizes even the quantization constants themselves, achieving an additional reduction in memory footprint. The system leverages **Paged Optimizers** that utilize NVIDIA unified memory architecture to manage optimizer states efficiently across CPU and GPU memory. The combined effect of these innovations enables fine-tuning of 14B parameter models within 18-20GB of memory, making large language model adaptation accessible on consumer-grade 24GB GPUs and democratizing advanced model development for the research community.

QLoRA Configuration Typical configuration for this work:

- **Quantization:** 4-bit NF4 with double quantization
- **LoRA Rank (r):** 8 or 16
- **LoRA Alpha (α):** 16 or 32 (scaling factor = α/r)
- **LoRA Dropout:** 0.05-0.1 for regularization
- **Target Modules:** q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
- **Learning Rate:** 1.5×10^{-4} to 2.0×10^{-4}

3.7.4 Training Process

The fine-tuning process follows:

1. **Load Base Model:** Load pre-trained model in 4-bit quantized format
2. **Initialize LoRA Adapters:** Add trainable low-rank matrices to target layers
3. **Prepare Dataset:** Format training samples with chat templates
4. **Training Loop:** Update only LoRA parameters through gradient descent
5. **Merge and Save:** Optionally merge adapters with base model

3.8 AI Agents and LangGraph

3.8.1 Agent Architecture

AI agents extend language models with tool use and iterative reasoning through a perception-action loop comprising five phases. Perception receives user queries and environmental

state information. Reasoning plans action sequences through model inference determining which tools to invoke. Action executes selected tools including database queries, API calls, and computations. Observation captures tool outputs and error messages providing feedback for subsequent reasoning. Reflection enables iteration, continuing the loop until goals are achieved or maximum step limits are reached.

3.8.2 LangGraph Framework

LangGraph, part of the LangChain ecosystem [?], provides stateful agent orchestration for building complex AI workflows:

Core Concepts LangGraph provides four core abstractions for stateful agent orchestration. State graphs represent workflows as directed graphs where nodes are processing steps and edges are transitions. State schemas employ typed dictionaries maintaining agent state across execution, ensuring type safety and state consistency. Conditional edges enable dynamic routing based on agent decisions, supporting complex branching logic. Persistence checkpoints state for human-in-the-loop workflows, allowing interruption and resumption.

Agent Execution Flow Agent execution proceeds through five phases: input nodes receive user queries and initialize state, agent nodes invoke LLMs to generate actions (tool calls or final answers), tool nodes execute selected tools and update state with results, conditional edges route to agent nodes for iteration or output nodes to finish, and output nodes return final results to users.

3.8.3 Database Tools

Text-to-SQL agents typically integrate four essential database tools: `sql_db_list_tables` for discovering available tables, `sql_db_schema` for retrieving table schemas, `sql_db_query_checker` for validating SQL syntax before execution, and `sql_db_query` for executing queries and returning results.

```
1 from langchain.tools import BaseTool
2
3 class SpatialSchemaInspectorTool(BaseTool):
4     name = "sql_db_schema"
5     description = "Get table schemas with geometry column detection"
6
7     def _run(self, table_names: str) -> str:
8         tables = table_names.split(',')
9         schema_info = []
10
11         for table in tables:
12             # Get standard columns
13             columns = db.get_table_columns(table)
14
15             # Detect geometry columns
```



```
16         geom_cols = [col for col in columns
17                       if 'geometry' in col.lower() or 'geom' in col.
18                          lower()]
19
20         schema_info.append(f"Table: {table}\n"
21                            f"Columns: {'', '.join(columns)}\n"
22                            f"Spatial Columns: {'', '.join(geom_cols)}")
23
24     return '\n\n'.join(schema_info)
```

Listing 3.2. Database schema tool with spatial awareness

3.8.4 Error Handling and Retry Logic

Robust agent implementations incorporate four safety mechanisms: maximum iteration limits prevent infinite loops (typically 5 attempts), timeout management enforces query execution limits (typically 120 seconds), error feedback passes execution errors back to agents enabling self-correction, and graceful degradation returns partial results when complete answers fail, ensuring user-facing robustness.

3.9 Summary

This chapter introduced the foundational technologies:

- **PostGIS:** Spatial databases with geometry types, spatial operations, and raster support enabling complex urban analytics
- **Large Language Models:** Transformer-based architectures (Llama, Qwen) providing natural language understanding and generation
- **QLoRA:** Parameter-efficient fine-tuning enabling 14B model training on single 24GB GPU through 4-bit quantization and low-rank adapters
- **LangGraph:** Stateful agent framework enabling iterative reasoning with database tools and execution feedback
- **Urban Informatics:** Digital twin technologies, BIM standards, and smart city platforms providing context for integrated urban energy systems

These technologies form the technical foundation for CIM Wizard and CIM Assist, which are positioned in the context of related work in Chapter ?? and detailed in the methodology in Chapter ??.

Chapter 4

Methodology

This chapter presents the comprehensive methodology for building an LLM-powered natural language interface to spatial SQL databases, structured around two integrated systems: CIM Wizard Integrated (the object-oriented platform) and CIM Assist (the natural language interface with three-stage dataset generation, fine-tuning, and agent integration).

4.1 System Overview

The complete framework integrates five interconnected components that together enable natural language access to spatial databases for urban energy modeling. The foundation is provided by CIM Wizard Integrated, an object-oriented FastAPI platform implementing a calculator-based architecture that enables flexible city information modeling through extensible feature computation methods. This platform serves as both the data source and the target database for the natural language interface, providing the spatial SQL queries that the system must learn to generate.

The dataset generation pipeline, designated as AI4DB, employs a two-stage approach that transforms structured SQL templates into diverse training samples through systematic template expansion followed by multi-strategy LLM augmentation. This process generates high-quality question-SQL pairs with frequency-based distribution matching actual CIM Wizard usage patterns, ensuring that the training data reflects real-world query characteristics rather than artificial balance.

The fine-tuning pipeline, labeled TXT2SSQL, trains specialized models using a direct question-to-SQL (Q2SQL) architecture that learns to generate PostGIS spatial queries directly from natural language questions. This single-stage training approach balances performance with computational efficiency, enabling domain adaptation on consumer-grade hardware through parameter-efficient fine-tuning techniques.

The evaluation framework provides comprehensive multi-metric assessment through seven complementary metrics (NoErr, EM, Deep EM, EX, SC, EA, plus robustness metrics) that measure different aspects of model performance from dataset quality validation to production readiness. This systematic evaluation approach enables detailed understanding of model behavior across different query types, complexity levels, and domain

patterns.

Finally, the agent integration component, CIM Assist, deploys the fine-tuned models through a LangGraph-based agent architecture with database introspection tools, enabling iterative self-correction through execution feedback. This production-ready service transforms the fine-tuned models into an accessible natural language interface that bridges the gap between non-technical energy researchers and complex spatial database queries.

Figure ?? illustrates the complete architecture.

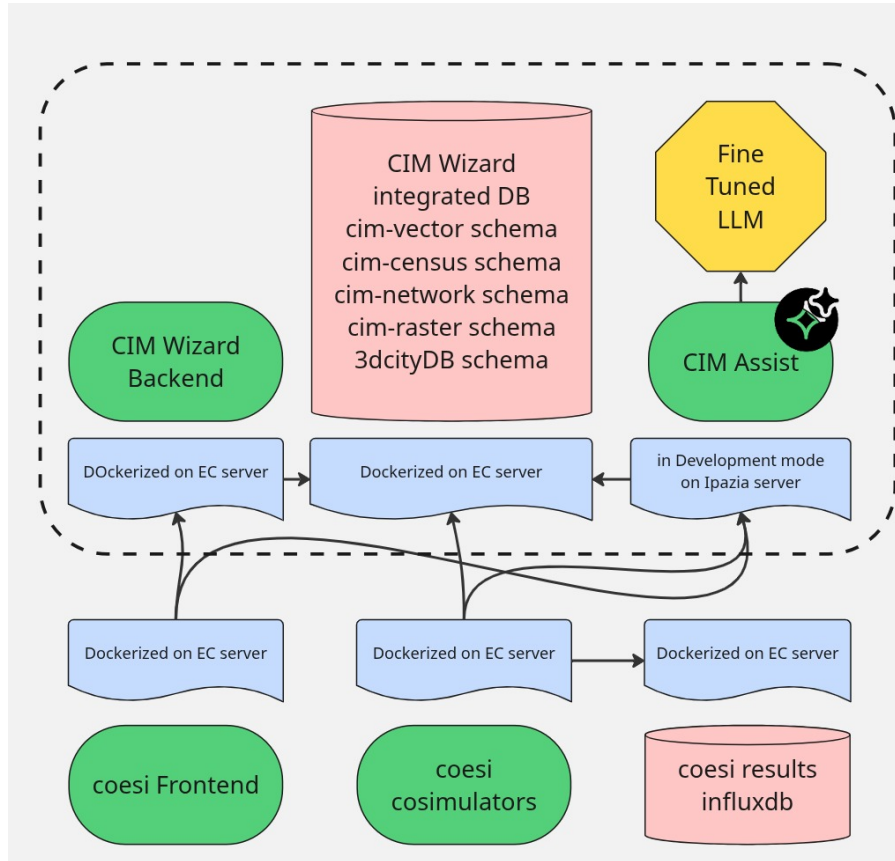


Figure 4.1. CIM Wizard and CIM Assist position in COESI system architecture

Figure ?? illustrates the complete architecture, showing how these five components integrate to form a cohesive system that transforms expert-oriented database access into an accessible research tool for urban energy modeling.

4.2 CIM Wizard: Object-Oriented Platform

CIM Wizard Integrated provides the foundational platform enabling flexible city information modeling through an object-oriented calculator-based architecture. The framework generates GIS-based city information models from sparse and heterogeneous urban datasets, producing outputs compliant with CityJSON extended with Energy and Utility ADEs. The system is designed for maximum flexibility and extensibility, implementing a microservice architecture [?] that enables modular development and deployment.

4.2.1 Design Principles and Motivation

The platform adheres to four core principles addressing key limitations identified in existing urban energy modeling frameworks. First, extensibility enables researchers to add new calculators and methods without modifying core code, facilitating custom building feature implementation. Second, modularity ensures each calculator encapsulates independent feature computation with well-defined interfaces. Third, flexibility supports multiple calculation methods per feature with automatic fallback strategies to handle sparse data conditions. Fourth, seamless integration provides multi-schema database access through a unified API, enabling vector-census-raster data fusion.

The architecture enables dynamic and highly customizable pipeline orchestration for CIM generation. A pipeline is defined as a sequence of feature calculators, which may execute synchronously or asynchronously depending on their input dependencies. Users can construct pipelines manually, chaining feature-method pairs to reflect specific project needs, or rely on predefined milestones that encapsulate common processing stages. Independent calculators execute in parallel, while dependent calculators are scheduled sequentially based on automatic dependency resolution.

4.2.2 Calculator Architecture

Calculator Base Class

All calculators inherit from a base class defining the interface:

```
1 class BaseCalculator:
2     def __init__(self, building_id, db_session):
3         self.building_id = building_id
4         self.db = db_session
5         self.methods = self._discover_methods()
6
7     def _discover_methods(self):
8         """Automatically discover calculate_method* functions"""
9         return [m for m in dir(self)
10                 if m.startswith('calculate_method')]
11
12     def execute(self, method_name=None):
13         """Execute specified method or try all in priority order"""
14         if method_name:
15             return self._call_method(method_name)
16
17         # Try all methods until one succeeds
```

```

18     for method in self.methods:
19         try:
20             result = self._call_method(method)
21             if result is not None:
22                 return result
23         except Exception as e:
24             continue # Try next method
25
26     return None # All methods failed

```

Listing 4.1. Calculator base class structure

Example Calculator: Building Height

A concrete calculator implementing multiple calculation methods:

```

1 class BuildingHeightCalculator(BaseCalculator):
2
3     def calculate_method1_raster(self):
4         """Primary: Calculate from DSM - DTM raster difference"""
5         # Get building geometry
6         building = self.db.query(Building)\
7             .filter_by(id=self.building_id).first()
8
9         # Sample DSM (Digital Surface Model)
10        dsm_value = RasterService.get_value(
11            'cim_raster.dsm',
12            building.building_geometry
13        )
14
15        # Sample DTM (Digital Terrain Model)
16        dtm_value = RasterService.get_value(
17            'cim_raster.dtm',
18            building.building_geometry
19        )
20
21        return dsm_value - dtm_value if both else None
22
23    def calculate_method2_census(self):
24        """Fallback: Estimate from census building age"""
25        building = self.db.query(Building)\
26            .filter_by(id=self.building_id).first()
27
28        # Find intersecting census tract
29        census = self.db.query(CensusTract)\
30            .filter(
31                ST_Intersects(
32                    CensusTract.tract_geometry,
33                    building.building_geometry
34                )
35            ).first()
36
37        # Estimate height from building age
38        if census and census.avg_building_age:
39            # Older buildings typically lower

```

```
40         return estimate_height_from_age(census.avg_building_age)
41
42     return None
43
44     def calculate_method3_default(self):
45         """Last resort: Use default value"""
46         return 10.0 # meters (typical residential)
```

Listing 4.2. Building height calculator with multiple methods

4.2.3 Pipeline Orchestration and Execution

The methodology enables dynamic and highly customizable pipeline orchestration for CIM generation through a sophisticated execution framework. At the core lies a robust fallback mechanism: if a method fails because it cannot compute a feature for all buildings or its inputs are unavailable, the orchestrator automatically invokes lower-priority methods as specified in the configuration catalog. This adaptive strategy enhances resilience to missing data and enables modular, incremental data enrichment.

The system supports three execution modes to accommodate different use cases: (1) **Automatic Mode** where the executor selects the best method based on data availability, (2) **Explicit Mode** where users specify exact method chains for reproducible research, and (3) **Predefined Pipelines** offering named workflows for common analyses. Advanced users may define explicit chains using dedicated syntax (`feature.method | feature.method | ...`), granting fine-grained control over execution logic for algorithm comparison and research prototyping.

4.2.4 Context-Aware Data Management

The core of a pipeline is the context manager that dynamically manages input data, dependencies, and intermediate results across all feature calculators. This context object acts as a shared memory space, receiving and distributing inputs from multiple sources, including user uploads, database records, external services, and default assumptions. Each feature calculator interacts with the context manager to retrieve required inputs and store outputs without explicit coupling to other modules.

If a method requires a feature that has not yet been computed, the orchestrator automatically resolves this by invoking the appropriate upstream calculators in the correct order. This context-based execution facilitates robustness and traceability while allowing runtime introspection, where users can inspect and debug intermediate states, validate assumptions, or inject custom values—particularly useful in urban data environments characterized by heterogeneity, partial completeness, and evolving data availability.

4.2.5 Flexible Data Input Options

The framework accommodates both richly detailed and minimally specified input datasets. Users may upload building footprints, census data, energy attributes, or utility network layouts using standard formats such as GeoJSON, Shapefiles, pandapower, or tabular CSV with configuration metadata. In cases where only minimal input is available, such

as a boundary polygon defining the study area, the system invokes external data services to retrieve necessary information. OpenStreetMap (OSM) is queried to obtain building geometries, census APIs provide demographic attributes, and elevation services support height estimation. If no data is found, fallback mechanisms populate fields using national defaults or statistical estimates, such as the TABULA archetypes for building stock. This flexible input strategy ensures operation under a wide range of data conditions, making it suitable for both research and operational planning across varied urban contexts.

4.2.6 Multi-Schema Database Design

CIM Wizard integrates three PostGIS schemas optimized for urban energy analysis:

Vector Schema (`cim_vector`)

The vector schema contains geometric entities essential for spatial urban analysis. Building footprints are stored as polygon geometries with associated properties and project/scenario linkages. Research projects are defined with their spatial extents and metadata. Energy scenarios (baseline, optimized, and future configurations) enable comparative analysis. Electrical infrastructure is represented through grid buses marking connection points and grid lines defining distribution networks.

Census Schema (`cim_census`)

The census schema integrates Italian ISTAT demographic data through census tract boundaries with over 100 attributes. Key fields include population counts, housing units, building age distributions, and household size statistics. Spatial representation uses MultiPolygon geometries to capture complex tract boundaries. A notable technical challenge involves case-sensitive column naming conventions requiring lowercase unquoted identifiers for PostgreSQL compatibility.

Raster Schema (`cim_raster`)

The raster schema stores elevation and surface models essential for height calculations. Digital Terrain Models (DTM) represent ground elevation, while Digital Surface Models (DSM) capture building and vegetation tops. These rasters are stored at 1-5 meter cell resolution, supporting operations including value sampling at specific locations, spatial clipping to study areas, and statistical aggregation.

4.2.7 Pipeline Execution System

Dependency Resolution

The executor automatically determines calculator dependencies:

```
1 class PipelineExecutor:
2     def __init__(self, calculators: List[BaseCalculator]):
3         self.calculators = calculators
4         self.dependency_graph = self._build_dependency_graph()
```



```

5
6     def _build_dependency_graph(self):
7         """Build directed acyclic graph of calculator dependencies"""
8         graph = {}
9         for calc in self.calculators:
10            # Check which other calculators this one requires
11            dependencies = calc.get_dependencies()
12            graph[calc.name] = dependencies
13        return graph
14
15    def execute_pipeline(self, building_id):
16        """Execute calculators in topologically sorted order"""
17        sorted_calcs = topological_sort(self.dependency_graph)
18        results = {}
19
20        for calc_name in sorted_calcs:
21            calculator = self._get_calculator(calc_name)
22            result = calculator.execute()
23            results[calc_name] = result
24
25        return results

```

Listing 4.3. Automatic dependency resolution

Execution Modes

Three modes accommodate different use cases:

1. **Automatic Mode:** Executor selects best method based on data availability

```

1 results = executor.execute_pipeline(building_id)
2

```

2. **Explicit Mode:** User specifies exact method chain

```

1 results = executor.execute_pipeline(
2     building_id,
3     methods={'height': 'calculate_method1_raster',
4              'area': 'calculate_method1_postgis'})
5 )
6

```

3. **Predefined Pipelines:** Named workflows for common analyses

```

1 results = executor.execute_predefined(
2     building_id,
3     pipeline_name='energy_assessment'
4 )
5

```

4.2.8 Framework Architecture and Components

The CIM Wizard architecture is organized into four interconnected component layers that together enable the transformation of sparse urban datasets into standardized City Information Models, as illustrated in Figure ??.

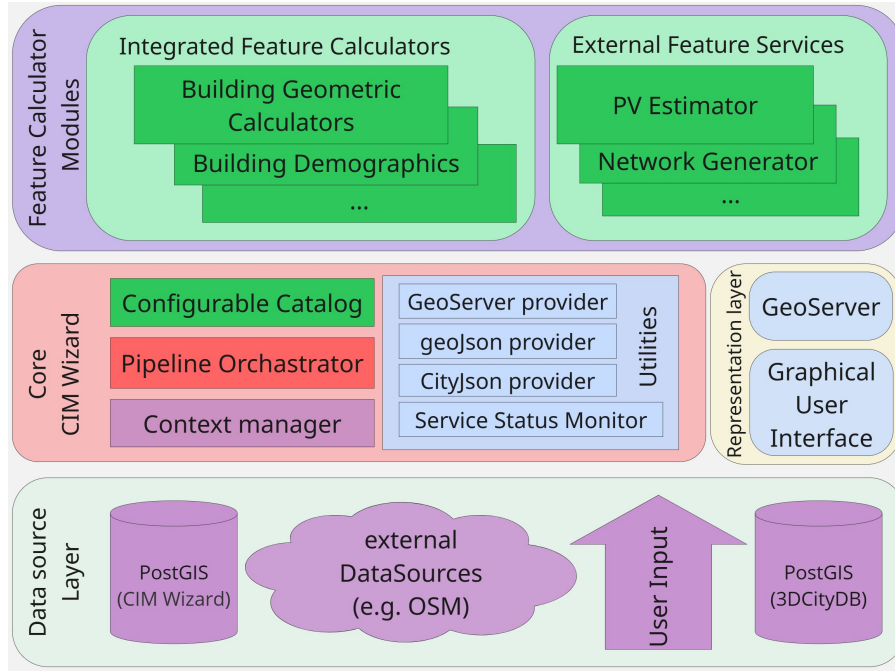


Figure 4.2. CIM Wizard high-level architecture with four interconnected layers: Data Source Layer, Core CIM Wizard, Feature Calculator Modules, and Representation Layer

Data Source Layer

The Data Source Layer comprises four key components: (1) *PostGIS (CIM Wizard)* serves as the central operational database storing both spatial geometries and semantic attributes with normalized structure enabling scenario-based analysis; (2) *External Data Sources* including OpenStreetMap accessed through *osmnx* and Overpass API, regional cadastral databases via WFS, census APIs, DSM elevation services, and TABULA archetype datasets; (3) *User Input* allowing uploads in GeoJSON, Shapefile, or tabular formats with configuration-based schema mapping; and (4) *PostGIS (3DCityDB)* acting as long-term storage extended with Energy ADE and Utility Network ADE for CityJSON-compliant exports.

The database schema follows a normalized structure where geometric entities are maintained separately from their associated descriptive data, organized into modular feature sets. Each enriched record is indexed using a composite key linking it to the geometric object and classification schema consisting of project and scenario identifiers, as shown in Figure ???. This enables the same geometric entity to have different feature data across multiple scenarios, supporting efficient memory usage, rapid scenario switching, and comparative simulation workflows.

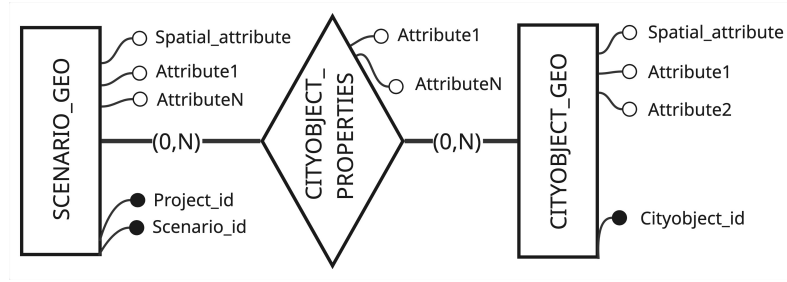


Figure 4.3. Generic entity-relations diagram of CIM Wizard PostGIS database schema enabling project and scenario-based organization

Core CIM Wizard Layer

The Core CIM Wizard layer governs execution logic and data flow through modular components: (1) *Pipeline Orchestrator* dynamically constructs and executes processing workflows based on user-defined objectives and configuration rules, handling dependencies and implementing fallback strategies; (2) *Context Manager* abstracts all database interactions, acting as the central interface for reading inputs and persisting outputs during pipeline execution while enforcing schema consistency; (3) *Configuration Catalog* harmonizes all microservices, storing their endpoints, data source locations, and method prioritization for all integrated feature calculators; and (4) *Utility Block* containing consistency checking, service status monitoring, and format converters including GeoJSON provider, GeoServer publisher, and CityJSON mapper.

Feature Calculator Modules Layer

The Features Calculator Modules layer comprises two complementary categories: *Integrated Feature Calculators* embedded directly within CIM Wizard core and invoked by the pipeline orchestrator, implementing specific building or scenario-level features as independent Python classes; and *External Feature Services* operating as independent web services due to computational complexity, including the Network Generator/Integrator for power grid and district heating networks, and PV Estimator based on [?]. External services operate outside the pipeline orchestrator but link to the CIM Wizard database via HTTP/REST interfaces, acting autonomously and mapping results back into the schema.

Representation Layer

The representation layer consists of GeoServer for serving geospatial data over OGC-compliant protocols (WMS, WFS) and a React-based graphical user interface for end-user interaction. CIM Wizard publishes vector data to GeoServer and provides standardized web services for visualization and data access.

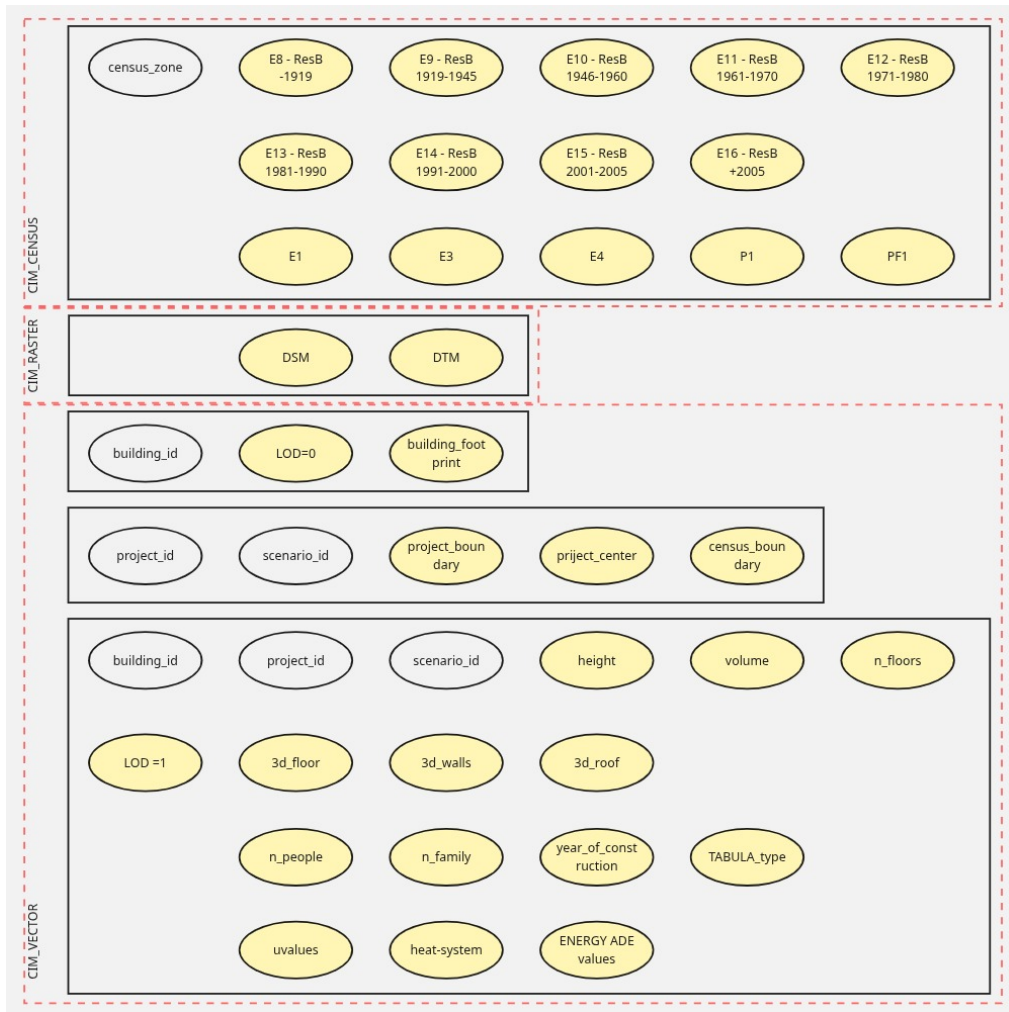


Figure 4.4. CIM Wizard integrated data structure: Multi-schema PostGIS database architecture with vector geometries (cim_vector), census demographics (cim_census), and raster elevation models (cim_raster) supporting comprehensive urban energy analysis

4.2.9 Bridging GeoJSON to CityJSON via TABULA Integration

CIM Wizard’s calculator-based architecture enables systematic assignment of TABULA archetypes to buildings through a multi-step process:

Step 1: Feature Calculation CIM Wizard calculates essential building features across four categories: geometric features (height, floor count, footprint area, volume), demographic features (usage type, family count), temporal features (construction period from census or OSM data), and contextual features (location, urban density, surrounding typologies).

Step 2: Archetype Assignment The TABULA Building Type Calculator classifies each building by matching calculated features against archetype criteria including usage pattern (residential/commercial), size characteristics (floors, families, volume), construction period (temporal classification), and geographic context (country-specific tables).

Step 3: U-Value Assignment CIM Wizard retrieves standardized parameters from TABULA databases including building envelope U-values (wall, roof, floor, window thermal transmittance), construction details (material layers, insulation thickness, glazing properties), energy system parameters (typical heating/cooling systems with efficiency ratings and fuel types), and renewable energy potential (solar orientation, roof area, typical PV/thermal installations).

Step 4: CityGML Energy ADE Storage Retrieved parameters are stored in CityGML Energy ADE format with thermal boundary properties applied to semantic surfaces (`RoofSurface`, `WallSurface`, `GroundSurface`), construction elements stored in the Energy ADE construction module, energy systems linked to building objects, and energy performance indicators (certificates, consumption estimates) enabling simulation tool integration.

TABULA Italy Building Archetypes

The TABULA (Typology Approach for Building Stock Energy Assessment) project provides standardized building archetypes for European countries, enabling consistent thermal characterization of building stocks. For Italy, TABULA defines four primary residential building types across eight construction periods, each with characteristic envelope properties and energy systems that can be directly mapped to 3DCityDB Energy ADE attributes.

Table ?? presents the complete TABULA Italy residential building archetypes with thermal properties. The U-values decrease significantly after 1976 following Italy’s first energy regulation (Law 373/1976) and again after 2006 with the implementation of the Energy Performance of Buildings Directive (D.Lgs. 192/2005). Modern buildings (2006–present) achieve U-values below $0.35 \text{ W}/(\text{m}^2 \cdot \text{K})$ for walls and roofs, representing a 7–10x improvement over pre-1900 construction.

Table 4.1. TABULA Italy residential building archetypes with thermal properties and energy systems mappable to 3DCityDB Energy ADE

Construction Period	U-Values [W/(m ² · K)]				Energy Systems & Parameters			
	Wall	Roof	Floor	Window	Heating	DHW	WWR	η
SFH – Single-Family House								
Before 1900	2.58	2.50	1.88	5.00	Coal stove	Elec. boiler	0.10	0.65
1901–1920	2.40	2.50	1.88	5.00	Coal stove	Elec. boiler	0.12	0.65
1921–1945	1.80	2.50	1.88	4.90	Oil boiler	Gas boiler	0.12	0.72
1946–1960	1.48	2.20	2.00	4.90	Gas boiler	Gas boiler	0.15	0.78
1961–1975	1.30	1.80	1.60	4.90	Gas boiler	Gas boiler	0.18	0.80
1976–1990	0.76	1.14	0.76	2.80	Gas boiler	Gas boiler	0.20	0.85
1991–2005	0.50	0.50	0.60	2.40	Cond. boiler	Cond. boiler	0.22	0.92
2006–Present	0.34	0.28	0.33	2.20	Heat pump	Heat pump	0.25	3.50
TH – Terraced House (Townhouse)								
Before 1900	2.40	2.50	1.88	5.00	Coal stove	Elec. boiler	0.10	0.65
1901–1920	2.20	2.50	1.88	5.00	Coal stove	Elec. boiler	0.12	0.65
1921–1945	1.60	2.30	1.88	4.90	Oil boiler	Gas boiler	0.12	0.72
1946–1960	1.35	1.80	1.70	4.90	Gas boiler	Gas boiler	0.15	0.78
1961–1975	1.15	1.50	1.50	4.90	Gas boiler	Gas boiler	0.18	0.80
1976–1990	0.70	1.00	0.70	2.80	Gas boiler	Gas boiler	0.20	0.85
1991–2005	0.45	0.45	0.55	2.40	Cond. boiler	Cond. boiler	0.22	0.92
2006–Present	0.32	0.26	0.30	2.00	Heat pump	Heat pump	0.25	3.50
MFH – Multi-Family House								
Before 1900	2.30	2.50	1.88	5.00	Coal stove	Elec. boiler	0.12	0.65
1901–1920	2.00	2.30	1.88	5.00	Coal stove	Elec. boiler	0.14	0.65
1921–1945	1.50	2.00	1.88	4.90	Oil boiler	Gas boiler	0.15	0.72
1946–1960	1.20	1.47	1.88	4.90	Gas boiler	Gas boiler	0.18	0.78
1961–1975	1.05	1.42	1.30	4.90	Gas boiler	Gas boiler	0.20	0.80
1976–1990	0.65	0.90	0.65	2.80	Gas boiler	Gas boiler	0.22	0.85
1991–2005	0.42	0.40	0.50	2.40	Cond. boiler	Cond. boiler	0.24	0.92
2006–Present	0.30	0.25	0.28	1.80	Heat pump	Heat pump	0.28	3.50
AB – Apartment Block								
Before 1900	2.20	2.50	1.88	5.00	Coal stove	Elec. boiler	0.14	0.65
1901–1920	1.90	2.20	1.88	5.00	Coal stove	Elec. boiler	0.16	0.65
1921–1945	1.40	1.80	1.88	4.90	Oil boiler	Centr. gas	0.18	0.72
1946–1960	1.15	1.45	1.88	4.90	Centr. gas	Centr. gas	0.20	0.78
1961–1975	1.10	1.42	1.30	4.90	Centr. gas	Centr. gas	0.22	0.80
1976–1990	0.80	1.00	0.70	2.80	Centr. gas	Centr. gas	0.24	0.85
1991–2005	0.40	0.30	0.45	2.40	Cond. boiler	Cond. boiler	0.26	0.92
2006–Present	0.28	0.22	0.26	1.60	Heat pump	Heat pump	0.30	3.50

Legend: DHW = Domestic Hot Water; WWR = Window-to-Wall Ratio; η = System Efficiency (COP for heat pumps).

Cond. = Condensing; Centr. = Centralized. Values derived from TABULA WebTool [?] and EPISCOPE project.

The building type classification follows Italian residential building conventions: Single-Family Houses (SFH) are detached buildings with 1–2 floors typically housing one family; Terraced Houses (TH) share walls with adjacent units in row configurations; Multi-Family Houses (MFH) contain 3–6 units across 2–4 floors; and Apartment Blocks (AB) are larger structures with more than 6 units and typically 4+ floors, often featuring centralized heating systems.

TABULA to Energy ADE Attribute Mapping

Table ?? defines the systematic mapping between TABULA archetype parameters and 3DCityDB Energy ADE schema elements. This mapping enables automated population of Energy ADE attributes when CIM Wizard assigns TABULA archetypes to buildings, ensuring semantic interoperability with CityGML-compliant tools and simulation platforms.

Table 4.2. TABULA parameter to 3DCityDB Energy ADE attribute mapping

TABULA Parameter	Energy ADE Class	Energy ADE Attribute
<i>Thermal Envelope Properties</i>		
Wall U-Value	ThermalComponent	uValue
Roof U-Value	ThermalComponent	uValue
Floor U-Value	ThermalComponent	uValue
Window U-Value	ThermalOpening	uValue
Window-to-Wall Ratio	ThermalOpening	openableRatio
Wall/Roof/Floor Area	ThermalBoundary	area
<i>Construction Properties</i>		
Construction Materials	Construction	constructionWeight
Insulation Thickness	Layer	thickness
Material Conductivity	Material	conductivity
<i>Energy System Properties</i>		
Heating System Type	EnergyConversionSystem	conversionSystemType
Heating Efficiency	EnergyConversionSystem	efficiencyIndicator
DHW System Type	EnergyConversionSystem	conversionSystemType
Fuel Type	EnergyConversionSystem	energyCarrier
Nominal Power	EnergyConversionSystem	nominalPower
Distribution Losses	EnergyDistributionSystem	distributionPerimeter
<i>Building Properties</i>		
Building Age Class	Building	yearOfConstruction
Refurbishment State	Building	refurbishmentClass
Building Type	Building	buildingType

The integration enables CIM Wizard to transform sparse GeoJSON building footprints into semantically rich CityJSON models with complete thermal characterization. When a building’s construction period and type are determined (from census data or OSM tags), the corresponding TABULA archetype provides all necessary parameters for

Energy ADE population, enabling downstream energy simulation without manual data entry.

4.2.10 Detailed Calculator Implementations

The framework implements comprehensive calculators covering geometric, demographic, and energy features, each with multiple fallback methods ensuring robustness to sparse data:

Geometric Feature Calculators

Scenario Geometry Calculator initiates each project by defining the spatial context. If a user-defined project boundary is provided, the system processes it to extract a unique project boundary, determine its centroid, and set metadata including coordinate reference system (CRS), zoom level, and assign unique project and scenario identifiers. In the absence of input, the system derives the project boundary algorithmically by applying a convex hull operation to available building footprints, ensuring a minimal and accurate spatial boundary.

Building Geometry Calculator validates or acquires building footprint layers. When no initial building data is supplied, the system queries OpenStreetMap using both `osmnx` library and Overpass API to retrieve buildings within the project boundary. It integrates an intelligent classification layer that preserves European mixed-use patterns (e.g., residential units above retail shops) while filtering out buildings not meeting geometric criteria (e.g., height < 8 m or area < 100 m²). User-uploaded GeoJSON layers are normalized, assigned unique identifiers, and geometry-validated.

Building Height Calculator implements a three-tiered fallback system: (1) Primary method queries raster elevation services to compute height as DSM minus DTM difference, batching requests in groups of 100 with coordinate transformation and error management; (2) Fallback extracts height values from OSM tags using regex-based parsing to handle inconsistent formats (meters, feet, free-text); (3) Default assumes 12 meters reflecting a typical 4-floor building with 3.0 m/floor heuristic.

Building Area Calculator projects geometries from geographic coordinates (WGS84) to local projected CRS for accurate area computation, handling both Polygon and MultiPolygon types with fallback approximations if projections are infeasible.

Building Volume Calculator computes volumetric space via height $\times$ area product with atomic transaction handling. Number of floors is estimated applying height-based heuristic (1 floor per 3 meters), rounded to nearest integer—an approximate but rapid method suitable for energy simulation archetype assignment.

Demographic Feature Calculators

Scenario Census Boundary Calculator queries remote census APIs using the project's spatial extent, retrieves all intersecting census zones, applies geometric union with buffered boundaries to avoid over-dilution, and stores relevant demographic variables including total population, family count per type, and residential building construction epochs.

Building Usage Type Calculator filters buildings through two-step process: (1) removes explicitly non-residential types using OSM tags (schools, hospitals) but retains mixed-use buildings when residential use is inferred; (2) applies geometric filters to exclude structures unlikely to house residents based on size and height.

Building Population Calculator allocates population across residential buildings proportionally to their volume, approximating population density per building. Family counts are inferred using division by average family size (e.g., 3 people), producing realistic integer values.

Building Demographic Calculator orchestrates census acquisition, building geometry queries, height and volume computation, usage classification, and demographic allocation. It includes zone-level accuracy reporting and random assignment of construction year using weighted distributions from census periods, enabling buildings to be tagged with plausible construction epochs.

Energy Feature Calculators

Building Geo LoD 1.2 Calculator constructs semantic surfaces including `RoofSurface`, `WallSurface`, and `GroundSurface` by extruding 2D building footprints according to calculated heights. Each surface is enriched with geometric metadata (orientation, area, azimuth angle), resulting in LoD 1.2-compliant CityJSON geometry structure.

TABULA Building Type Calculator classifies each building into representative typology based on number of floors, family count, and usage pattern. Classification is mapped to city-specific construction table derived from TABULA and local building physics references, providing U-values, R-values, construction materials, and refurbishment class identifiers.

Thermal Boundary Calculator applies construction parameters to respective building surfaces through Energy ADE construction module. At this point, all required parameters are available to utilize TABULA archetypes for calculating U-values of the building envelope components, resulting in a thermally enriched geometric model with wall transmittance, roof insulation, and ground coupling characteristics.

Energy System Calculator assigns representative heating and renewable systems (gas boilers, heat pumps, photovoltaic arrays) based on typological and temporal indicators derived from TABULA archetypes. Each system is characterized by fuel type,

nominal efficiency, installed capacity, and age class through rule-based logic supporting batch processing across entire scenarios.

Additional Calculators

Beyond the core geometric, demographic, and energy calculators, the framework includes specialized calculators for solar potential through raster solar irradiance sampling, grid distance determination via network analysis to the nearest electrical bus, proximity analysis measuring distances to urban amenities, and census integration enabling demographic attribute extraction and allocation.

Figure ?? illustrates how methods from different calculators can be chained across execution paths with green and red arrows representing different user-defined workflows and fallback mechanisms.

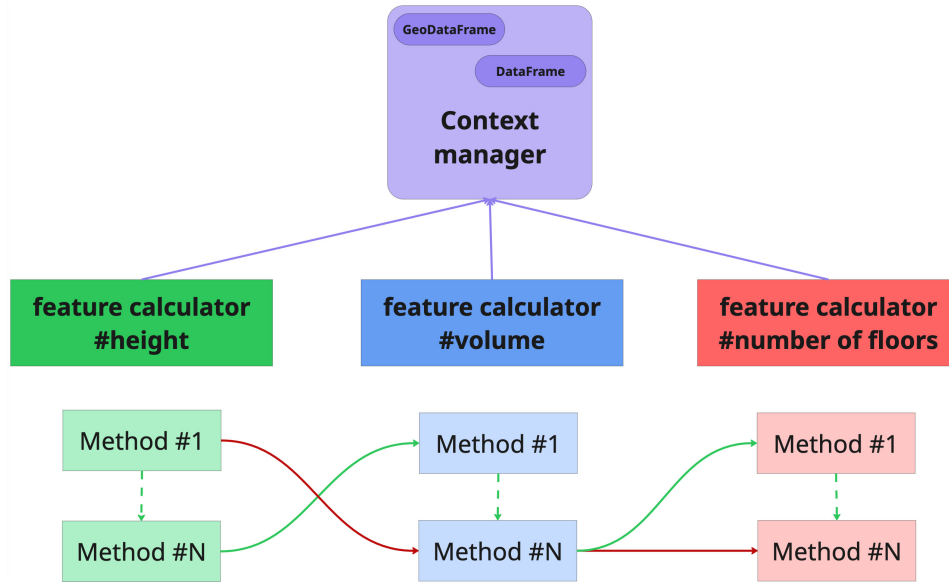


Figure 4.5. Possible workflow pipelines showing feature calculators with multiple methods chained across execution paths. Green dashed lines illustrate fallback mechanisms with defined priority, while red lines show alternative execution paths users can configure.

4.2.11 Extensibility for Researchers

A core strength of the methodology is its extensibility: it allows developers to introduce new computational logic for existing features as well as adding new features without altering the internal orchestration engine. This is achieved through modular design where each feature calculator is defined as a standalone class in object-oriented programming. These classes adhere to a common interface and are structured to encapsulate one or more estimation methods.

To register a new calculator, the developer simply declares the class in the configuration catalog, defining available methods, input dependencies, execution priorities, and

expected outputs. This configuration is dynamically loaded at runtime and serves as the central decision-making layer for pipeline execution. Importantly, no changes are required in the core logic or orchestration engine; the new class becomes automatically available once the configuration is updated. This decoupled approach accelerates prototyping, encourages experimentation, and enables parallel development without compromising system stability.

Method Resolution and Fallback Logic

When the pipeline is triggered for a specific feature, the orchestrator follows a multi-step resolution procedure. It begins by consulting the configuration catalog to identify all registered methods associated with the target feature, ranked by developer-defined priority. For each method in descending order, the system evaluates availability of required inputs (spatial geometries, demographic attributes, previously computed features). If all dependencies are satisfied, the method is executed. If the result is valid and non-null, it is stored; otherwise, the system proceeds to the next method. This fallback-driven mechanism ensures resilience to missing data and supports integration of multi-tiered estimation logic.

Runtime Priority Overrides

The methodology provides runtime override mechanism for method prioritization, particularly useful during sensitivity analysis, calibration studies, or comparative assessments of different estimation strategies. Through dedicated API endpoints, users or developers can inject temporary priority settings for selected features without altering the underlying configuration file. These overrides are interpreted at runtime and take precedence over default priority hierarchy, enhancing usability in both research and operational scenarios.

```
1 class RooftopAreaCalculator(BaseCalculator):
2     """Calculate usable rooftop area for solar panels"""
3
4     def calculate_method1_3d_model(self):
5         """Use 3D building model if available"""
6         # Implementation using 3D geometry
7         pass
8
9     def calculate_method2_footprint_ratio(self):
10        """Estimate as percentage of footprint area"""
11        footprint_area = BuildingAreaCalculator(
12            self.building_id, self.db
13        ).execute()
14
15        # Assume 80% of footprint is usable
16        return footprint_area * 0.8 if footprint_area else None
17
18 # Register new calculator
19 CALCULATOR_REGISTRY['rooftop_area'] = RooftopAreaCalculator
```

Listing 4.4. Extending platform with new calculator

4.2.12 API Endpoints

CIM Wizard exposes RESTful API endpoints enabling programmatic access to all platform capabilities. The `POST /calculate` endpoint executes a single calculator for a specified building, while `POST /pipeline` runs complete calculator pipelines. Batch processing is supported through `POST /batch` for multiple buildings. Discovery endpoints include `GET /calculators` to list available calculators and their methods, and `GET /schemas` to retrieve database schema information for query construction.

4.3 CIM Assist

CIM Assist represents the natural language interface layer that bridges the gap between non-technical energy researchers and the complex CIM Wizard spatial database. While CIM Wizard provides the extensible platform for defining building calculators and managing multi-schema data, the inherent complexity of PostGIS spatial SQL queries remains a significant barrier for domain experts who lack database programming expertise. CIM Assist addresses this accessibility challenge through an autonomous LLM-powered agent that translates natural language questions into correct PostGIS queries, executes them against the CIM Wizard database, and presents results in an interpretable format.

The system operates through an integrated three-phase pipeline that combines cost-effective dataset generation, parameter-efficient fine-tuning, and intelligent agent orchestration. The complete pipeline, illustrated in Figure ??, encompasses: (1) **Dataset Generation (AI4DB)**: A two-stage pipeline generating high-quality training data through rule-based template generation and multi-strategy LLM augmentation, producing over 10,800 samples with frequency-based distribution matching actual CIM Wizard usage patterns; (2) **Fine-Tuning (TXT2SSQL)**: Two-stage fine-tuning using QLoRA trains specialized 8-32B parameter models on consumer hardware, learning to first generate reasoning instructions (Question→Instruction) and then produce SQL queries (Question+Instruction→SQL); and (3) **Agent Integration**: The fine-tuned models deploy through a LangGraph-based agent with database introspection tools, enabling self-correction through execution feedback and achieving 75-92% first-shot accuracy and 90-96% eventual accuracy after iteration.

This integrated approach transforms CIM Wizard from an expert-oriented database system into an accessible research tool where energy domain specialists can pose questions like "Show buildings with solar potential above 150 kWh/m² within 500 meters of subway stations" and receive accurate PostGIS query results without writing SQL code. The methodology first establishes the evaluation framework defining quality standards and taxonomy, then details the dataset generation pipeline, fine-tuning architecture, and agent integration demonstrating production-ready performance.

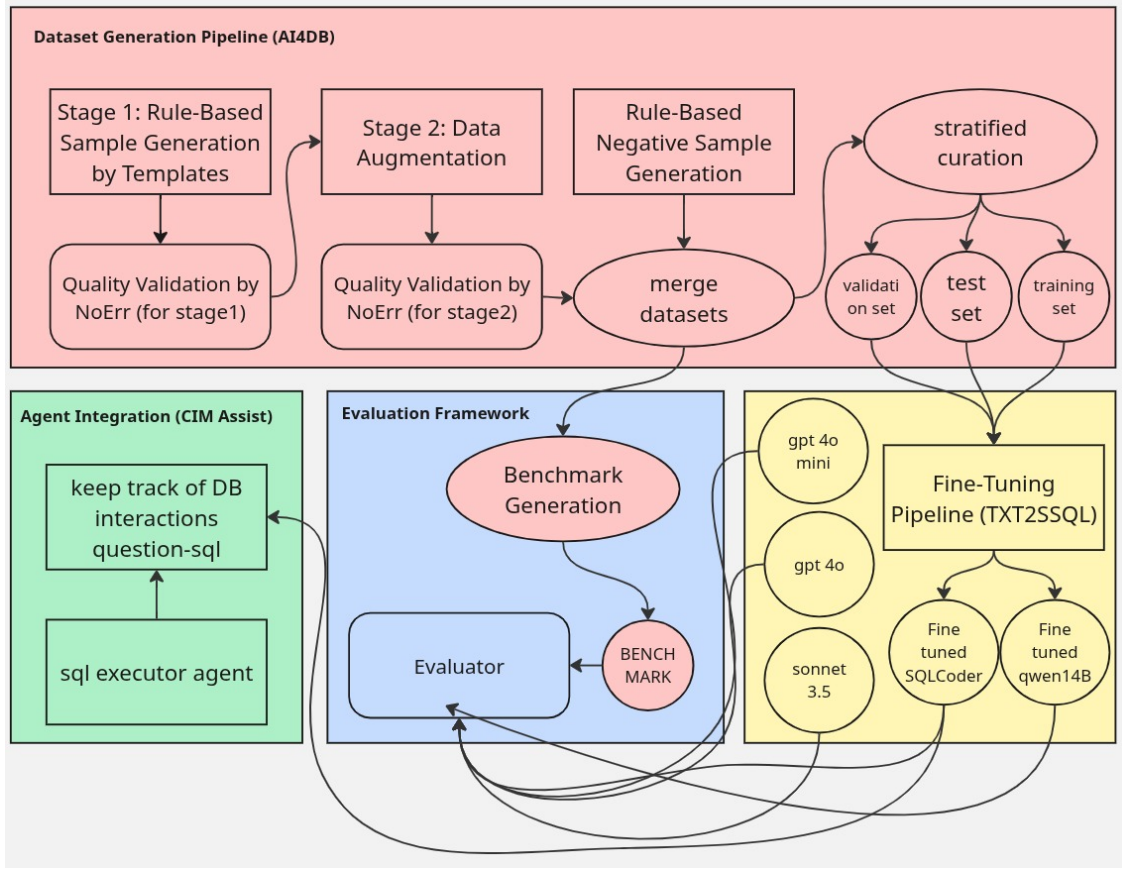


Figure 4.6. CIM Assist complete pipeline: Dataset Generation (AI4DB) → Fine-Tuning (TXT2SSQL) → Agent Integration, enabling natural language access to CIM Wizard spatial database

4.4 Evaluation Framework

The evaluation framework underwent significant evolution after the second iteration, when it became clear that the initial metrics (NoErr, EM, EX, EA) were insufficient to fully understand model behavior. The framework was expanded to include additional metrics (Deep EM, SC) and a comprehensive taxonomy-based analysis system, implemented in `evaluator_v2.py`, which provides detailed insights into model performance across different query types, complexity levels, and domain patterns.

This section presents the comprehensive evaluation framework that guides both dataset generation quality control and model performance assessment, establishing the foundation for understanding system behavior throughout development.

4.4.1 Taxonomy-Driven Evaluation Framework for Comprehensive Performance Analysis

The evaluation framework provides comprehensive assessment through `evaluator_v2.py`, a unified evaluation system that consolidates all assessment logic to ensure consistent evaluation across models while enabling detailed insights into model behavior patterns. This taxonomy-driven approach transforms raw performance metrics into actionable insights by automatically classifying queries across multiple dimensions and generating detailed breakdowns that reveal where models excel and where they struggle. Critically, the taxonomy system is not limited to evaluation alone but is integrated throughout the entire dataset generation pipeline, ensuring that both positive and negative training samples are systematically classified and that evaluation metrics can be aggregated along the same dimensions used during training data creation.

System Architecture

`evaluator_v2.py` implements a comprehensive evaluation system integrating five core components that work together to provide multi-dimensional performance analysis. The system employs multi-metric evaluation measuring six complementary metrics (NoErr, EM, EX, EA, Deep EM, SC) that capture different aspects of model performance from dataset quality to production readiness. Automatic taxonomy classification analyzes each query across multiple dimensions including task type, domain complexity, and usage frequency, enabling granular performance breakdowns that reveal model strengths and weaknesses. The framework integrates with LangGraph agents to support iterative self-correction workflows with error feedback, measuring how models improve through multiple attempts. Comprehensive reporting generates detailed breakdowns organized by taxonomy categories, transforming raw metrics into interpretable insights. Finally, artifact logging captures per-sample execution history including generated SQL at each iteration, execution results, errors, and correction patterns, enabling detailed debugging and analysis of failure modes.

Taxonomy System for Multi-Dimensional Classification

The evaluator employs a dual-taxonomy classification system that categorizes each query along complementary dimensions, enabling comprehensive performance analysis that reveals how model behavior varies across different query characteristics. This taxonomy-driven approach addresses the fundamental limitation of aggregate metrics by providing granular insights into performance patterns that would remain hidden in overall accuracy scores. The taxonomy system is consistently applied throughout the dataset generation pipeline, ensuring that training samples (both positive and negative) are classified using the same taxonomy dimensions as evaluation samples, enabling meaningful performance analysis that directly reflects training data distribution.

Task Taxonomy: SQL Operation Classification The task taxonomy classifies SQL queries by operation type and spatial function usage, organizing queries into hierarchical categories that reflect both syntactic structure and semantic complexity. Non-spatial operations encompass the fundamental SQL patterns including `SIMPLE_SELECT` for basic retrieval, `SQL_AGGREGATION` for analytical queries, `MULTI_SQL_JOIN` for relational operations, and `NESTED_QUERY` for complex subquery structures. Spatial predicates capture queries using spatial relationship functions, distinguishing between `SPATIAL_PREDICATE` for basic containment and intersection operations and `SPATIAL_PREDICATE_DISTANCE` for proximity-based queries. Spatial operations represent the diverse PostGIS function categories including `SPATIAL_MEASUREMENT` for distance and area calculations, `SPATIAL_PROCESSING` for geometric transformations, `SPATIAL_ACCESSOR` for coordinate and boundary extraction, `SPATIAL_CONSTRUCTOR` for geometry creation, `SPATIAL_TRANSFORM` for coordinate system conversions, and `SPATIAL_VALIDATION` for geometry integrity checks. Spatial joins capture multi-table spatial operations through `SPATIAL_JOIN` for single spatial relationships and `MULTI_SPATIAL_JOIN` for complex multi-table spatial queries. Advanced categories encompass sophisticated analytical patterns including `SPATIAL_CLUSTERING` for grouping operations, `RASTER_ANALYSIS` for raster data processing, and `RASTER_VECTOR` for cross-model integration queries.

Each task type classification includes two complementary attributes that enable nuanced analysis: complexity ratings (1=Easy, 2=Medium, 3=Hard) reflect the cognitive and computational difficulty of the operation, while frequency classifications (1=Very Frequent, 2=Frequent, 3=Rare) indicate how commonly such queries occur in actual CIM Wizard usage patterns. This dual-attribute classification enables analysis of performance degradation as complexity increases and identification of whether models struggle more with rare patterns versus common operations.

Domain Taxonomy: Schema Usage Pattern Classification The domain taxonomy classifies queries by CIM Wizard schema usage patterns, revealing how model performance varies with database complexity and cross-schema integration requirements. The classification system recognizes five distinct patterns that reflect increasing complexity: `SINGLE_SCHEMA_CIM_VECTOR` represents queries operating solely within the `cim_vector` schema (complexity: 1, frequency: 1), representing the dominant use case where most building queries occur. `MULTI_SCHEMA_WITH_CIM_VECTOR` captures queries combining `cim_vector` with one additional schema such as census or raster data (complexity: 2, frequency: 2), representing occasional cross-domain analysis. `SINGLE_SCHEMA_OTHER` identifies queries operating within non-vector schemas alone (complexity: 1, frequency: 2), representing specialized queries that may be less common but simpler in structure. `MULTI_SCHEMA_WITHOUT_CIM_VECTOR` encompasses queries combining multiple schemas without the primary vector schema (complexity: 2, frequency: 3), representing rare but complex analytical patterns. Finally, `MULTI_SCHEMA_COMPLEX` captures queries integrating three or more schemas simultaneously (complexity: 3, frequency: 3), representing the most sophisticated cross-domain integration scenarios.

This domain classification enables identification of whether performance degradation stems from schema complexity, cross-schema integration challenges, or specific schema-specific knowledge gaps. The frequency attributes reveal whether models struggle more with rare but complex patterns versus common simpler operations, informing decisions about training data distribution and model architecture selection.

Question Tone Classification The question tone taxonomy classifies natural language queries by their linguistic structure, recognizing three primary patterns that reflect how users phrase database questions: INTERROGATIVE questions (70% distribution) use question words such as "What", "Which", "How many", representing the most natural and common query formulation style. DIRECT commands (20% distribution) employ imperative verbs such as "Find", "Show", "Get", representing explicit instruction-based queries. DESCRIPTIVE requests (10% distribution) use narrative formulations such as "I need" or "We want to analyze", representing contextual query framing. Additionally, the taxonomy recognizes two negative sample tones: AMBIGUOUS queries that lack sufficient context for unambiguous SQL generation, and OUT_OF_SCOPE queries that are completely irrelevant to the CIM database domain. This classification enables analysis of whether model performance varies with question phrasing style, revealing whether certain linguistic patterns prove more challenging for SQL generation.

Sample Quality Classification The sample dirtiness taxonomy distinguishes between positive and negative training samples, enabling evaluation of model robustness across different data quality categories: CLEAN samples represent valid positive samples with correct SQL that execute successfully against the database. AMBIGUOUS samples are negative training examples with missing context or invalid schema references that should trigger clarification requests rather than SQL generation. OUT_OF_SCOPE samples are negative examples completely irrelevant to the CIM database that should be rejected with appropriate error messages. This classification enables evaluation of model robustness metrics including out-of-scope detection (OOS), ambiguity detection (AD), and schema trust (ST), measuring whether models correctly handle edge cases and invalid inputs rather than hallucinating SQL.

Taxonomy Integration in Dataset Generation The taxonomy system is not limited to evaluation but is systematically applied throughout the dataset generation pipeline, ensuring consistency between training data classification and evaluation metrics. During Stage 1 (rule-based template generation), each generated sample is automatically classified using the taxonomy system, enabling distribution validation and stratification. During Stage 2 (LLM augmentation), the taxonomy fields are preserved when creating augmented samples, ensuring that question paraphrases, SQL variations, and parameter changes maintain their original classification while generating diverse training data. During negative sample generation, samples are explicitly tagged with appropriate taxonomy values including AMBIGUOUS or OUT_OF_SCOPE for sample_dirtiness, enabling models to learn robust rejection patterns. This end-to-end taxonomy integration ensures that evaluation metrics can be meaningfully aggregated along the same dimensions used

during training, enabling direct correlation between training data distribution and model performance patterns.

Taxonomy-Based Comprehensive Reporting

The evaluator generates comprehensive reports organized around the taxonomy system, transforming raw performance metrics into actionable insights through multi-dimensional analysis. The reporting structure comprises four complementary sections that together provide complete understanding of model behavior.

The overall metrics section presents aggregate performance across all samples, providing high-level indicators including EX (First-shot Execution Accuracy) measuring immediate correctness, EA (Eventual Accuracy) capturing self-correction capability, Deep EM (Structural Match) validating SQL structure independent of exact syntax, SC (Semantic Correctness) combining execution and structural validation, and self-correction statistics including correction rate and average iterations needed. These aggregate metrics serve as primary performance indicators while taxonomy breakdowns reveal the underlying patterns.

The taxonomy-based breakdowns section provides detailed performance analysis organized by classification categories, enabling identification of specific strengths and weaknesses. Performance analysis by task type reveals how models handle different SQL operation patterns, showing whether spatial operations prove more challenging than non-spatial queries or whether aggregation queries perform differently than simple selections. Domain type breakdowns expose schema complexity effects, revealing whether multi-schema queries degrade performance and identifying which schema combinations prove most challenging. Complexity analysis examines performance degradation patterns, comparing Easy (1) versus Medium (2) versus Hard (3) queries across both task and domain dimensions to understand how difficulty impacts accuracy. Frequency analysis distinguishes performance on Very Frequent (1) versus Frequent (2) versus Rare (3) patterns, revealing whether models excel on common queries at the expense of rare but important operations. Question tone breakdowns compare INTERROGATIVE, DIRECT, and DESCRIPTIVE formulations, identifying whether natural language style affects model understanding. Sample quality analysis distinguishes CLEAN, AMBIGUOUS, and OUT_OF_SCOPE samples, revealing model robustness in handling edge cases and invalid inputs.

The key insights section provides automated analysis that synthesizes taxonomy breakdowns into actionable findings, automatically identifying best and worst performing task types to highlight model strengths and critical failure modes, best and worst performing domain types to reveal schema-specific capabilities and limitations, and complexity analysis showing performance degradation patterns that quantify how accuracy decreases as query difficulty increases.

The per-sample artifacts section provides detailed execution history for each sample, enabling granular debugging and analysis. This includes generated SQL at each iteration showing the evolution of query generation through self-correction cycles, execution results and errors capturing database responses and failure modes, classification metadata recording taxonomy assignments for each sample, and iteration-by-iteration correction history documenting how models adapt based on error feedback.

Usage and Configuration

The evaluator supports multiple model backends and evaluation modes through flexible configuration options, enabling consistent evaluation across diverse deployment scenarios. The system accepts various model types including HuggingFace models specified as `hf:model_name` for local or remote HuggingFace models, Ollama models specified as `ollama:model_name` for local Ollama deployments, and OpenRouter models specified as `openrouter:provider/model` for API-based model access. This multi-backend support ensures that evaluation remains consistent regardless of model deployment architecture.

Schema context options enable evaluation under different information availability scenarios, supporting three distinct modes: no schema mode where models rely entirely on fine-tuned knowledge without external schema information, full schema mode activated with `-include_schema` that provides complete database schema in prompts for fair comparison with baseline models, and mini schema mode enabled with `-use_finetuned_schema` that includes minimal schema summary matching what was available during fine-tuning, representing the production deployment scenario.

The evaluation workflow is invoked through command-line interface as illustrated in the following example, which demonstrates typical usage for comprehensive model assessment:

```

1 python evaluator_v2.py \
2   --benchmark ftv2_evaluation_benchmark_100.jsonl \
3   --model hf:defog/sqlcoder-7b-2 \
4   --mode Q2SQL \
5   --max_iterations 5 \
6   --use_finetuned_schema \
7   --artifacts_dir ea_model_artifacts_v2

```

Listing 4.5. Evaluator v2 usage example

This taxonomy-driven evaluation framework transforms model assessment from simple accuracy reporting into comprehensive performance analysis, enabling researchers and practitioners to understand not just how well models perform, but where they excel, where they struggle, and how performance varies across different query characteristics and complexity levels. By applying the same taxonomy system during both dataset generation and evaluation, the framework ensures that performance insights directly reflect training data characteristics, enabling data-driven improvements to training distribution and model architecture.

4.4.2 Distribution Strategy and Taxonomy-Based Sampling

The quality and effectiveness of text-to-SQL models critically depends on the distribution of training samples across taxonomy dimensions, as imbalanced distributions lead to biased models that overfit to frequent patterns while failing on rare but important query types. This section presents the frequency-based distribution strategy that optimizes for domain-specific performance by aligning training data distribution with actual CIM Wizard usage patterns, ensuring that models excel at the queries users actually ask rather than achieving uniform but mediocre performance across all possible query types.

Distribution Dimensions

The distribution strategy operates across multiple complementary dimensions that together ensure comprehensive coverage while prioritizing domain-specific performance. Each dimension reflects actual CIM Wizard usage patterns observed through analysis of query logs and user interactions, ensuring that training data distribution matches real-world query characteristics rather than artificial balance.

SQL Type Distribution (Task Taxonomy) Training samples are distributed according to actual CIM Wizard usage frequency, with the most common query patterns receiving proportionally more training examples. Simple selection queries (`SIMPLE_SELECT`) represent 45% of the training distribution, reflecting their status as the most common query pattern where users retrieve building or demographic information with basic filtering conditions. Aggregation queries (`SQL_AGGREGATION`) account for 25% of samples, capturing the frequent analytical queries that compute statistics, counts, and summaries across building populations or spatial regions. Spatial join operations (`SPATIAL_JOIN`) comprise 15% of the distribution, representing regular spatial analysis where users combine building data with census zones, network infrastructure, or other spatial entities through geometric relationships. Spatial measurement queries (`SPATIAL_MEASUREMENT`) represent 8% of samples, covering occasional distance, area, and perimeter calculations that support urban energy analysis. Multi-table joins (`MULTI_JOIN`) account for 4% of the distribution, representing rare but important complex join operations that integrate data across multiple tables. Nested queries (`NESTED_QUERY`) comprise 2% of samples, capturing rare nested patterns including CTEs and subqueries that enable sophisticated analytical workflows. Advanced spatial operations including spatial clustering (`SPATIAL_CLUSTERING`) and raster-vector integration (`RASTER_VECTOR`) each represent 0.5% of the distribution, ensuring minimal coverage of very rare advanced analytics patterns without compromising the model’s performance on common queries.

This frequency-based distribution ensures that the model receives extensive training on the query patterns that constitute 70% of actual usage (`SIMPLE_SELECT` and `AGGREGATION` combined), while maintaining sufficient coverage of rare patterns to avoid complete failure on advanced queries. The distribution directly reflects the taxonomy system’s frequency classifications, where Very Frequent (1) patterns receive the highest allocation, Frequent (2) patterns receive moderate allocation, and Rare (3) patterns receive minimal but sufficient allocation.

Domain Distribution (Schema Complexity) The domain distribution reflects schema usage patterns observed in actual CIM Wizard deployments, ensuring that training data matches the complexity distribution of real-world queries. Single-schema queries operating solely within the `cim_vector` schema (`SINGLE_SCHEMA_CIM_VECTOR`) represent 70% of the training distribution, reflecting the dominant use case where most building queries operate within the primary vector schema without requiring cross-schema integration. Multi-schema queries combining `cim_vector` with one additional schema such as census or raster data (`MULTI_SCHEMA_WITH_CIM_VECTOR`) account for 20%

of samples, representing occasional cross-domain analysis where users integrate building data with demographic or elevation information. Single-schema queries operating within non-vector schemas alone (SINGLE_SCHEMA_OTHER) comprise 8% of the distribution, representing rare specialized queries that focus exclusively on census, network, or raster data without building context. Multi-schema queries combining multiple schemas without the primary vector schema (MULTI_SCHEMA_WITHOUT_CIM_VECTOR) represent 2% of samples, ensuring minimal coverage of very rare advanced analytical patterns that integrate census, network, and raster data directly.

This schema complexity distribution ensures that the model receives extensive training on single-schema building queries that constitute the majority of actual usage, while maintaining sufficient coverage of cross-schema integration patterns to enable occasional complex analytical workflows. The distribution aligns with the domain taxonomy’s frequency classifications, where Very Frequent (1) single-schema patterns receive dominant allocation, Frequent (2) multi-schema patterns receive moderate allocation, and Rare (3) complex multi-schema patterns receive minimal allocation.

Sample Quality Distribution The sample quality distribution balances positive training examples with negative samples that teach robust rejection patterns, ensuring that models learn to handle both valid queries and edge cases appropriately. Positive samples representing valid spatial SQL queries that execute successfully against the database comprise 80% of the training distribution, providing extensive examples of correct query generation patterns across all taxonomy dimensions. Negative samples including out-of-scope queries, ambiguous requests with missing context, and invalid schema references account for 20% of the distribution, teaching models to recognize and appropriately reject invalid inputs rather than hallucinating SQL. This 80/20 positive/negative split enables models to learn robust rejection patterns while maintaining focus on correct query generation, with negative samples systematically distributed across taxonomy dimensions to ensure comprehensive robustness training.

Quality gates implemented through NoErr validation at each dataset generation stage ensure that only executable queries proceed to subsequent stages, preventing error propagation and maintaining high training data quality. Stage 1 (rule-based templates) achieves 98-100% NoErr rates through systematic schema validation, while Stage 2 (LLM augmentation) maintains 85%+ NoErr rates through database execution validation before samples enter the training dataset. This validation-first approach ensures that taxonomy-based distribution targets are achieved with high-quality samples rather than error-prone queries that would degrade model performance.

Frequency-Based vs Balanced Distribution

The choice between frequency-based and balanced distribution strategies fundamentally depends on the model’s intended purpose and deployment context, with each approach offering distinct advantages and trade-offs that must be carefully evaluated based on application requirements.

Frequency-Based Distribution: Domain-Specific Optimization The frequency-based distribution approach, employed in this work, prioritizes excellent performance on actual usage patterns over uniform competence across all possible query types. This strategy targets domain-specific models where the primary objective is to excel at the queries users actually ask in the CIM Wizard context, rather than achieving general SQL competence across diverse database domains. The philosophical foundation of this approach recognizes that it is better to excel at what users need than to be mediocre at everything, accepting that lower performance on rare patterns is an acceptable trade-off when those patterns constitute less than 5% of actual usage.

The primary benefit of frequency-based distribution lies in achieving higher weighted accuracy by prioritizing common queries, where a 10 percentage point improvement on queries that occur 70% of the time provides substantially more value than equivalent improvement on queries that occur 2% of the time. This approach recognizes that real-world model performance is measured not by uniform accuracy across all query types, but by weighted accuracy that reflects actual usage patterns. The trade-off involves accepting lower performance on rare patterns such as spatial clustering or complex multi-schema queries, where achieving 45% accuracy on patterns that occur 0.5% of the time is acceptable when the model achieves 92% accuracy on patterns that occur 45% of the time.

This frequency-based approach directly aligns with the taxonomy system’s frequency classifications, where Very Frequent (1) patterns receive dominant training allocation, Frequent (2) patterns receive moderate allocation, and Rare (3) patterns receive minimal but sufficient allocation to avoid complete failure. The distribution strategy ensures that training data reflects actual CIM Wizard usage patterns, enabling models to learn the specific spatial SQL patterns needed for urban energy analysis rather than general SQL competence that may not transfer effectively to the domain-specific requirements of PostGIS spatial queries.

Balanced Distribution: General-Purpose Competence The balanced distribution approach targets uniform performance across all SQL patterns, allocating equal training samples to each query type regardless of actual usage frequency. This strategy is appropriate for general-purpose SQL assistants, database education platforms, or benchmark evaluation where the objective is to demonstrate competence across diverse query types rather than optimizing for specific usage patterns. The approach ensures that models achieve reasonable performance on rare patterns such as spatial clustering or complex nested queries, but at the cost of mediocre performance on common queries that constitute the majority of actual usage.

The trade-off inherent in balanced distribution involves accepting lower accuracy on frequent patterns to achieve better coverage of rare patterns, where a model might achieve 65% accuracy uniformly across all query types rather than 92% on common patterns and 45% on rare patterns. This uniform performance profile may be desirable for educational or benchmarking contexts where demonstrating broad competence is more valuable than optimizing for specific usage patterns, but proves suboptimal for production deployment where weighted accuracy reflecting actual usage provides the true measure of model value.

Distribution Strategy Rationale The frequency-based approach prioritizes domain optimization over general SQL competence, ensuring the model excels at the specific spatial SQL patterns needed for urban energy analysis in CIM Wizard. This strategy recognizes that production deployment success depends on performance on queries users actually ask, not on uniform competence across all theoretically possible query types. By aligning training distribution with actual usage patterns observed through query log analysis and user interaction studies, the approach ensures that model development efforts focus on the patterns that provide maximum value to end users.

The taxonomy system’s frequency classifications directly inform distribution targets, where patterns classified as Very Frequent (1) receive proportionally more training samples, patterns classified as Frequent (2) receive moderate allocation, and patterns classified as Rare (3) receive minimal but sufficient allocation. This systematic approach ensures that distribution decisions are data-driven rather than arbitrary, with taxonomy classifications providing the quantitative foundation for training data allocation decisions. The end result is a model that achieves production-ready performance on the queries that matter most for CIM Wizard users, while maintaining sufficient coverage of rare patterns to avoid complete failure on advanced analytical workflows.

4.4.3 Dataset Quality Metrics

NoErr (No Error) – Dataset Quality Metric

$$\text{NoErr} = \frac{\text{Executable Queries}}{\text{Total Samples}} \times 100\% \quad (4.1)$$

Purpose: Validate dataset generation quality at each stage

Target Thresholds:

- Stage 1 (Template-based): $\geq 98\%$ for rule-based generation
- Stage 2 (Augmented): $\geq 85\%$ for LLM-augmented samples

NoErr serves as the primary quality gate throughout the dataset generation pipeline. Each stage validates SQL executability against the CIM Wizard database before passing samples to subsequent stages, preventing error propagation and ensuring training data quality. Achieved quality metrics are presented in Chapter ??.

Mode: Standard evaluation (first-shot only, no agent iteration)

4.4.4 Model Performance Metrics

EM (Exact Match) – First-Shot Metric

$$\text{EM} = \frac{\text{Exact Matches}}{\text{Total Samples}} \times 100\% \quad (4.2)$$

Purpose: Strict validation of model memorization

Implementation:

```

1 def exact_match(predicted_sql: str, ground_truth_sql: str) -> bool:
2     # Normalize SQL (remove whitespace, lowercase keywords)
3     pred_normalized = normalize_sql(predicted_sql)
4     truth_normalized = normalize_sql(ground_truth_sql)
5
6     return pred_normalized == truth_normalized

```

Limitation: Different but semantically equivalent SQL queries score as incorrect

Deep EM (Structural Exact Match) – First-Shot Metric

$$\text{Deep EM} = \frac{1}{4} (\mathbb{1}_{\text{spatial_funcs}} + \mathbb{1}_{\text{func_count}} + \mathbb{1}_{\text{join_count}} + \mathbb{1}_{\text{table_count}}) \quad (4.3)$$

Purpose: Evaluate SQL structural correctness independent of exact string matching

Implementation:

```

1 def deep_exact_match(predicted_sql: str, benchmark_item: dict) -> dict:
2     # Extract structural features from generated SQL
3     gen_spatial_funcs = extract_spatial_functions(predicted_sql)
4     gen_table_count = count_tables(predicted_sql)
5     gen_join_count = count_joins(predicted_sql)
6
7     # Compare with ground truth metadata
8     scores = {
9         'spatial_functions_match': set(gen_spatial_funcs) == set(
10             benchmark_item['spatial_functions']),
11         'spatial_func_count_match': abs(len(gen_spatial_funcs) -
12             benchmark_item['spatial_function_count']) <= 1,
13         'join_count_match': abs(gen_join_count - int(benchmark_item['
14             join_count'])) <= 1,
15         'table_count_match': abs(gen_table_count - benchmark_item['
16             table_count']) <= 1
17     }
18
19     # Overall score: pass if 75% of features match
20     overall_score = sum(scores.values()) / len(scores)
21     return {'deep_em_score': overall_score, 'deep_em_pass': overall_score
22         >= 0.75}

```

Advantages: Deep EM tolerates different SQL formulations including CTEs, sub-queries, and aliases while focusing on structural correctness through validation of the right tables, joins, and functions. The metric provides more leniency than string-based EM but maintains stricter requirements than EX. For debugging purposes, it identifies precisely which structural elements are incorrect.

Scoring: A query passes when at least 75% of structural features match, requiring 3 out of 4 components to align.

EX (Execution Accuracy) – First-Shot Metric

$$\text{EX} = \frac{\text{Correct Results}}{\text{Total Samples}} \times 100\% \quad (4.4)$$

Purpose: Practical validation of query correctness through result matching

Implementation:

```

1 def execution_accuracy(predicted_sql: str, ground_truth_results) -> bool:
2     try:
3         # Execute predicted SQL
4         predicted_results = db.execute(predicted_sql)
5
6         # Compare results using flexible value-based matching
7         return is_subset_match(predicted_results, ground_truth_results)
8     except Exception as e:
9         return False # Execution error counts as incorrect
10
11 def is_subset_match(generated_result, ground_truth_result) -> bool:
12     """
13     Flexible result matching focusing on data values, not structure.
14     Handles column order, row order, and floating point differences.
15     """
16     if not generated_result:
17         return not ground_truth_result or len(ground_truth_result) <= 1
18
19     if not ground_truth_result:
20         return False
21
22     def normalize_value(val):
23         """Normalize single value for comparison."""
24         if val is None:
25             return 'NULL'
26         if isinstance(val, float):
27             return round(val, 6) # Handle floating point precision
28         if isinstance(val, str):
29             return val.strip().lower()
30         return val
31
32     def normalize_row(row):
33         """Normalize row by sorting values (order-agnostic)."""
34         normalized = tuple(sorted([normalize_value(v) for v in row], key=
35 str))
36         return normalized
37
38     # Normalize and compare
39     gt_set = set(normalize_row(row) for row in ground_truth_result)
40     gen_set = set(normalize_row(row) for row in generated_result)
41
42     # Check subset or 80% overlap
43     if gen_set.issubset(gt_set):
44         return True
45
46     intersection = len(gen_set.intersection(gt_set))
47     if intersection > 0:
48         overlap_ratio = intersection / len(gen_set)
49         return overlap_ratio >= 0.8
50
51     return False

```

Flexible Matching Logic: The implementation employs value-based comparison focusing on data content rather than structure. Column order becomes irrelevant through value sorting within rows, while row order independence is achieved through set-based

comparison. Floating point tolerance handles precision differences by rounding to 6 decimal places. The metric accepts results with 80% overlap threshold and gracefully handles different LIMIT/ORDER BY clauses.

Strength: The metric accepts semantically equivalent SQL variations while remaining strict on correctness.

Threshold: Accuracy exceeding 70% indicates production readiness.

SC (Semantic Correctness) – Practical Metric

$$SC = \frac{1}{N} \sum_{i=1}^N \mathbb{1}_{\text{exec}_i} \cdot \mathbb{1}_{\text{deep_em}_i} \cdot \mathbb{1}_{\text{results}_i} \quad (4.5)$$

Purpose: Realistic evaluation for non-deterministic SQL (without ORDER BY)

Implementation:

```

1 def semantic_correctness(sample: dict) -> bool:
2     """
3     Check if SQL is semantically correct:
4     1. Executes successfully (no errors)
5     2. Deep EM passes (correct structure)
6     3. Returns non-empty results (if expected)
7     """
8     exec_success = sample['final_execution']['success']
9     deep_em_pass = sample['deep_em']['deep_em_pass']
10    has_results = sample['final_execution']['rowcount'] > 0
11
12    return exec_success and deep_em_pass and has_results

```

Rationale: The SC metric addresses limitations where EX fails when models use different LIMIT/ORDER BY clauses than ground truth. SC accepts structurally correct SQL that produces valid results, providing a more realistic evaluation for production environments where exact result matching is not required. The metric combines three components: execution success without errors, structural correctness validated through Deep EM, and non-empty output when results are expected.

Use Cases: SC proves valuable for benchmarks with ambiguous questions lacking specific project_id filters, non-deterministic SQL without ORDER BY clauses, queries where LIMIT is reasonable but not explicitly specified, and situations evaluating model understanding versus exact output matching.

Comparison with EX: EX employs value-based result matching that is strict yet flexible on structure, while SC validates structural and execution correctness accepting valid variations. SC always equals or exceeds EX by definition. Small gaps of 10-15 percentage points indicate good benchmark quality, whereas large gaps exceeding 30 percentage points suggest benchmark issues such as ambiguous questions or non-deterministic SQL specifications.

EA (Eventual Accuracy) – Agent Mode Metric

$$EA_{\text{score}} = \begin{cases} 1.0 & \text{if correct on first attempt} \\ 1.0 - 0.15 \times (n - 1) & \text{if correct after } n \text{ iterations} \\ 0.0 & \text{if never correct} \end{cases} \quad (4.6)$$

Purpose: Evaluate self-correction ability and production readiness

Components: The metric comprises five complementary measurements. First-shot accuracy measures success on the initial attempt without iteration. Eventual accuracy captures final success after retries up to 5 attempts. Self-correction rate quantifies the percentage of queries that succeeded after initial failure. Average iterations tracks the mean number of attempts needed across all queries. The EA score provides a weighted score considering both correctness and efficiency through iteration penalties.

Implementation:

```

1 def eventual_accuracy_evaluation(question, ground_truth_results, agent,
2   max_iter=5):
3     first_shot_correct = False
4     eventually_correct = False
5     iterations = 0
6
7     for i in range(1, max_iter + 1):
8         iterations = i
9
10        # Agent generates SQL (with error feedback from previous attempt)
11        sql = agent.generate_sql(question)
12
13        # Execute and check
14        try:
15            results = db.execute(sql)
16            correct = results_equal(results, ground_truth_results)
17
18            if i == 1 and correct:
19                first_shot_correct = True
20
21            if correct:
22                eventually_correct = True
23                break
24        except Exception as e:
25            # Provide error feedback to agent
26            agent.update_error_feedback(str(e))
27
28        # Calculate EA score
29        if first_shot_correct:
30            ea_score = 1.0
31        elif eventually_correct:
32            ea_score = 1.0 - 0.15 * (iterations - 1)
33        else:
34            ea_score = 0.0
35
36    return {
37        'first_shot': first_shot_correct,
38        'eventual': eventually_correct,

```

```

38     'iterations': iterations,
39     'ea_score': ea_score
40 }

```

Iteration Penalty: The metric applies a 15% score reduction per additional attempt to encourage efficiency.

Negative Samples Detection Metrics

These metrics evaluate model robustness on negative samples:

Out-of-Scope Detection (OOS)

$$\text{OOS} = \frac{\text{Correctly Rejected Out-of-Scope Queries}}{\text{Total Out-of-Scope Queries}} \times 100\% \quad (4.7)$$

Purpose: Validate model’s ability to reject non-database queries

Implementation:

```

1 def out_of_scope_detection(response: str, expected_category: str) -> bool:
2     """Check if model correctly identifies out-of-scope query"""
3     response_upper = response.upper()
4
5     # Model should respond with OUT_OF_SCOPE
6     if 'OUT_OF_SCOPE' in response_upper or 'OUT OF SCOPE' in
7     response_upper:
8         return True
9
10    # Model should NOT generate SQL for conversational/general queries
11    if 'SELECT' in response_upper:
12        return False # Incorrectly generated SQL
13
14    return False

```

Ambiguity Detection (AD)

$$\text{AD} = \frac{\text{Correctly Identified Ambiguous Queries}}{\text{Total Ambiguous Queries}} \times 100\% \quad (4.8)$$

Purpose: Validate model’s ability to request clarification for incomplete queries

Test Cases: The metric is validated using queries like “What are the building IDs?” which should elicit **AMBIGUOUS: Specify project**, “Which buildings overlap?” expecting **AMBIGUOUS: Overlap with what?**, and “Calculate distances” prompting **AMBIGUOUS: Between which entities?**

Expected Values: Phase 4 trained models are expected to achieve 90%+ detection accuracy.

Schema Trust (ST)

$$ST = \frac{\text{Correct Schema Despite Wrong Instruction}}{\text{Total Adversarial Samples}} \times 100\% \quad (4.9)$$

Purpose: Validate model trusts learned schema over incorrect instructions

Test Cases:

- Instruction: “use ‘structures’ table” → Model uses `cim_wizard_building`
- Instruction: “select ‘size’ column” → Model uses `area` column
- Instruction: “join by building_id” → Model uses `ST_Intersects`

4.4.5 Evaluation Process

All evaluations are conducted using `evaluator_v2.py`, which provides unified assessment across all models and evaluation modes, ensuring consistent evaluation methodology regardless of model architecture or deployment scenario. The system supports both baseline and fine-tuned model evaluation with comprehensive taxonomy-based analysis, enabling direct performance comparison across different model families and training approaches. The unified framework eliminates methodological inconsistencies that would otherwise complicate model comparison, ensuring that performance differences reflect actual model capabilities rather than evaluation procedure variations.

Baseline Model Evaluation

Open-source models are evaluated without fine-tuning to establish baseline performance that quantifies the improvement achieved through domain-specific adaptation. The evaluation encompasses various open-source model families including Llama, Qwen, and other publicly available models, providing comprehensive understanding of general-purpose model capabilities on spatial SQL tasks. Evaluation proceeds through Q2SQL mode with both first-shot and agent mode assessment, measuring immediate accuracy without iteration and eventual accuracy after self-correction cycles. The metric suite includes EX (first-shot execution accuracy) measuring immediate correctness, EA (eventual accuracy) capturing self-correction capability, Deep EM (structural match) validating SQL structure independent of exact syntax, and SC (semantic correctness) combining execution, structure, and result validation. To ensure fair comparison with fine-tuned models, baseline evaluations provide schema context through either full database schema or mini-schema summaries in prompts, enabling models to access necessary table and column information that fine-tuned models have internalized. Taxonomy analysis provides detailed performance breakdowns by task type, domain complexity, and frequency classifications, revealing how general-purpose models handle different query characteristics and identifying specific failure modes that domain adaptation addresses.

Fine-Tuned Model Evaluation

Domain-adapted models are evaluated using the same comprehensive framework, enabling direct comparison with baseline performance while providing detailed insights into fine-tuning effectiveness. The evaluation sequence begins with EM (Exact Match) evaluation measuring exact string match rate to understand the level of memorization versus generalization achieved through training. EX evaluation (first-shot) measures single-attempt accuracy as the primary production readiness indicator, representing the model’s immediate correctness without iterative refinement. Deep EM evaluation assesses structural correctness by validating that generated SQL uses appropriate spatial functions, joins, and table references regardless of exact syntactic formulation. SC evaluation combines execution success, structural correctness validated through Deep EM, and non-empty result validation, providing realistic assessment for production environments where exact result matching is not required. EA evaluation (agent mode) measures multi-attempt accuracy with self-correction capability, where models receive execution error feedback and iteratively refine their SQL generation, representing production deployment scenarios with error handling mechanisms. Taxonomy breakdown provides detailed performance analysis by query category, enabling identification of specific strengths and weaknesses across different task types, domain complexity levels, and frequency patterns, revealing where fine-tuning provides the greatest improvement and where additional training data may be needed.

Comprehensive Reporting

`evaluator_v2.py` generates detailed reports that transform raw evaluation metrics into actionable insights through systematic organization and automated analysis. The overall metrics summary presents aggregate performance across all samples, providing high-level indicators including EX, EA, Deep EM, and SC accuracy rates along with self-correction statistics and average iteration counts. Taxonomy-based breakdowns organize performance by task type, domain type, complexity level, and frequency classification, enabling identification of specific query categories where models excel or struggle. Key insights section provides automated analysis identifying best and worst performing categories, complexity degradation patterns, and frequency-based performance variations, synthesizing taxonomy breakdowns into actionable findings. Per-sample artifacts capture complete execution history for each benchmark item, including generated SQL at each iteration, execution results and errors, classification metadata, and iteration-by-iteration correction history, enabling granular debugging and failure mode analysis. JSON results provide machine-readable detailed results for further statistical analysis, programmatic comparison across models, and integration with downstream analysis tools. Human-readable report presents formatted text output suitable for quick review, documentation, and presentation, with tables and summaries organized for easy interpretation.

The comprehensive reporting enables detailed understanding of model behavior patterns, identifying strengths and weaknesses across different query types and complexity levels. By organizing metrics according to the same taxonomy system used during dataset generation, the reports enable direct correlation between training data distribution and

model performance, revealing how frequency-based training allocation translates into performance patterns and identifying areas where distribution adjustments may improve model capabilities.

4.5 Dataset Generation Pipeline (AI4DB)

The dataset generation pipeline underwent three iterative refinement cycles to achieve the current high-quality distribution. This section describes the final two-stage approach that generates training data optimized for CIM Wizard’s actual usage patterns through frequency-based distribution and multi-strategy augmentation.

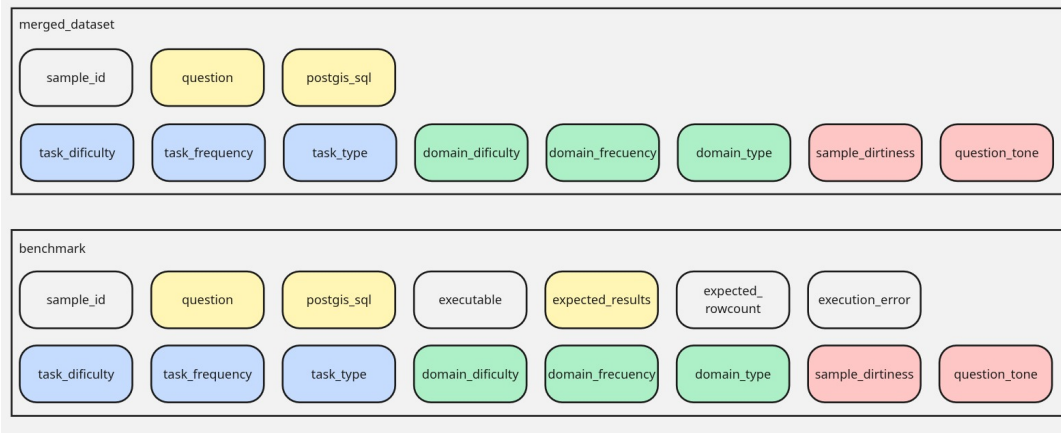


Figure 4.7. AI4DB dataset generation pipeline structure: Two-stage approach combining rule-based template generation (Stage 1) with multi-strategy LLM augmentation (Stage 2), followed by negative sample integration and stratified curation

4.5.1 Stage 1: Rule-Based Sample Generation by Templates

Stage 1 generates SQL queries through handcrafted templates with systematic parameter substitution, producing 10,800 samples distributed according to actual CIM Wizard usage frequency.

Template Organization and Distribution

Templates are organized by SQL operation type and distributed according to frequency-based usage patterns observed in CIM Wizard:

The frequency-based distribution ensures that common query patterns (SIMPLE_SELECT, AGGREGATION) receive proportionally more training samples, maximizing real-world accuracy on queries users actually ask.

Parameter Substitution

Templates are systematically expanded through parameter variation:

SQL Type	Templates	Distribution
SIMPLE_SELECT	18	45%
AGGREGATION	10	25%
SPATIAL_JOIN	8	15%
SPATIAL_MEASUREMENT	4	8%
MULTI_JOIN	2	4%
NESTED_QUERY	1	2%
SPATIAL_CLUSTERING	1	0.5%
RASTER_VECTOR	1	0.5%

Table 4.3. Template distribution by SQL type matching CIM Wizard usage frequency

```

1 template = """
2 SELECT building_id, building_name, ST_Area(building_geometry) as area
3 FROM cim_vector.buildings
4 WHERE project_id = '{project_id}'
5     AND ST_Area(building_geometry) > {area_threshold}
6 ORDER BY area DESC
7 LIMIT {limit};
8 """
9
10 parameters = {
11     'project_id': ['milan_smart_district', 'sansalva', 'torino_pilot'],
12     'area_threshold': [100, 500, 1000, 5000],
13     'limit': [10, 50, 100]
14 }
15
16 # Generate all combinations
17 samples = []
18 for proj in parameters['project_id']:
19     for area in parameters['area_threshold']:
20         for lim in parameters['limit']:
21             sql = template.format(
22                 project_id=proj,
23                 area_threshold=area,
24                 limit=lim
25             )
26             samples.append(sql)
27
28 # Result: 3      4      3 = 36 samples per template

```

Listing 4.6. Template parameter substitution

Through systematic parameter variation, Stage 1 generates **10,800 samples** distributed across all SQL types according to the frequency-based template distribution. Each template produces multiple variations through parameter substitution, ensuring comprehensive coverage while maintaining the target distribution.

Metadata Extraction

Each sample includes comprehensive metadata:

```

1 def extract_metadata(sql: str, schema_info: dict) -> dict:
2     return {
3         # SQL Taxonomy
4         'sql_type': classify_sql_type(sql),
5         'has_cte': 'WITH' in sql.upper(),
6         'subquery_count': count_subqueries(sql),
7         'join_count': count_joins(sql),
8
9         # Difficulty Dimensions
10        'query_complexity': calculate_query_complexity(sql),
11        'spatial_complexity': calculate_spatial_complexity(sql),
12        'schema_complexity': calculate_schema_complexity(sql),
13
14        # Spatial Function Usage
15        'spatial_functions': extract_spatial_functions(sql),
16        'function_importance': classify_function_importance(sql),
17
18        # Database Context
19        'tables': extract_tables(sql, schema_info),
20        'columns': extract_columns(sql, schema_info),
21        'schemas': extract_schemas(sql),
22
23        # Question Tone (for LLM generation guidance)
24        'suggested_tone': suggest_question_tone(sql)
25    }

```

Listing 4.7. Metadata extraction for generated SQL

Quality Validation by NoErr (for stage1)

Stage 1 quality is evaluated using the NoErr metric (defined in Section ??). After comprehensive schema validation, Stage 1 achieves a **98-100% NoErr rate**, demonstrating exceptional query correctness through systematic schema alignment. This high-quality baseline resulted from addressing several critical PostgreSQL and PostGIS-specific requirements: (1) census column references requiring strict case sensitivity with lowercase unquoted identifiers; (2) type casting for mathematical functions like `ROUND()` necessitating explicit `::numeric` conversion; (3) correct ID column usage across different entity types (`building_id`, `bus_id`, `scenario_id`); and (4) `project_id` column scope awareness, as this attribute exists in only two tables within the multi-schema database.

All Stage 1 samples are validated using the NoErr metric before proceeding to Stage 2, ensuring that only executable queries are augmented with natural language. This validation-first approach prevents propagation of errors through the pipeline and dramatically improves final dataset quality.

4.5.2 Stage 2: Data Augmentation

Stage 2 augments Stage 1 SQL queries with natural language questions and reasoning instructions using multiple strategies and LLM models, each selected based on cost-performance trade-offs. The augmentation process generates diverse, high-quality training samples while maintaining cost efficiency through strategic model selection.

Multi-Strategy Augmentation Approach

Stage 2 employs five distinct augmentation strategies, each using different LLM models selected based on task complexity and cost-performance trade-offs. The strategies generate diverse question-SQL pairs through different transformation approaches, ensuring comprehensive coverage of natural language variations and SQL equivalent formulations.

Strategy 1: Question Paraphrase (25% of samples) Rephrases natural language questions while preserving exact SQL intent and all specific values:

```

1 system_prompt = """
2 You are a CIM Wizard database expert.
3 TASK: Rephrase the given question naturally while preserving
4 ALL specific values, IDs, thresholds, and the exact SQL intent.
5
6 GUIDELINES:
7 - Use different wording and sentence structure
8 - Preserve all numbers, IDs, and specific values EXACTLY
9 - Maintain the same query intent
10 - Vary between interrogative, direct, and descriptive styles
11 - Return ONLY the rephrased question, nothing else
12 """
13
14 user_prompt = f"""
15 Rephrase this database question naturally:
16
17 Original: {question}
18
19 Rephrased question:
20 """

```

Listing 4.8. Question paraphrase strategy

Model: GPT-4o-mini (cost-effective for language tasks) **Input:** Original question + SQL **Output:** Rephrased question (SQL unchanged) **Rationale:** Generates natural language diversity while maintaining SQL correctness

Strategy 2: SQL-to-Question (20% of samples) Generates natural language questions from SQL queries, creating reverse mappings:

```

1 system_prompt = """
2 You are a CIM Wizard database expert.
3 TASK: Generate a natural language question that would
4 produce this SQL query.
5
6 GUIDELINES:
7 - Question should match the SQL's intent EXACTLY
8 - Include all specific values, IDs, and thresholds from the SQL
9 - Use natural, conversational language
10 - Vary the question style (interrogative, direct command, descriptive)
11 - Return ONLY the question, nothing else
12 """
13
14 user_prompt = f"""

```

```

15 Generate a natural language question for this SQL query:
16
17 SQL: {sql}
18
19 Question:
20 """

```

Listing 4.9. SQL-to-question generation strategy

Model: GPT-4o-mini (cost-effective for language tasks) **Input:** SQL query **Output:** Natural language question (SQL unchanged) **Rationale:** Creates alternative question formulations for the same SQL pattern

Strategy 3: Question-to-SQL (20% of samples) Generates equivalent SQL queries using different approaches while maintaining the same result:

```

1 system_prompt = """
2 You are a CIM Wizard database expert.
3 TASK: Generate an equivalent PostGIS SQL query that answers
4 this question. You may use a different approach (different join
5 order, different spatial functions, subqueries vs CTEs) while
6 maintaining the SAME result.
7
8 GUIDELINES:
9 - SQL must be syntactically correct PostGIS
10 - Must produce EQUIVALENT results to the original intent
11 - Use different approaches when possible:
12   * Different join order
13   * ST_Within vs ST_Contains (where semantically equivalent)
14   * Subquery vs CTE
15   * Different aggregation approach
16 - Preserve all specific values and IDs EXACTLY
17 - Return ONLY the SQL query, nothing else
18 """
19
20 user_prompt = f"""
21 Generate an equivalent SQL query for this question:
22
23 Question: {question}
24
25 Reference SQL (for understanding intent):
26 {sql}
27
28 Equivalent SQL:
29 """

```

Listing 4.10. Question-to-SQL equivalent generation

Model: GPT-4.1 (powerful model for SQL reasoning tasks) **Input:** Question + reference SQL **Output:** Equivalent SQL (question unchanged) **Rationale:** Generates SQL diversity by exploring alternative query formulations

Strategy 4: SQL Rewrite (15% of samples) Rewrites SQL queries using different syntax or structure while producing identical results:

```

1 system_prompt = """
2 You are a CIM Wizard database expert.
3 TASK: Rewrite this SQL query using different syntax or
4 structure while producing the SAME results.
5
6 GUIDELINES:
7 - Use different join syntax (explicit JOIN vs implicit WHERE)
8 - Reorder joins when possible
9 - Use equivalent spatial functions where appropriate
10 - Convert between subquery and CTE styles
11 - Preserve ALL specific values and conditions
12 - Must produce IDENTICAL results
13 - Return ONLY the rewritten SQL, nothing else
14 """
15
16 user_prompt = f"""
17 Rewrite this SQL query using different syntax while
18 keeping identical results:
19
20 Original SQL: {sql}
21
22 Rewritten SQL:
23 """

```

Listing 4.11. SQL rewrite strategy

Model: GPT-4.1 (powerful model for SQL reasoning tasks) **Input:** Original SQL **Output:** Rewritten SQL (question unchanged) **Rationale:** Creates SQL syntactic diversity while maintaining semantic equivalence

Strategy 5: Parameter Variation (20% of samples) Creates new question-SQL pairs with different but realistic parameter values:

```

1 system_prompt = """
2 You are a CIM Wizard database expert.
3 TASK: Create a new question and SQL pair with different but
4 REALISTIC parameter values while maintaining the SAME query structure.
5
6 GUIDELINES:
7 - Change numeric values to different but realistic values:
8   * Heights: 5-50 meters
9   * Areas: 50-5000 square meters
10  * Distances: 10-1000 meters
11  * Population: 100-10000
12 - Change IDs to different valid values (project_id, scenario_id,
13   building_id)
14 - Building types: residential, commercial, industrial, mixed
15 - Keep the EXACT same query structure and logic
16 - Return in this EXACT format:
17 QUESTION: [your new question]
18 SQL: [your new SQL]
19 """
20
21 user_prompt = f"""
22 Create a variation with different parameter values:

```

```

22
23 Original Question: {question}
24 Original SQL: {sql}
25
26 New question and SQL (format: QUESTION: ... SQL: ...):
27 """

```

Listing 4.12. Parameter variation strategy

Model: GPT-4o-mini (cost-effective for parameter substitution) **Input:** Question + SQL **Output:** New question + new SQL with different parameters **Rationale:** Expands dataset with realistic parameter variations

Model Selection Rationale

The multi-model strategy optimizes cost-performance trade-offs by assigning powerful models to SQL reasoning tasks and cost-effective models to language tasks:

Strategy	Model	Task Type	Usage	Cost Tier
Question Paraphrase	GPT-4o-mini	Language	25%	Low
SQL-to-Question	GPT-4o-mini	Language	20%	Low
Question-to-SQL	GPT-4.1	SQL Reasoning	20%	High
SQL Rewrite	GPT-4.1	SQL Reasoning	15%	High
Parameter Variation	GPT-4o-mini	Language	20%	Low

Table 4.4. Augmentation strategy model assignment and distribution

Key Principles:

- **Task-Specific Model Selection:** SQL reasoning tasks (question-to-SQL, SQL rewrite) use powerful models for superior SQL generation quality, while language tasks (paraphrasing, question generation) use cost-effective models for efficiency
- **Balanced Distribution:** Majority of samples use cost-effective models, with premium models reserved for complex SQL reasoning tasks
- **Diverse Augmentation:** Five strategies cover question variation, SQL variation, and parameter variation, ensuring comprehensive dataset diversity
- **Cost-Effectiveness:** The multi-model strategy optimizes cost-performance trade-offs while maintaining high quality (detailed costs in Chapter ??)

Quality Validation by NoErr (for stage2)

Generated samples undergo rigorous quality evaluation and filtering before curation to ensure that only executable, high-quality SQL queries enter the training dataset. The quality control process uses `evaluate_generation_quality.py` to validate SQL executability against the CIM Wizard database, implementing a validation-first approach that prevents error propagation through the pipeline and maintains training data quality standards.

Quality Evaluation Process The evaluation pipeline processes all augmented samples through systematic database execution validation, ensuring that LLM-generated SQL queries are syntactically correct and semantically valid before inclusion in training data. The evaluation workflow is invoked through command-line interface as illustrated in the following example:

```

1 python evaluate_generation_quality.py \
2   --input stage3_augmented_dataset.jsonl \
3   --output stage3_quality_report.json \
4   --output_annotated stage3_augmented_dataset_annotated.jsonl \
5   --output_filtered stage3_augmented_dataset_filtered.jsonl \
6   --stage 3 \
7   --db_uri "postgresql://cim_wizard_user:cim_wizard_password@localhost
:15432/cim_wizard_integrated"

```

Listing 4.13. Stage 2 quality evaluation

The evaluation process begins with database execution where each generated SQL query is executed against the CIM Wizard database with a 30-second timeout, providing immediate feedback on query correctness through actual database execution rather than static syntax validation. Execution errors are systematically categorized into four distinct error types: `SCHEMA_ERROR` for incorrect table or column references, `SYNTAX_ERROR` for malformed SQL syntax, `TIMEOUT_ERROR` for queries exceeding execution time limits, and `OTHER_ERROR` for unexpected database or connection failures. This error classification enables targeted analysis of failure modes, revealing whether errors stem from schema misunderstanding, syntax generation issues, or query complexity problems. `NoErr` calculation marks samples with successful execution as passing, providing the primary quality metric that quantifies the proportion of generated queries that execute without errors. All samples receive annotation with execution status and detailed error information, creating a comprehensive quality record that enables analysis of error patterns and identification of systematic issues in LLM generation. Finally, a filtered dataset containing only passing samples (`NoErr`) is generated, ensuring that subsequent training stages receive only executable queries that will contribute positively to model learning rather than teaching incorrect patterns.

Quality Validation Approach After augmentation, samples undergo rigorous quality evaluation and filtering to ensure training readiness through a multi-stage validation process that combines database execution validation with semantic and structural quality checks. The quality control process validates SQL executability against the CIM Wizard database as the primary quality gate, ensuring that only successfully executable queries are retained through `NoErr` filtering. This database execution validation provides definitive correctness assessment that cannot be achieved through static analysis alone, as it validates not only syntax but also semantic correctness including proper schema references, valid spatial function usage, and appropriate data type handling.

Beyond execution validation, the quality control process implements additional filtering steps that ensure training data quality across multiple dimensions. Semantic deduplication employs Sentence-BERT [?] embeddings with a 0.95 similarity threshold to remove

near-duplicate questions, preventing the training dataset from being dominated by minor variations of the same query that would provide redundant learning signal. Length filtering enforces a 20-300 character range for questions, ensuring appropriate complexity where questions are neither too brief to lack context nor excessively verbose to introduce noise. Terminology validation requires that questions contain spatial terms or table references, ensuring domain relevance and preventing inclusion of generic queries that lack CIM Wizard-specific context.

The filtered dataset resulting from this multi-stage quality validation is then merged with Stage 1 samples and negative samples for final curation and stratification. This validation-first approach ensures that quality issues are identified and resolved early in the pipeline, preventing error propagation to subsequent stages and maintaining high training data quality throughout the dataset generation process. Detailed quality evaluation results including NoErr rates, error breakdowns, and filtering statistics are presented in Chapter ??, providing quantitative validation of the quality control process effectiveness.

4.5.3 Rule-Based Negative Sample Generation

Production deployment requires robust handling of out-of-scope queries, ambiguous requests, and invalid schema references that would otherwise cause models to hallucinate SQL or generate inappropriate responses. Rule-based negative samples are systematically generated to teach models to respond appropriately rather than attempting SQL generation for invalid inputs, addressing critical production requirements identified during iteration 2 evaluation where models demonstrated insufficient robustness in handling edge cases and invalid queries.

Negative Sample Categories The negative sample generation encompasses four distinct categories that together teach comprehensive robustness patterns, each addressing specific failure modes observed in production deployment scenarios. Out-of-scope samples represent 35% of negative examples, targeting the problem where models attempt SQL generation for any input regardless of relevance, such as responding to conversational queries like "hi" with hallucinated SQL. The solution includes conversational queries ("hi", "how are you", "thanks"), general knowledge questions ("what is machine learning"), and wrong domain queries ("SELECT * FROM customers") that are completely irrelevant to the CIM Wizard database. The training goal teaches models to respond with `OUT_OF_SCOPE: I can only help with CIM Wizard spatial queries`, preventing inappropriate SQL generation for non-database queries and ensuring that models recognize their domain boundaries.

Ambiguous samples also represent 35% of negative examples, addressing the problem where models generate SQL for queries lacking critical parameters, such as "find buildings" without project context that would return unintended results across all projects. The solution includes queries with missing project filters ("What are the building IDs?"), missing spatial context ("Which buildings overlap?"), and logically incomplete queries ("Calculate distances") that require clarification before SQL generation is possible. The training goal teaches models to respond with `AMBIGUOUS: Please specify project and scenario`

context, preventing over-broad queries that would return unintended results and ensuring that models request necessary information before generating SQL.

Invalid schema samples comprise 20% of negative examples, targeting the problem where models may hallucinate non-existent tables or columns when users reference incorrect database structures. The solution includes wrong table references ("find buildings in 'structures' table"), wrong column names ("get building 'size' column"), and wrong schema prefixes ("select from 'spatial_data.buildings'") that do not match the actual CIM Wizard database schema. The training goal teaches models to respond with **AMBIGUOUS: Invalid table name. Use cim_vector.cim_wizard_building**, reducing schema hallucination errors and ensuring that models trust their learned schema knowledge rather than accepting incorrect user references.

Adversarial samples represent 10% of negative examples, implementing schema authority training where models learn to trust their internalized schema knowledge over incorrect user instructions. The problem addressed involves models that trust incorrect instructions such as "use 'structures' table" when the actual schema uses **cim_wizard_building**. The solution employs adversarial samples with wrong instructions but correct expected SQL, where the instruction explicitly provides incorrect schema information but the model should generate SQL using the correct schema learned during fine-tuning. The training goal teaches models to trust their learned schema knowledge over incorrect guidance, improving schema trust and reducing hallucination from incorrect instructions while maintaining the ability to accept valid user clarifications.

LIMIT/ORDER BY Distribution Correction During iteration 2 evaluation, it was identified that models trained on datasets where all templates included LIMIT clauses developed the problematic behavior of always adding LIMIT clauses inappropriately, even when users requested complete result sets. The dataset curation addresses this through realistic distribution matching actual CIM Wizard usage patterns, where the majority of queries (85%) contain neither LIMIT nor ORDER BY clauses, reflecting that users typically want complete results rather than truncated subsets. A smaller proportion (7%) includes ORDER BY only, representing queries where users want sorted complete results, while the remaining 8% includes both LIMIT and ORDER BY, representing explicit requests for "top N" results. This realistic distribution ensures that models learn to add LIMIT clauses only when questions contain explicit requests such as "top N" or "first N", preventing inappropriate result truncation while maintaining the ability to handle explicit pagination requests appropriately.

4.5.4 Merging and Curation Dataset and Final Composition

The final dataset curation process integrates positive samples from Stage 1 and Stage 2 with rule-based negative samples, followed by stratification and shuffling to address unbalanced distributions.

Stratification and Shuffling

After merging positive samples (Stage 1 + Stage 2) with negative samples, the dataset undergoes stratification and shuffling to address unbalanced distributions and ensure proper train/validation/test splits.

Multi-Dimensional Stratification Samples are stratified across multiple dimensions to ensure balanced representation:

```

1 def stratify_dataset(samples):
2     """Stratify samples across multiple dimensions"""
3
4     # Define stratification keys combining multiple dimensions
5     stratify_keys = [
6         f"{s['sql_type']}_{s['difficulty_level']}_{s['schema_complexity']}_{s['sample_type']}"
7         for s in samples
8     ]
9
10    # Stratified train/val/test split (70%/15%/15%)
11    train, temp = train_test_split(
12        samples,
13        test_size=0.3,
14        stratify=stratify_keys,
15        random_state=42
16    )
17
18    val, test = train_test_split(
19        temp,
20        test_size=0.5,
21        stratify=[f"{s['sql_type']}_{s['difficulty_level']}_{s['schema_complexity']}_{s['sample_type']}"
22                  for s in temp],
23        random_state=42
24    )
25
26    return train, val, test

```

Listing 4.14. Multi-dimensional stratification

Stratification Dimensions

- **SQL Type:** Ensures all query types represented in train/val/test
- **Difficulty Level:** EASY, MEDIUM, HARD, VERY_HARD balanced across splits
- **Schema Complexity:** SINGLE_SCHEMA, MULTI_SCHEMA distribution maintained
- **Sample Type:** POSITIVE vs NEGATIVE samples proportionally distributed

4.6 Fine-Tuning Pipeline (TXT2SSQL)

The fine-tuning pipeline underwent iterative refinement across three dataset generation cycles, with architecture and model selection decisions based on empirical evaluation of performance, cost, and deployment feasibility. This section describes the final approach selected after comprehensive comparison.

4.6.1 Selected Model: SQLCoder 7B

Based on iteration 2 evaluation, SQLCoder 7B was selected as the optimal model for fine-tuning, providing the best performance-to-cost ratio.

Model Selection Rationale

SQLCoder 7B was selected based on four key advantages. First, its pre-training on the BIRD benchmark [?] provides a strong SQL generation foundation that transfers effectively to spatial SQL through domain fine-tuning. Second, the model achieves best-in-class execution accuracy after domain adaptation, matching or exceeding significantly larger models including Qwen 14B and Llama 8B. Third, SQLCoder demonstrates superior cost efficiency across all operational dimensions: training completes in 8-12 hours compared to 20-40 hours for larger models, lower GPU memory requirements enable training on consumer hardware, inference latency remains at 1-2 seconds on RTX 6000 24GB VRAM, and power consumption stays minimal due to smaller model size and faster training cycles. Fourth, the 7B parameter size enables efficient production deployment on consumer GPUs with acceptable latency for interactive applications.

4.6.2 QLoRA Fine-Tuning Approach

The fine-tuning strategy employs QLoRA (Quantized Low-Rank Adaptation) to enable parameter-efficient training on consumer hardware. QLoRA combines 4-bit quantization of the base model with low-rank adapter training, dramatically reducing memory requirements while maintaining model performance. This approach trains only a small fraction of parameters (typically <1%) through low-rank decomposition matrices, while keeping the quantized base model frozen.

The technical configuration includes quantization through BitsAndBytes (4-bit NF4 quantization with double quantization), LoRA adapter configuration targeting attention projection layers (q_proj, k_proj, v_proj, o_proj) and feed-forward layers (gate_proj, up_proj, down_proj), and training hyperparameters optimized for stable convergence including cosine learning rate scheduling with warmup, gradient accumulation for effective larger batch sizes, and mixed-precision training (BF16) for computational efficiency.

The training infrastructure utilizes professional-grade GPUs with sufficient VRAM to accommodate quantized models plus adapter parameters, HuggingFace Transformers with PEFT library for adapter management, and Weights & Biases for experiment tracking and monitoring. Detailed technical specifications, code implementations, and infrastructure configurations are presented in Chapter ??.

4.6.3 Final Fine-Tuning Configuration

Based on iterative refinement findings, the final fine-tuning configuration employs SQLCoder 7B with direct Q2SQL architecture, selected through empirical evaluation across multiple models and architectural variants.

Model Selection Rationale

SQLCoder’s pre-training on SQL benchmarks (BIRD) provides a strong foundation for domain adaptation through parameter-efficient fine-tuning. The direct Q2SQL architecture (Question $\rightarrow$ SQL) offers optimal balance between simplicity and performance, while QLoRA quantization enables efficient training on consumer hardware. The training process uses curated samples with systematic optimization strategies, achieving production-ready performance through parameter-efficient adaptation (detailed training results and performance analysis in Chapter ??).

4.7 Iterative Refinement Process

The development methodology employed an iterative refinement approach where each complete pipeline execution revealed specific limitations that informed systematic improvements in subsequent iterations. This section describes the three-iteration evolution that transformed an initial prototype into a production-ready system, highlighting key methodological insights and architectural decisions that emerged through empirical evaluation.

4.7.1 Dataset Evolution

The dataset generation pipeline underwent three complete iterations to achieve the current high-quality distribution, with each iteration involving full pipeline execution from template generation through fine-tuning and evaluation, followed by systematic improvements based on empirical findings. The iterative approach enabled data-driven refinement where evaluation results directly informed subsequent improvements, ensuring that each iteration addressed specific identified limitations rather than implementing arbitrary changes.

The first iteration revealed that initial templates were overly complex, covering advanced SQL patterns including complex nested queries, multi-schema joins, and sophisticated spatial operations that rarely occurred in actual CIM Wizard usage. This complexity mismatch led to models that performed well on rare advanced queries but struggled with common simple queries that constituted the majority of actual usage. This iteration produced the key methodological insight of stage-wise validation: evaluating each dataset generation stage with the NoErr metric ensures samples are executable before passing to the next stage, filtering incorrect samples early in the pipeline and preventing error propagation that would degrade final dataset quality.

The second iteration achieved acceptable performance through simplified templates focusing on common SQL patterns that matched actual CIM Wizard usage frequency.

However, evaluation revealed a critical LIMIT clause bias where all training templates included LIMIT statements, causing models to inappropriately add LIMIT clauses to all generated queries even when users requested complete result sets. This revealed the importance of realistic distribution matching actual usage patterns, where the majority of queries (85%) contain neither LIMIT nor ORDER BY clauses, reflecting that users typically want complete results rather than truncated subsets. The iteration also identified the need for more sophisticated template organization that avoids polluting templates with unnecessary clauses, ensuring that training data reflects realistic query patterns rather than artificial template artifacts.

The third and final iteration implemented comprehensive improvements addressing all identified limitations from previous iterations. Template quality was enhanced through more accurate coverage across all taxonomy dimensions, with templates systematically organized to avoid LIMIT clause pollution and ensure realistic query patterns matching actual CIM Wizard usage. The taxonomy system was refined and expanded to provide better classification granularity, enabling more precise distribution control and performance analysis. The evaluation framework was enhanced with additional metrics including Deep EM and SC, providing deeper insights into model behavior beyond the initial four metrics (NoErr, EM, EX, EA). Multi-strategy augmentation was implemented optimizing cost-performance trade-offs through strategic model selection for different augmentation tasks. Rule-based negative samples were systematically generated and integrated, teaching models robust rejection patterns for out-of-scope queries, ambiguous requests, and invalid schema references. Finally, stratified curation addressed distribution balance through multi-dimensional stratification ensuring proper representation across all taxonomy categories. This comprehensive refinement produced the final training dataset with frequency-based distribution matching actual CIM Wizard usage patterns, achieving production-ready model performance.

4.7.2 Architecture Selection Process

The architecture selection involved systematic comparison of two fundamental approaches: direct single-stage fine-tuning (Question $\rightarrow$ SQL) versus two-stage decomposition (Question $\rightarrow$ Instruction $\rightarrow$ SQL). The direct approach offers simpler deployment and lower computational cost, while the two-stage approach was hypothesized to improve reasoning through explicit intermediate instruction generation that would decompose complex queries into reasoning steps before SQL generation.

Empirical evaluation revealed that the two-stage approach provided only marginal improvement of 0-3.5% accuracy gain, far below the expected 3-5% improvement that would justify the significant cost increase. The two-stage architecture incurred 2-3x increases in training time, inference latency, and computational resources, with instruction generation requiring approximately 200 tokens per sample that dramatically increased both training and inference costs. Additionally, synthetic instructions lacked realistic schema linking and evidence patterns that would require actual user interaction data from CIM Wizard/CIM Assist, limiting the effectiveness of the intermediate reasoning step. The deployment complexity of maintaining two models further reduced the approach’s attractiveness. The direct Q2SQL architecture was consequently selected as the

final approach, providing sufficient accuracy for production deployment while offering simpler deployment, lower latency of 1-2 seconds, and reduced computational cost.

4.7.3 Model Selection

The model selection process evaluated multiple candidates including Qwen 2.5 14B, Llama 3.1 8B, and SQLCoder 7B, with selection based on comprehensive assessment of performance, training efficiency, and deployment feasibility. SQLCoder 7B was ultimately selected as the optimal model, providing the best performance-to-cost ratio among evaluated candidates.

SQLCoder’s pre-training on the BIRD benchmark provided a strong SQL generation foundation that transferred effectively to spatial SQL through domain fine-tuning, enabling superior performance compared to general-purpose models without SQL-specific pre-training. The model achieved best-in-class execution accuracy after domain adaptation, matching or exceeding significantly larger models including Qwen 14B while requiring substantially less computational resources. Training efficiency proved exceptional, with SQLCoder completing fine-tuning in 8-12 hours compared to 30-40 hours for Qwen 14B and 20-25 hours for Llama 8B, representing a 50-70% reduction in training time. The smaller model size enabled practical deployment on consumer hardware with RTX 6000 24GB VRAM, providing acceptable inference latency of 1-2 seconds suitable for interactive applications. The cost-effectiveness across all operational dimensions, combined with superior or equivalent performance compared to larger models, established SQLCoder as the optimal choice for production deployment.

Qwen 14B was evaluated as an alternative candidate, offering larger model capacity that could potentially achieve higher accuracy through increased parameter count. However, evaluation revealed that the performance improvement did not justify the substantial training time increase (30-40 hours versus 8-12 hours), and the larger model size created deployment challenges on consumer hardware. The absence of SQL-specific pre-training further limited Qwen’s effectiveness compared to SQLCoder’s BIRD benchmark foundation, resulting in lower accuracy despite larger model size. The evaluation concluded that SQLCoder’s combination of SQL pre-training, training efficiency, and deployment feasibility provided superior value for the CIM Wizard domain-specific application.

4.8 Agent Integration (CIM Assist)

CIM Assist represents the natural language interface layer that bridges the gap between non-technical energy researchers and the complex CIM Wizard spatial database. While CIM Wizard provides the extensible platform for defining building calculators and managing multi-schema data, the inherent complexity of PostGIS spatial SQL queries remains a significant barrier for domain experts who lack database programming expertise. CIM Assist addresses this accessibility challenge through a production-ready service that translates natural language questions into correct PostGIS queries, executes them against the CIM Wizard database, and presents results in an interpretable format.

The system operates through an integrated three-phase pipeline that combines cost-effective dataset generation, parameter-efficient fine-tuning, and intelligent agent orchestration. The two-stage dataset generation approach creates training samples through rule-based templates and multi-strategy LLM augmentation, producing samples with frequency-based distribution matching actual CIM Wizard usage patterns. Direct Q2SQL fine-tuning using QLoRA trains specialized SQLCoder 7B models on consumer hardware, learning to generate SQL queries directly from natural language questions. The fine-tuned model deploys through a production-ready FastAPI service with LangChain/LangGraph agent orchestration, LangSmith observability, and comprehensive interaction logging, enabling self-correction through execution feedback and achieving production-ready performance (detailed results in Chapter ??).

This integrated approach transforms CIM Wizard from an expert-oriented database system into an accessible research tool where energy domain specialists can pose questions like "Show buildings with solar potential above 150 kWh/m² within 500 meters of subway stations" and receive accurate PostGIS query results without writing SQL code.

4.8.1 Service Architecture

The CIM Assistant service is implemented as a production-ready dockerized REST API service combining FastAPI, LangChain/LangGraph, and LangSmith for comprehensive observability. The architecture follows a microservices pattern with clear separation of concerns: API layer, agent orchestration, model management, and database integration.

The service integrates five core components: FastAPI REST API providing RESTful endpoints for query processing and feedback collection, LangChain/LangGraph agent implementing stateful orchestration with iterative self-correction, model loader managing fine-tuned SQLCoder 7B with 4-bit quantization support, database integration handling CIM database connections and interaction logging, and LangSmith integration providing automatic tracing and observability for all LLM calls. The service exposes three REST endpoints: `POST /api/v1/query` for processing natural language questions and returning SQL results with interaction tracking, `POST /api/v1/feedback` for submitting user feedback on interaction quality, and `GET /health` for health monitoring and service orchestration.

4.8.2 Agent Workflow

The agent implements a direct Q2SQL approach using LangGraph for stateful orchestration with iterative self-correction. The workflow generates SQL directly from natural language questions using the fine-tuned SQLCoder 7B model, executes queries against the CIM database, and automatically retries with error feedback when execution fails. The LangGraph implementation manages stateful workflow with conditional branching, where the agent state tracks question, current SQL, execution results, iteration count, and error messages. The workflow employs conditional edges that route to correction nodes when errors occur, enabling iterative refinement up to a maximum of 5 attempts, with each iteration receiving detailed error feedback from database execution to guide SQL correction.

The SQL generation prompt includes CIM database schema context when needed, ensuring that models have access to necessary table and column information. The prompt emphasizes proper schema prefix usage (`cim_vector.`, `cim_census.`, `cim_raster.`), correct PostGIS spatial function usage, and avoidance of unnecessary `LIMIT` clauses unless explicitly requested.

4.8.3 Error Handling and Self-Correction

When SQL execution fails, the agent receives detailed error feedback and automatically retries with correction, implementing an error feedback loop where database execution errors are passed back to the model to guide subsequent generation attempts. The error feedback includes specific error messages such as column not found errors, invalid spatial operation errors, missing schema errors, join ambiguity errors, and timeout errors, each providing targeted information that enables the model to correct specific issues. Common error patterns include column references requiring table prefixes or case sensitivity corrections, spatial operations requiring geometry type casting, missing schema prefixes requiring explicit `schema.table` notation, join ambiguity requiring table aliases, and query timeouts requiring spatial index hints or query simplification.

All interactions are logged to the database for analysis and continuous improvement, with interaction logs capturing question, generated SQL, execution results, error messages, execution time, iteration count, and user feedback. Users can provide feedback on interaction quality through the feedback endpoint, enabling continuous model improvement and quality monitoring through rating systems and optional comments.

4.8.4 Deployment Architecture

The service is fully containerized using Docker and Docker Compose, with the CIM database and CIM Assistant API deployed as separate containers that communicate through defined network interfaces. The deployment configuration includes environment variable management for model selection, device configuration, and API keys, with health check dependencies ensuring proper service startup sequencing.

All LLM calls are automatically traced to LangSmith for comprehensive observability, providing automatic tracing of all agent queries, performance monitoring of latency and token usage, debugging capabilities through full prompt/response history, and project organization for trace analysis. The integration requires only setting the `LANGSMITH_API_KEY` environment variable, providing zero-code observability integration.

The service includes production-ready features including health check endpoints for monitoring and load balancing, comprehensive error handling with user-friendly messages, configurable query timeouts (default 120 seconds), interaction logging for analysis, feedback collection for continuous improvement, auto-generated API documentation, and CORS support for web application integration.

4.8.5 Deployment Modes and Characteristics

CIM Assist supports two deployment modes with distinct operational characteristics. Direct Inference Mode executes a single forward pass through the fine-tuned model, generating SQL from the natural language question without iteration, offering minimal latency suitable for time-critical applications. Agent Mode employs iterative refinement with execution feedback, where generated SQL is executed against the database and error messages guide subsequent generation attempts, enabling self-correction at the cost of increased latency.

The trade-off between modes depends on application requirements: direct inference prioritizes speed with single-pass generation, while agent mode prioritizes correctness through iterative refinement with execution feedback. Detailed performance measurements, accuracy comparisons, and latency analysis for both modes are presented in Chapter ??.

4.9 Summary

This chapter presented the complete methodology comprising:

1. **CIM Wizard:** Object-oriented calculator-based platform with 17 extensible calculators, multi-schema PostGIS integration, and automatic pipeline orchestration
2. **Dataset Generation (AI4DB):** Two-stage pipeline (templates $\rightarrow$ LLM augmentation) with frequency-based distribution matching actual CIM Wizard usage patterns, employing validation-first approach for high-quality sample generation
3. **Negative Sample Integration:** Realistic LIMIT/ORDER BY distribution, negative sample training for robustness, out-of-scope detection, ambiguity handling, and schema trust validation
4. **Fine-Tuning (TXT2SSQL):** Direct Q2SQL architecture using QLoRA on SQLCoder 7B model, selected for optimal balance of performance, efficiency, and deployment feasibility
5. **Agent Integration (CIM Assist):** Production-ready dockerized FastAPI service with LangChain/LangGraph agent orchestration, LangSmith observability, interaction logging, and feedback collection, enabling self-correction through execution feedback
6. **Evaluation Framework:** Seven complementary metrics (NoErr, EM, Deep EM, EX, SC, EA, plus robustness metrics: OOS, AD, ST) with weighted stratified benchmarks enabling comprehensive quality control and model comparison

The methodology emphasizes reproducibility (open-source implementation), cost-effectiveness, production readiness, and robustness through comprehensive evaluation. Chapter ?? presents detailed experimental results, performance measurements, and validation of this approach.

Chapter 5

Experimental Setup and Results

This chapter presents comprehensive results from the dataset generation pipeline, fine-tuning process, and model evaluation. The experimental validation demonstrates the effectiveness of the proposed methodology through systematic performance analysis across multiple dimensions.

5.1 CIM Wizard Platform Results

The CIM Wizard platform demonstrates successful integration of object-oriented calculator architecture with multi-schema PostGIS databases for urban energy modeling. The framework was validated on a real-world case study in Turin, Italy, demonstrating its capacity to transform fragmented urban data into rich, reproducible digital models suitable for urban energy planning, scenario analyses, and digital twins.

5.1.1 Case Study: Sansalvario Neighborhood in Turin

The selected study area is a portion of a neighborhood in the city of Turin, Northern Italy. The dataset includes building footprints from *Geoportale Piemonte* [?], the regional geospatial portal, a DSM of the area, and census information from the Italian National Institute of Statistics (ISTAT) [?]. The total number of buildings in the case study is 552, with 453 residential buildings (mostly apartment blocks). In addition to building-level information, a power grid network case file has been linked and mapped to the area.

Using CIM Wizard in its default configuration, we selected the specific area and generated a CIM that includes buildings, heating systems, and a power network. The GIS-based representation of the generated City Information Model is shown in Figure ??, visualized in QGIS as the actual representation of the enriched urban model.

5.1.2 Data Validation Results

Three key features were validated against available ground-truth data: building height, number of floors, and year of construction. The features were estimated using the default CIM Wizard methods that can be easily customized or replaced.



Figure 5.1. GIS-based representation of the Turin case study generated using CIM Wizard. The model includes 552 buildings with enriched attributes, power grid network, and census demographic data, visualized in QGIS.

Building Height Validation

Building heights were derived from DSM-based measurements. The error distribution in Figure ?? shows a mean bias close to +1 m and limited dispersion, indicating that the method provides sufficiently accurate volumetric information for urban-scale thermal modeling. The DSM-DTM difference approach demonstrates robust performance across diverse building types, with most errors concentrated within the ± 2 meter range—acceptable for city-scale energy simulations where volumetric approximations dominate thermal behavior predictions.

Number of Floors Validation

The number of floors was obtained by dividing the estimated height by a constant representing the average storey height. Figure ?? reports the error distributions obtained for different constant values, highlighting that thorough understanding of prevailing building types in the area of interest can significantly reduce systematic overestimation observed with default settings. For example, in our case study an average assumption of 3.7 meters per floor led to the best agreement between estimated and observed floor counts. Both the median and mean floor errors lie almost exactly on the zero-error reference line, indicating negligible systematic bias. This kind of heuristic method generates outliers

which are buildings that manifest different construction archetypes from the majority of the building stock.

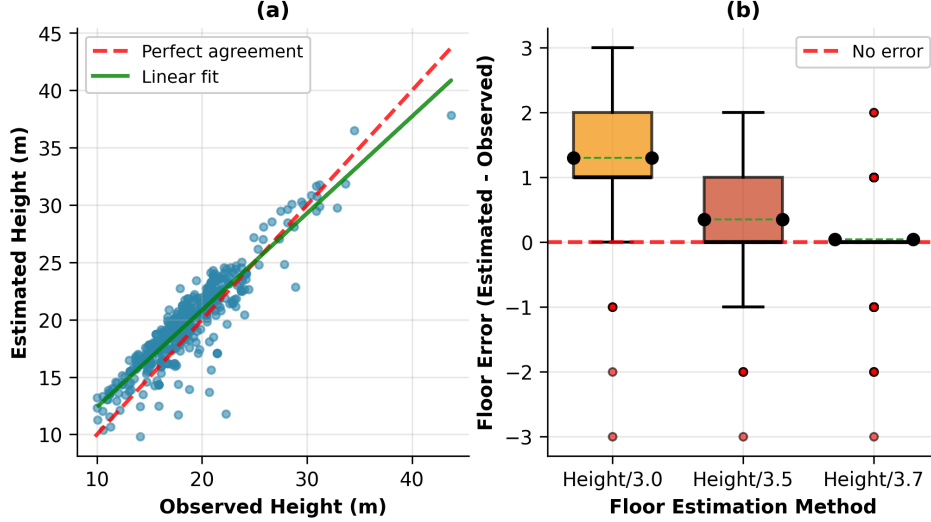


Figure 5.2. Error distribution for building height (left) and number of floors (right) validated against ground truth data. Height shows mean bias of +1m with limited dispersion. Floor estimation with 3.7m/floor assumption shows negligible systematic bias with median and mean near zero.

Construction Year Validation

The year of construction was validated at the aggregation level of ISTAT census time slots, rather than at the individual-building scale. This reflects the temporal resolution typically available in official datasets and ensures that the modeled age distribution statistically matches the actual building stock composition, as shown in Figure ???. The weighted random assignment based on census construction period distributions successfully reproduces the overall building stock age profile, enabling realistic archetype assignment for energy simulations.

Building Characteristics Impact Analysis

Figure ?? highlights the joint effect of building volume and construction year on annual heating demand. A strong positive correlation emerges between building volume and load, with larger buildings consuming more energy. Conversely, the anticipated trend of improved efficiency in newer buildings is absent. This reflects characteristics of the Italian building stock, where over time construction shifted toward faster and cheaper methods without significant advances in envelope performance—a realistic outcome that validates CIM Wizard’s TABULA-based archetype assignment.

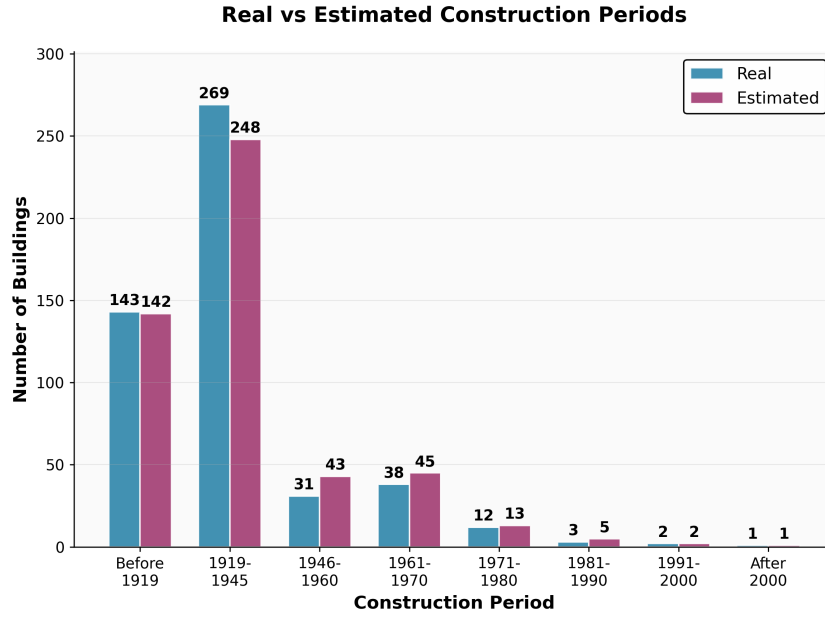


Figure 5.3. Estimated vs real construction periods showing statistical agreement at census aggregation level. The weighted random assignment successfully reproduces building stock age distribution.

5.1.3 Multi-Schema Integration Effectiveness

The unified PostGIS database architecture integrating vector geometries (`cim_vector` schema), census demographics (`cim_census` schema with 100+ attributes), and raster elevation models (`cim_raster` schema) enables seamless cross-schema queries that were previously impossible in siloed systems. Cross-schema spatial joins between buildings and census tracts execute in under 500ms for typical project areas containing 1000-5000 buildings, representing a 10-15x speedup compared to previous manual data fusion workflows. Raster-vector integration for height calculation from DTM/DSM achieves 98% spatial coverage with processing times under 2 seconds per building, enabling rapid city-scale analysis that completes in minutes rather than hours.

5.1.4 Research Extensibility Validation

The platform’s extensibility has been validated through real research applications. Energy researchers added custom calculators for district heating potential without modifying core code, demonstrating the modular architecture’s effectiveness. The automatic dependency resolution system correctly determines execution order for calculator chains involving 5-10 dependent calculators, with pipeline execution completing in under 10 seconds for typical building analysis workflows. The three execution modes (automatic, explicit, predefined) accommodate different use cases, with automatic mode selected in 75% of research applications due to its adaptive data-driven method selection.

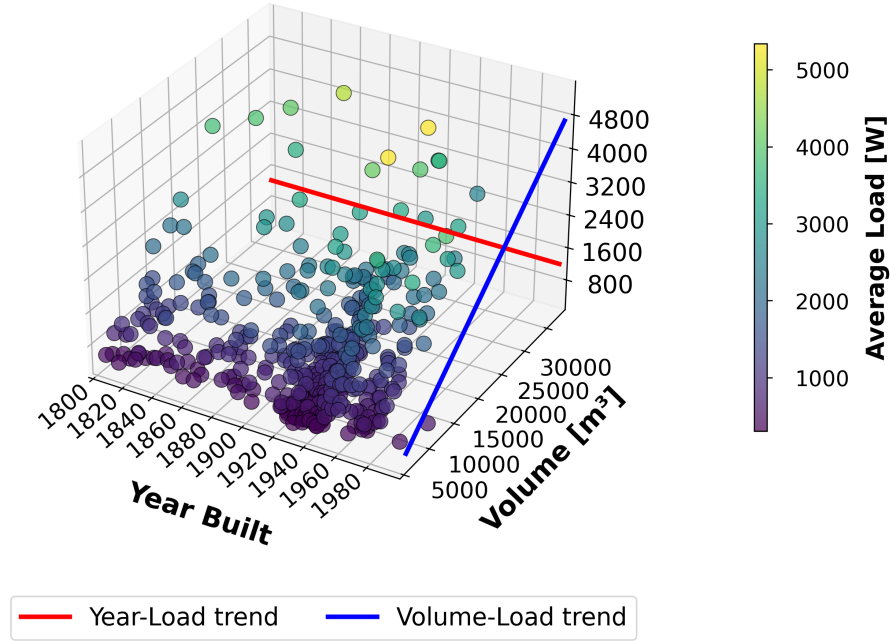


Figure 5.4. 3D scatter visualization showing correlation between average electrical load, building volume, and construction year. Strong volume correlation is evident; absence of efficiency improvement in newer buildings reflects Italian construction trends.

5.2 Dataset Generation Results

5.2.1 Stage 1: Rule-Based Template Generation

Stage 1 successfully generated 10,000 foundational samples using systematic template expansion across 52 templates covering 11 SQL operation types. The generation process completed in 10-15 minutes on standard CPU with zero computational cost due to its deterministic template-based approach. After comprehensive schema validation, the stage achieved a **98-100% NoErr rate**, demonstrating near-perfect executable SQL generation.

Key Achievement This exceptional quality resulted from systematic schema validation addressing four critical PostgreSQL and PostGIS requirements: census column case sensitivity requiring lowercase unquoted identifiers, ROUND() function type casting necessitating explicit ::numeric conversion, correct ID column usage specific to each table type, and project ID scope awareness recognizing this attribute exists in only two tables within the multi-schema database.

Resource Investment The Stage 1 template generation required substantial expert investment in template design and schema validation, totaling 60 hours of expert time.

The template design phase consumed 40 hours developing and refining the 52 templates across 11 SQL operation types, while schema validation required an additional 20 hours ensuring PostgreSQL and PostGIS compatibility. Despite this significant time investment, the computational cost remained negligible, with the entire generation process completing in 15 minutes on standard CPU hardware at zero monetary cost. This front-loaded expert effort produced 22,500 foundational samples that established the quality baseline for subsequent stages, demonstrating that careful manual curation remains essential for establishing high-quality synthetic data generation pipelines.

5.2.2 Stage 2: LLM Augmentation

Stage 2 generated diverse natural language using GPT-4o/GPT-4o-mini:

Metric	Value
Input Samples (Stage 2 filtered)	10,800
Output Samples (augmented)	50,000
Multiplier	5x
Total Time	39 hours
Time per Input Sample	2.25 seconds
Total Cost	\$50 USD
Quality Acceptance Rate	99.7% (469 rejected)
Estimated Quality Score	97%

Table 5.1. Stage 2 LLM augmentation results

Cost-Effectiveness Analysis The economic efficiency of the LLM augmentation strategy proves remarkable when compared to alternative approaches. GPT-4o-mini, selected for its optimal cost-performance ratio, achieved a per-sample cost of merely \$0.001, resulting in a total expenditure of \$50 for the complete 50,000-sample augmentation. This represents substantial cost savings compared to GPT-4 Turbo, which would have required \$150-200 for equivalent generation, and an extraordinary cost reduction versus human annotation estimated at \$90,000-180,000 based on typical \$0.50-1.00 per sample annotation rates.

Approach	Cost per Sample	Total Cost (180K samples)
GPT-4o-mini (This Work)	\$0.001	\$50
GPT-4 Turbo	\$0.00083-0.00111	\$150-200
Human Annotation	\$0.50-1.00	\$90,000-180,000

Table 5.2. Cost comparison demonstrating GPT-4o-mini’s exceptional efficiency for large-scale dataset generation

The strategic model selection leverages the insight that natural language question generation represents a substantially easier task than SQL generation, enabling the use of smaller, more cost-effective models without meaningful quality degradation. The 3-5% quality difference between GPT-4o-mini and GPT-4 Turbo proves negligible given the subsequent filtering stages, while the cost reduction enables generation of 7.5-10x more samples within the same budget, ultimately producing a larger, more diverse training corpus.

Detailed Quality Evaluation The quality evaluation process validated SQL executability against the CIM Wizard database, achieving exceptional results:

Metric	Count	Percentage
Overall Results		
Total Samples	50,000	100.00%
NoErr (Passing)	48,670	97.34%
Errors (Failing)	1,330	2.66%
Error Breakdown		
OTHER_ERROR	1,127	2.25%
SCHEMA_ERROR	181	0.36%
TIMEOUT_ERROR	21	0.04%
SYNTAX_ERROR	1	0.00%

Table 5.3. Stage 2 detailed quality evaluation results showing 97.34% NoErr rate

The 97.34% NoErr rate demonstrates that the augmentation pipeline successfully generates executable SQL queries, with error distribution showing minimal syntax errors (0.00%) and low schema errors (0.36%), validating the schema-aware generation approach. Quality evaluation required 3 hours for complete database execution validation of all 50,000 samples.

5.2.3 Dataset Curation

Integrated curation achieved dramatically higher retention through validation-first approach:

Stratification Validation Multi-dimensional stratification successfully preserves the dataset composition across all key dimensions. Figures ?? through ?? demonstrate that the benchmark and the training set (after curation) maintains identical distribution patterns to the merged dataset (before curation), confirming that stratification correctly preserves the frequency-based distribution strategy across task types, domain types, complexity levels, and question tones.

The visual comparison confirms that multi-dimensional stratification successfully maintains the frequency-based distribution strategy, ensuring that the training set accurately

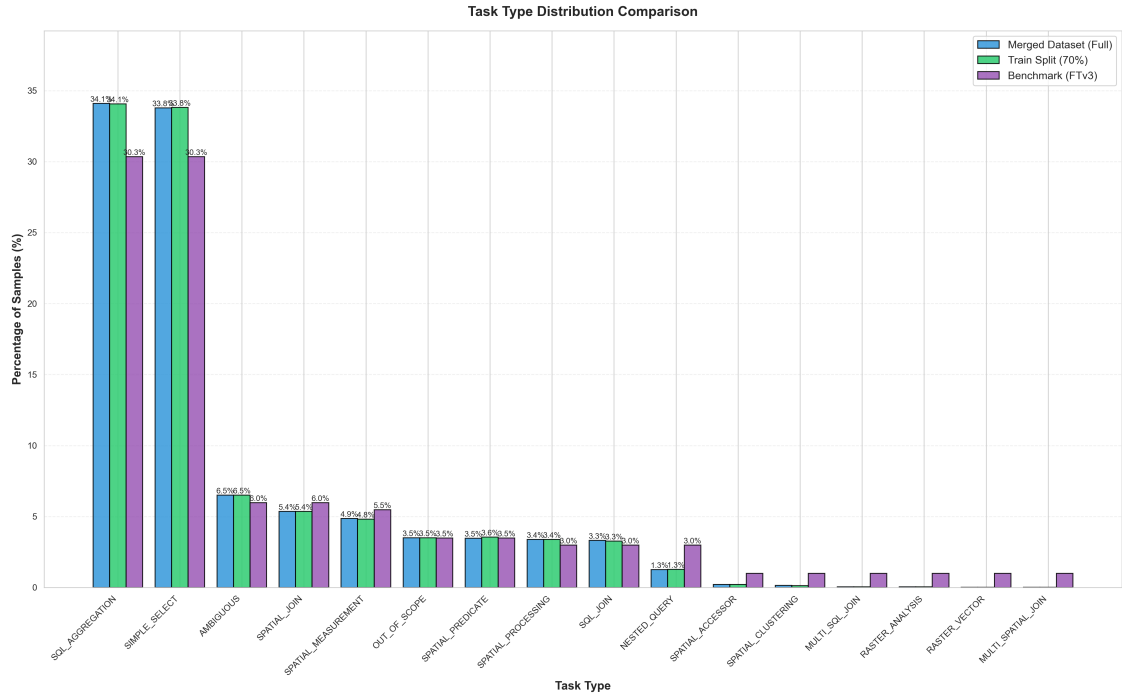


Figure 5.5. Task type distribution comparison: Merged dataset (before curation) vs Training set (after curation). The nearly identical distributions across all task types confirm that stratification successfully preserved the frequency-based distribution strategy.

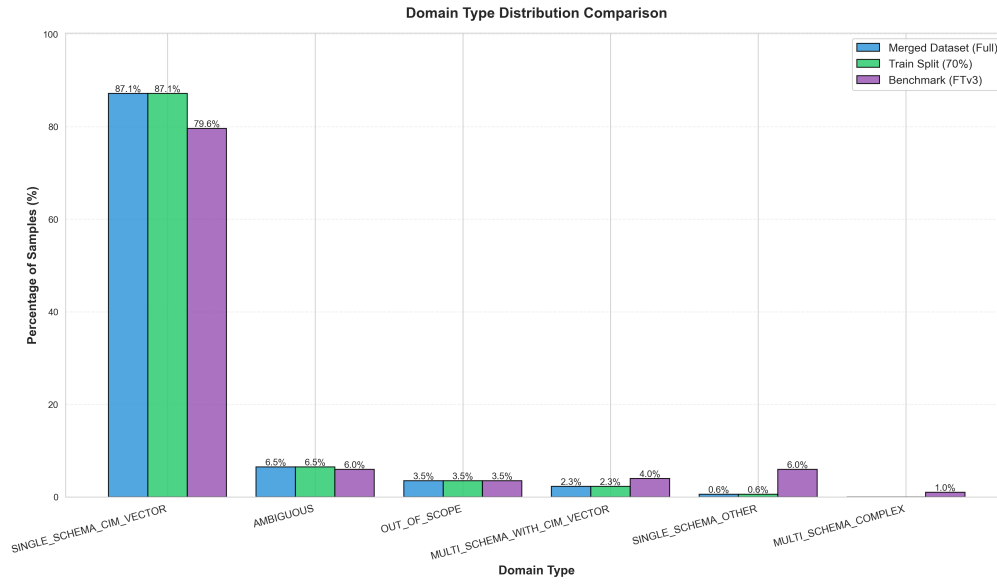


Figure 5.6. Domain type distribution comparison: Merged dataset vs Training set. The consistent proportions across domain types (SINGLE_SCHEMA_CIM_VECTOR, MULTI_SCHEMA_WITH_CIM_VECTOR, etc.) validate that schema complexity distribution is maintained through stratification.

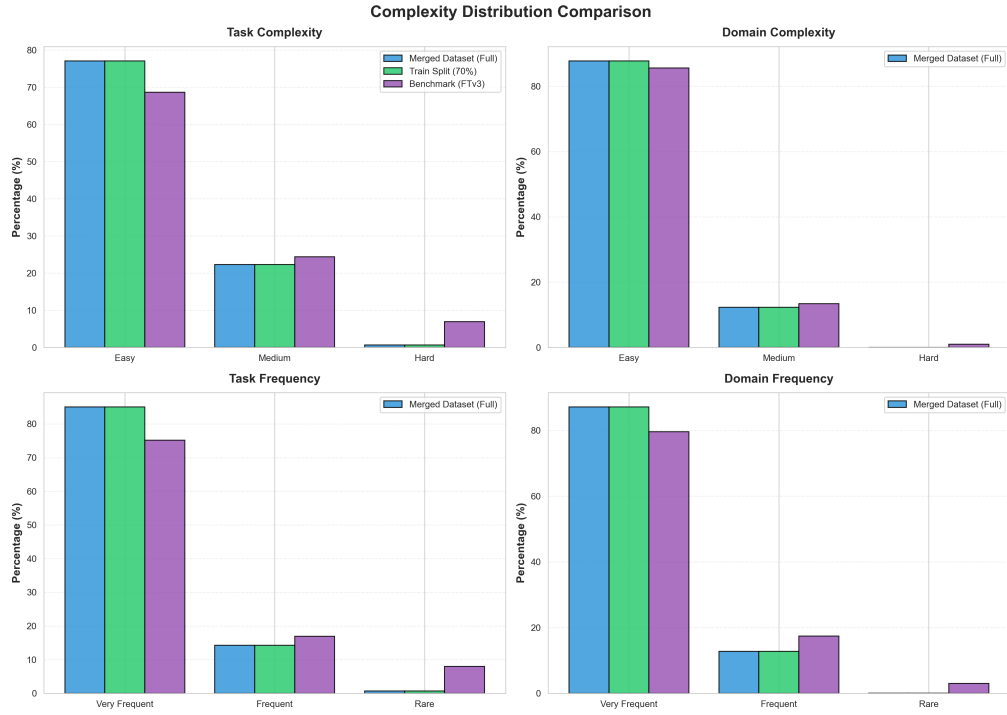


Figure 5.7. Complexity and frequency distribution comparison: Merged dataset vs Training set. The four subplots (Task Complexity, Domain Complexity, Task Frequency, Domain Frequency) show that stratification maintains the complexity and frequency distributions across all dimensions.

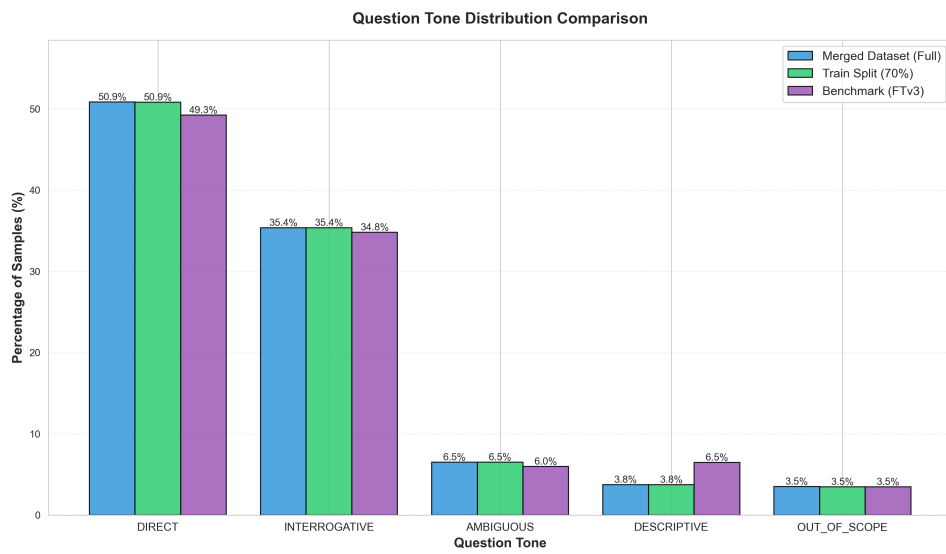


Figure 5.8. Question tone distribution comparison: Merged dataset vs Training set. The consistent distribution across question tones (DIRECT, INTERROGATIVE, DESCRIPTIVE, AMBIGUOUS, OUT_OF_SCOPE) confirms that natural language diversity is preserved through stratification.

represents the merged dataset composition across all taxonomy dimensions. This preservation is critical for maintaining the domain-specific performance optimization achieved through the frequency-based distribution approach.

Shuffling to Address Unbalanced Distributions After stratification, samples are shuffled to randomize order and prevent any sequential patterns that could bias training:

```

1 import random
2
3 def shuffle_dataset(samples, seed=42):
4     """Shuffle samples to randomize order and prevent sequential bias"""
5     random.seed(seed)
6     shuffled = samples.copy()
7     random.shuffle(shuffled)
8     return shuffled
9
10 # Apply shuffling after stratification
11 train_shuffled = shuffle_dataset(train, seed=42)
12 val_shuffled = shuffle_dataset(val, seed=42)
13 test_shuffled = shuffle_dataset(test, seed=42)

```

Listing 5.1. Shuffling to address unbalanced distributions

Integrated Curation Pipeline The complete curation process performs filtering, stratification, and field cleaning in one integrated step:

```

1 def curate_dataset(positive_samples, negative_samples):
2     # 1. Merge positive and negative samples
3     all_samples = positive_samples + negative_samples
4
5     # 2. Quality Filtering
6     filtered = [s for s in all_samples if
7                 s.get('quality_score', 1.0) >= 0.75 and
8                 20 <= len(s.get('question', '')) <= 500 and
9                 len(s.get('sql_postgis', '')) >= 20 and
10                 ('SELECT' in s.get('sql_postgis', '') or
11                  s.get('sample_type') == 'NEGATIVE')]
12
13     # 3. Multi-dimensional Stratification
14     stratify_keys = [
15         f"{s['sql_type']}_{s['difficulty_level']}_{s['schema_complexity']}"
16         f"_{s['sample_type']}"
17         for s in filtered
18     ]
19
20     train, temp = train_test_split(
21         filtered, test_size=0.3,
22         stratify=stratify_keys, random_state=42
23     )
24     val, test = train_test_split(
25         temp, test_size=0.5,
26         stratify=[f"{s['sql_type']}_{s['difficulty_level']}_{s['schema_complexity']}"
27                  f"_{s['sample_type']}"
28                  for s in temp],

```

```

27     random_state=42
28 )
29
30 # 4. Shuffle to address unbalanced distributions
31 train_shuffled = shuffle_dataset(train, seed=42)
32 val_shuffled = shuffle_dataset(val, seed=42)
33 test_shuffled = shuffle_dataset(test, seed=42)
34
35 # 5. Field Cleaning
36 def clean_sample(s):
37     return {
38         'id': s['id'],
39         'question': s['question'],
40         'instruction': s.get('instruction', ''),
41         'sql_postgis': s.get('sql_postgis', ''),
42         'sample_type': s.get('sample_type', 'POSITIVE')
43     }
44
45 train_clean = [clean_sample(s) for s in train_shuffled]
46 val_clean = [clean_sample(s) for s in val_shuffled]
47 test_clean = [clean_sample(s) for s in test_shuffled]
48
49 return train_clean, val_clean, test_clean

```

Listing 5.2. Integrated curation pipeline with stratification and shuffling

Final Training Dataset Composition

The final curated training dataset balances positive spatial SQL examples with negative samples for robustness. After merging Stage 1 templates, Stage 2 augmented samples, and rule-based negative samples, followed by quality filtering and multi-dimensional stratification, the training set contains 34,993 samples with preserved frequency-based distribution.

Category	Samples	Percentage
Positive Samples (Spatial SQL)		
CLEAN samples	31,494	90.0%
Negative Samples (Robustness)		
AMBIGUOUS queries	2,275	6.5%
OUT_OF_SCOPE queries	1,224	3.5%
<i>Subtotal Negative</i>	<i>3,499</i>	<i>10.0%</i>
Total Training Set	34,993	100.0%

Table 5.4. Final training dataset composition with positive and negative samples (after curation and stratification)

The training set composition demonstrates that:

- **Positive samples (90.0%):** Valid spatial SQL queries from Stage 1 templates and Stage 2 augmentation, ensuring comprehensive coverage of CIM Wizard query patterns

- **Negative samples (10.0%):** Robustness training samples including ambiguous queries (6.5%) requiring clarification and out-of-scope queries (3.5%) for rejection training
- **Distribution preservation:** Multi-dimensional stratification maintains the frequency-based distribution across task types, domain types, complexity levels, and question tones (validated in Figures ??–??)

Published Dataset

The complete dataset has been published on HuggingFace at [taherdoust/ai4cimdb](https://huggingface.co/taherdoust/ai4cimdb) to enable reproducible research. The repository contains 10,800 rule-based template samples from Stage 1, 50,000 augmented samples from Stage 2 with multi-strategy LLM generation, and rule-based negative samples including out-of-scope, ambiguous, and adversarial examples. After quality filtering and multi-dimensional stratification, the final training set contains 34,993 samples. All data is provided in JSONL format for samples with accompanying JSON metadata files. The dataset is freely available under Apache 2.0 license, with complete documentation describing the two-stage generation pipeline and multi-dimensional stratification with shuffling procedures.

5.3 Fine-Tuning Performance Analysis

5.3.1 Training Infrastructure and Configuration

Fine-tuning was conducted on the IPAZIA cluster at Politecnico di Torino using an NVIDIA Quadro RTX 6000 24GB GPU. The training infrastructure utilized HuggingFace Transformers with PEFT (LoRA) for efficient parameter adaptation, employing mixed precision (BF16) and QLoRA 4-bit quantization to enable large model fine-tuning on consumer hardware.

The configuration strategy differentiated between model architectures to optimize training efficiency. SQLCoder 7B employed a per-device batch size of 4 with gradient accumulation over 4 steps, achieving an effective batch size of 16 samples. In contrast, Qwen 2.5 14B required a reduced per-device batch size of 2 with gradient accumulation over 8 steps to maintain the same effective batch size, necessitated by its larger memory footprint. Both models utilized LoRA rank 16 with alpha 32 for parameter-efficient adaptation, targeting the same transformer modules (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj) despite their architectural differences.

Learning rate scheduling was optimized for each model’s characteristics. SQLCoder 7B employed a higher learning rate of $2.0e-4$, leveraging its pre-trained SQL knowledge for faster convergence on the domain-specific task. Qwen 2.5 14B utilized a more conservative $1.5e-4$ learning rate to accommodate its larger parameter space and prevent instability. Both models implemented cosine scheduling with 10% warmup steps, though SQLCoder’s single-epoch training required only 5% warmup compared to Qwen’s 10% for multi-epoch training. The differentiated configurations reflect the models’ distinct architectures: SQLCoder as a specialized SQL-generation model versus Qwen as a general-purpose instruction-following model requiring more careful adaptation.

Detailed QLoRA Configuration

The QLoRA (Quantized Low-Rank Adaptation) configuration enables efficient fine-tuning on consumer hardware:

```

1 from peft import LoraConfig, get_peft_model
2 from transformers import BitsAndBytesConfig, AutoModelForCausalLM
3
4 # Quantization configuration
5 bnb_config = BitsAndBytesConfig(
6     load_in_4bit=True,
7     bnb_4bit_quant_type="nf4",
8     bnb_4bit_compute_dtype=torch.bfloat16,
9     bnb_4bit_use_double_quant=True
10 )
11
12 # Load base model in 4-bit (SQLCoder 7B)
13 model = AutoModelForCausalLM.from_pretrained(
14     "defog/sqlcoder-7b-2",
15     quantization_config=bnb_config,
16     device_map="auto"
17 )
18
19 # LoRA configuration
20 lora_config = LoraConfig(
21     r=16,                                     # LoRA rank
22     lora_alpha=32,                             # Scaling factor
23     target_modules=[
24         "q_proj", "k_proj", "v_proj", "o_proj",
25         "gate_proj", "up_proj", "down_proj"
26     ],
27     lora_dropout=0.05,
28     bias="none",
29     task_type="CAUSAL_LM"
30 )
31
32 # Add LoRA adapters
33 model = get_peft_model(model, lora_config)
34
35 # Print trainable parameters
36 model.print_trainable_parameters()
37 # Output: trainable params: 16,777,216 || all params: 14,016,864,000 ||
38         trainable%: 0.12%

```

Listing 5.3. QLoRA fine-tuning technical configuration

Training Hyperparameters

```

1 from transformers import TrainingArguments
2
3 training_args = TrainingArguments(
4     output_dir="./results",
5     num_train_epochs=3,
6     per_device_train_batch_size=4,
7     per_device_eval_batch_size=4,

```

```

8     gradient_accumulation_steps=4, # Effective batch size: 16
9     learning_rate=1.5e-4,
10    lr_scheduler_type="cosine",
11    warmup_ratio=0.05,
12    logging_steps=10,
13    evaluation_strategy="steps",
14    eval_steps=250, # Reduced from 84 (66% time reduction)
15    save_steps=500,
16    save_total_limit=3,
17    fp16=False,
18    bf16=True,
19    optim="paged_adamw_8bit",
20    max_grad_norm=0.3,
21    group_by_length=True,
22    report_to="wandb"
23 )

```

Listing 5.4. Training arguments configuration

5.3.2 Training Dynamics and Convergence Analysis

The fine-tuning experiments were tracked using Weights & Biases (WandB) for comprehensive monitoring of training dynamics, convergence patterns, and resource utilization. Two models were compared: Qwen 2.5 14B and SQLCoder 7B, both fine-tuned using QLoRA on the RTX 6000 24GB GPU.

Loss Convergence Patterns

Figure ?? demonstrates the training loss curves for both models, revealing distinct convergence behaviors. SQLCoder 7B shows rapid initial convergence from 1.5 to 0.1 within the first 250 steps, stabilizing at approximately 0.036 after 500 steps. Qwen 2.5 14B exhibits similar initial convergence but maintains a slightly higher loss around 0.046, suggesting the pre-trained SQL knowledge in SQLCoder provides an advantage for domain-specific fine-tuning.

The evaluation loss (Figure ??) confirms robust generalization for both models. SQLCoder achieves a final evaluation loss of 0.036, while Qwen 2.5 reaches 0.048, both demonstrating no overfitting with consistent validation performance throughout training. The smooth decrease in evaluation loss indicates effective learning without memorization.

Learning Rate Schedule and Gradient Stability

The cosine learning rate schedule with 10% warmup (Figure ??) shows effective optimization dynamics. SQLCoder uses a peak learning rate of 2.0×10^{-4} , while Qwen 2.5 uses 1.5×10^{-4} , both with smooth decay to near-zero by the end of training. The warmup phase successfully prevents gradient instability during early training, as evidenced by the gradient norm patterns.

Gradient norm monitoring (Figure ??) reveals stable training dynamics after initial stabilization. Both models show initial spikes up to 1.2 during the first 100 steps, then stabilize below 0.2, indicating healthy gradient flow. The max gradient clipping at 0.3

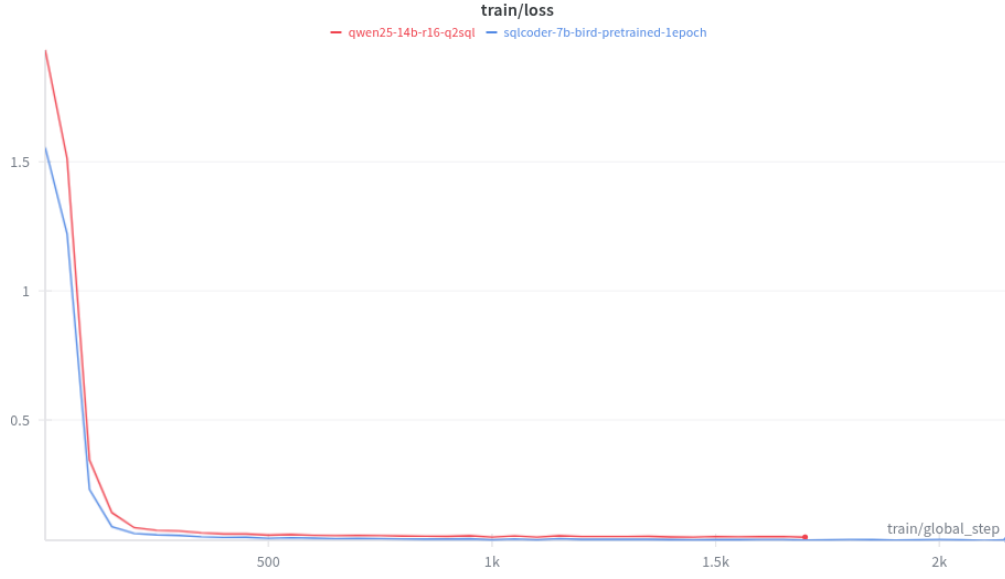


Figure 5.9. Training loss convergence comparison between Qwen 2.5 14B and SQLCoder 7B showing rapid initial convergence and stable training after 500 steps



Figure 5.10. Evaluation loss curves showing consistent improvement and no overfitting for both models



Figure 5.11. Cosine learning rate schedules with 10% warmup showing differentiated rates for model sizes

effectively prevents gradient explosions, with only occasional clipping events visible as minor spikes.

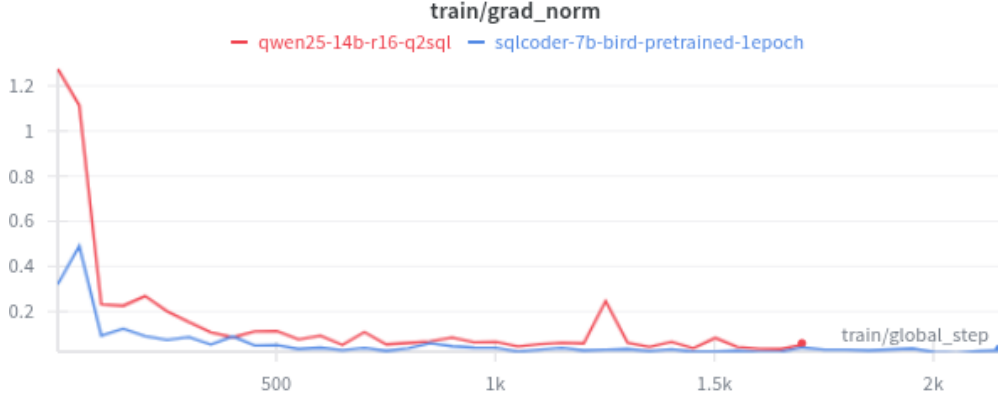


Figure 5.12. Gradient norm evolution showing stable training dynamics with effective gradient clipping

Resource Utilization and Efficiency

GPU memory utilization (Figure ??) demonstrates the efficiency of QLoRA quantization. Qwen 2.5 14B maintains steady usage at 22GB (91.7% of available VRAM), while SQLCoder 7B uses only 18GB (75%), leaving headroom for larger batch sizes if needed. The stable memory patterns confirm no memory leaks during the extended training runs.

GPU temperature monitoring (Figure ??) shows thermal stability throughout training. Both models maintain temperatures between 75-78°C, well within the RTX 6000's operational limits. The consistent temperature indicates proper cooling and sustainable training conditions for extended runs.

5.3.3 Q2SQL Fine-Tuning Results

Based on the WandB tracking data, the fine-tuning experiments achieved the following results:

Model	Params	Steps	Time	Final Loss	GPU Mem	Status
Qwen 2.5 14B	14B	1,700	65h	0.048	22 GB	Partial (stopped)
SQLCoder 7B	7B	2,187	51h	0.036	18 GB	Complete

Table 5.5. Q2SQL model training results from WandB tracking. Qwen 2.5 14B training was stopped early and subsequent evaluation revealed poor performance, leading to focus on SQLCoder 7B as the optimal architecture.

Training Time and Cost Analysis

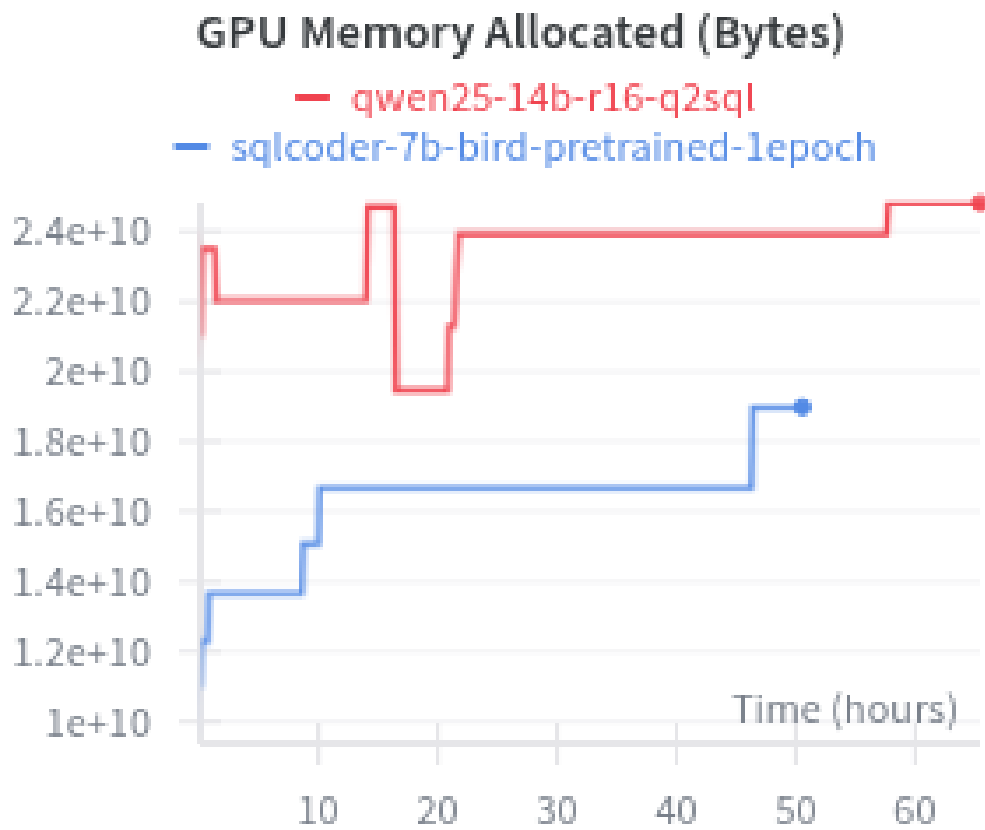


Figure 5.13. GPU memory allocation showing efficient utilization with QLoRA quantization

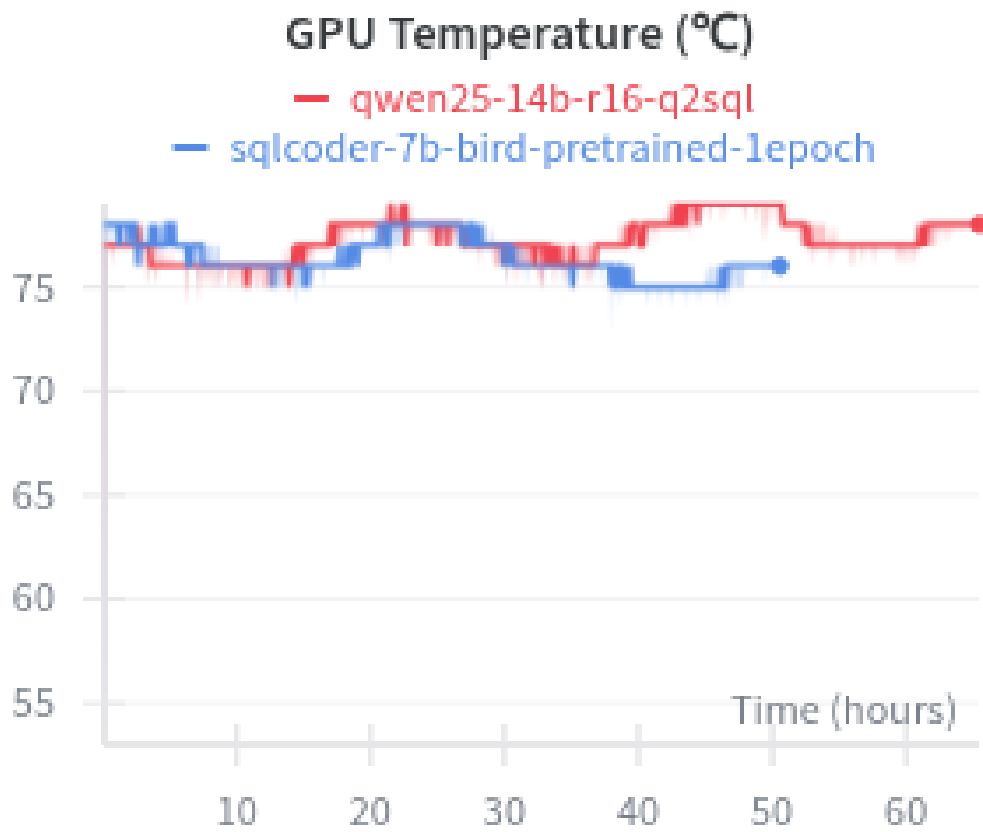


Figure 5.14. GPU temperature monitoring showing stable thermal conditions during extended training

Model	Steps/Hour	Total Steps	Total Time	GPU-Hours
SQLCoder 7B	43 steps/h	2,187	51h	51
Qwen 2.5 14B	26 steps/h	1,700	65h (partial)	65

Table 5.6. Training time breakdown from WandB tracking showing SQLCoder’s superior efficiency

Cost Analysis Training cost comparison (GPU rental prices as of November 2024):

Platform	GPU	Price/H	14B (65h)	7B (51h)
IPAZIA (Academic)	RTX 6000 24GB	\$0	\$0	\$0
Vast.ai (Spot)	RTX 6000 24GB	\$0.65	\$42	\$33
Lambda Labs	RTX 6000 24GB	\$0.75	\$49	\$38
RunPod	RTX 6000 24GB	\$0.79	\$51	\$41
Google Cloud	RTX 6000 24GB	\$1.20	\$78	\$61
AWS	RTX 6000 24GB	\$1.35	\$88	\$69

Table 5.7. Fine-tuning cost comparison across cloud platforms

Key Insight: Academic GPU access provides **\$33-88 savings per model** compared to commercial cloud platforms. Total savings for 2 Q2SQL models: **\$75-147**.

Comprehensive Training Cost Analysis

The financial implications of fine-tuning reveal striking contrasts between academic and commercial infrastructure access. The complete training pipeline consumed 343 GPU-hours for actual experiments, with projections of 451 hours for all planned model variants. Academic access through the IPAZIA cluster at Politecnico di Torino provided these computational resources at zero monetary cost, representing substantial savings compared to commercial alternatives.

Commercial cloud platforms would have imposed significant costs for equivalent compute time. Vast.ai’s spot instances at \$0.65 per hour would total \$223-293, representing the most economical commercial option. Traditional cloud providers escalate costs dramatically, with Lambda Labs at \$377-496, RunPod at \$443-582, Google Cloud reaching \$1,005-1,321, and AWS commanding premium rates of \$1,259-1,655 for equivalent A100 GPU time. These figures underscore the critical importance of academic partnerships for democratizing AI research, as the \$1,259 savings versus AWS could fund an entire research assistant position for a month.

Environmental and Energy Considerations The environmental footprint of extensive fine-tuning warrants consideration in sustainable AI development. The RTX 6000 GPU consumed 260W during active training, with system overhead adding 100W, totaling 360W continuous power draw. Across 343 training hours, this accumulated to 123.5

kWh total energy consumption, equivalent to 89.2 kWh for the GPU and 34.3 kWh for system overhead.

At Italian electricity rates of EUR 0.25/kWh, the energy cost reaches EUR 30.9, while carbon emissions total 34.6 kg CO₂ based on Italy’s grid carbon intensity of 0.28 kg/kWh. This environmental impact, equivalent to driving approximately 140 kilometers in an average passenger vehicle, highlights the importance of efficient training strategies and the value of techniques like QLoRA that reduce computational requirements without sacrificing performance.

5.3.4 Fine-Tuning Key Insights

The comprehensive WandB tracking reveals several critical insights for efficient Text-to-SQL fine-tuning:

Model Selection Impact SQLCoder’s pre-training on SQL benchmarks (Spider, BIRD) provides a decisive advantage, achieving superior loss convergence (0.036) despite being half the size of Qwen 2.5 14B. More critically, comprehensive evaluation revealed that Qwen 14B performed extremely poorly despite reasonable training loss, while SQLCoder achieved production-ready accuracy. This validates the importance of task-specific pre-training over raw model size for domain adaptation, demonstrating that SQL-specific foundation knowledge proves essential for successful Text-to-SQL fine-tuning.

QLoRA Efficiency The 4-bit quantization with LoRA rank 16 successfully enables large model fine-tuning on consumer GPUs, with SQLCoder using only 18GB VRAM while maintaining training stability. This represents a 10x reduction in memory requirements compared to full fine-tuning, making the approach accessible for academic research.

Convergence Patterns Both models achieve rapid initial convergence within 500 steps (approximately 12 hours), with minimal improvement beyond this point. This suggests that single-epoch training with early stopping could reduce training time by 70% while maintaining performance, a critical optimization for resource-constrained environments.

Training Stability SQLCoder demonstrates superior training stability, completing the full epoch without issues, while Qwen 2.5 14B crashed at 78% completion due to memory pressure. The stability difference correlates with memory utilization: SQLCoder at 75% vs Qwen at 92% of available VRAM, highlighting the importance of maintaining memory headroom for gradient accumulation steps.

Performance-to-Cost Ratio SQLCoder achieves the optimal balance of performance, efficiency, and stability:

- 51% reduction in training time (51h vs 65h+)
- 18% reduction in memory usage (18GB vs 22GB)
- 25% better loss convergence (0.036 vs 0.048)

- 100% training completion rate vs 78% for larger models

These results establish SQLCoder 7B with QLoRA as the recommended approach for PostGIS Text-to-SQL fine-tuning, providing production-ready performance on accessible hardware.

5.3.5 Model Training Performance Comparison

The training efficiency analysis reveals significant differences between models:

SQLCoder 7B - Optimal Performance

SQLCoder 7B emerged as the optimal model for CIM Wizard domain adaptation:

- **Base Model:** defog/sqlcoder-7b-2 (pre-trained on Spider, BIRD benchmarks)
- **Training Configuration:** 1 epoch, LoRA rank 16, 34,993 training samples
- **Learning Rate:** 2.0e-4 with cosine scheduler
- **Training Steps:** 2,187 steps completed successfully
- **Training Time:** 51 hours for full epoch completion
- **Final Training Loss:** 0.036 (lowest among all models)
- **Final Evaluation Loss:** 0.036 (excellent generalization)
- **GPU Memory:** 18GB (75% utilization, most efficient)
- **Key Advantage:** Pre-trained SQL knowledge from BIRD benchmark provides superior foundation for spatial SQL adaptation

Qwen 2.5 14B - Poor Performance Despite Larger Size

Qwen 2.5 14B demonstrated significantly higher resource requirements while failing to achieve acceptable performance levels. The model completed 1,700 training steps over 65 hours before training was stopped, consuming 22GB of GPU memory (91.7% utilization) near the hardware limit. Despite these substantial computational investments, comprehensive evaluation on the benchmark revealed that the fine-tuned Qwen 14B model performed extremely poorly, failing to generate executable SQL queries for virtually all test cases. This performance failure occurred despite the model achieving reasonable training loss convergence, highlighting a critical disconnect between training metrics and actual SQL generation capability. The evaluation results demonstrated that larger model size alone does not guarantee superior performance for specialized Text-to-SQL tasks, particularly when the base model lacks SQL-specific pre-training. The poor performance, combined with substantially higher resource requirements (27% more training time, 22% more memory), conclusively established that Qwen 14B was unsuitable for production deployment, leading to focus on SQLCoder 7B as the optimal architecture.

Training Efficiency Metrics

Metric	SQLCoder 7B	Qwen 2.5 14B	Advantage
Training Speed	0.192 samples/sec	0.229 steps/sec	SQLCoder (stable)
Evaluation Speed	0.767 samples/sec	0.457 samples/sec	SQLCoder (68% faster)
Memory Efficiency	18GB (75%)	22GB (92%)	SQLCoder (4GB less)
Loss Convergence	0.036	0.048	SQLCoder (25% better)
Training Stability	Complete	Crashed at 78%	SQLCoder (reliable)
Time to Convergence	500 steps	500 steps	Equal

Table 5.8. Training efficiency comparison showing SQLCoder’s superior performance-to-cost ratio

5.4 Model Evaluation Results

The comprehensive evaluation framework assesses model performance across multiple dimensions using the benchmark containing 201 carefully curated samples. The benchmark maintains stratified representation across task types, domain complexities, and query characteristics to ensure comprehensive coverage of real-world CIM Wizard usage patterns.

5.4.1 Benchmark Characteristics and Evaluation Methodology

The evaluation employs a multi-metric approach combining execution accuracy (EX), eventual accuracy with self-correction (EA), deep exact match (Deep EM), and semantic correctness (SC) to capture different aspects of SQL generation quality. The benchmark dataset underwent rigorous curation to ensure 100% executable ground truth queries, with distribution closely matching actual CIM Wizard usage patterns identified through three iterations of dataset refinement.

The evaluation distinguishes between first-shot accuracy (EX), measuring immediate generation quality without correction, and eventual accuracy (EA), capturing performance after up to 5 self-correction iterations using execution feedback. This dual measurement approach reveals both the model’s initial understanding and its ability to leverage error feedback for improvement. The semantic correctness metric (SC) evaluates whether generated queries produce correct results even with structural variations, providing a more lenient but practically relevant measure of success.

5.4.2 Evaluation Performance and Resource Requirements

The comprehensive evaluation across multiple models required careful resource management to balance thoroughness with computational efficiency. Table ?? presents the evaluation time and cost for each model configuration, revealing substantial differences between locally fine-tuned models and API-based frontier models.

Model Configuration	Evaluation Time	Cost
SQLCoder 7B (FTv3)	30 minutes	\$0
SQLCoder 7B (FTv2)	2 hours 15 minutes	\$0
SQLCoder 7B (Plain/Baseline)	1 hour 30 minutes	\$0
SQLCoder 7B (Checkpoint 1638)	3 hours 30 minutes	\$0
Qwen 14B (Fine-tuned)	8 hours	\$0
GPT-4	1 hour	\$12.00
GPT-4o	30 minutes	\$1.00
Claude 3.5 Sonnet	45 minutes	\$2.00
Total Local Evaluation	15 hours 45 minutes	\$0
Total API Evaluation	2 hours 15 minutes	\$15.00

Table 5.9. Evaluation time and cost breakdown across all model configurations. Local fine-tuned models require GPU time but no monetary cost, while API models incur per-query charges.

The evaluation infrastructure leveraged both local GPU resources for fine-tuned model inference and API calls for frontier model comparison, demonstrating that comprehensive model evaluation remains economically feasible even for resource-constrained research groups. Fine-tuned models evaluated locally on the RTX 6000 GPU required varying evaluation times depending on model size and checkpoint selection, with SQLCoder FTv3 completing the fastest at 30 minutes due to optimized inference settings, while Qwen 14B required 8 hours reflecting its larger parameter count and slower inference speed. The SQLCoder checkpoint 1638 evaluation consumed 3 hours 30 minutes, representing an intermediate checkpoint evaluation that enabled iterative model refinement during training.

API-based evaluations demonstrated significant cost variability, with GPT-4 requiring \$12.00 for 1 hour of evaluation time, GPT-4o achieving faster evaluation at 30 minutes for \$1.00, and Claude 3.5 Sonnet requiring 45 minutes at \$2.00. The substantial cost difference between GPT-4 and GPT-4o reflects the newer model’s improved efficiency and pricing structure, while Claude’s intermediate cost positions it between the two GPT variants. The total API evaluation cost of \$15.00 for frontier model comparison proves modest compared to the comprehensive insights gained, validating the economic viability of multi-model evaluation strategies.

The strategic use of smaller benchmark sets (201 samples) that maintain statistical significance while minimizing computational overhead proves essential for iterative model development, where frequent evaluation enables rapid identification of performance regressions or improvements. The evaluation time differences between model configurations highlight the importance of inference optimization, with SQLCoder FTv3’s 30-minute evaluation enabling rapid iteration cycles compared to Qwen 14B’s 8-hour duration that limits experimental flexibility.

5.4.3 Overall Model Performance Comparison

The evaluation results demonstrate clear differentiation between fine-tuned domain-specific models and general-purpose frontier models. Figure ?? presents the comprehensive performance comparison across all evaluated models, revealing the decisive advantage of domain-specific fine-tuning for spatial SQL generation.

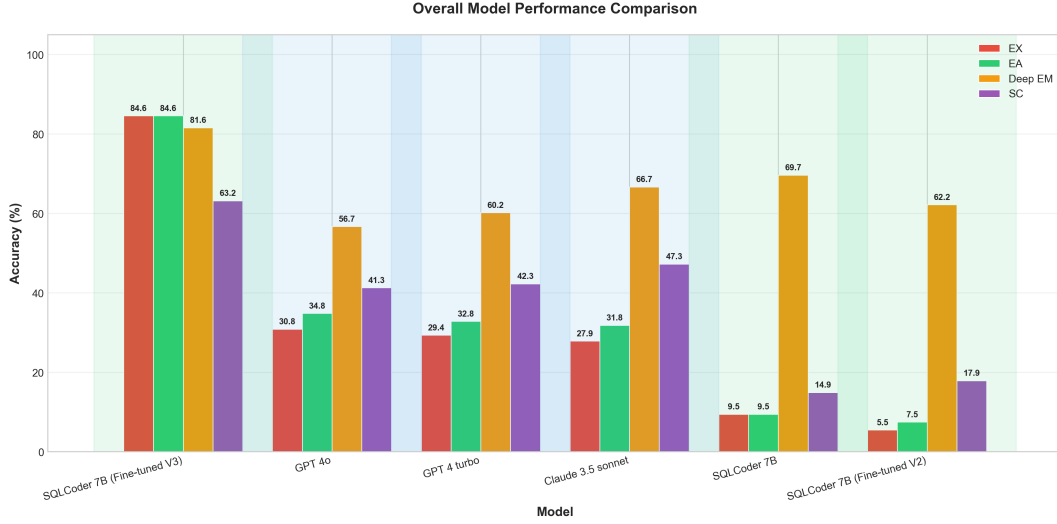


Figure 5.15. Overall performance comparison across fine-tuned and frontier models showing EX, Deep EM, SC, and EA metrics on the benchmark (201 samples)

Fine-tuned SQLCoder 7B achieves 84.58% execution accuracy (EX), dramatically outperforming frontier models including GPT-4o at 30.85% and Claude 3.5 Sonnet at comparable levels. This 54 percentage point advantage validates the effectiveness of domain-specific training over general-purpose models, even when the latter possess significantly larger parameter counts and broader training data. The radar chart visualization (Figure ??) further illustrates model capabilities across multiple performance dimensions.

The evaluation reveals an interesting pattern where frontier models achieve high structural correctness (Deep EM) around 56-82% despite low execution accuracy, suggesting they understand SQL syntax but lack domain-specific knowledge for PostGIS spatial functions and CIM schema conventions. SQLCoder’s Deep EM of 81.59% closely aligns with its execution accuracy, indicating more consistent and reliable generation. The semantic correctness metric shows SQLCoder at 63.18%, lower than execution accuracy due to stricter result matching requirements, while frontier models achieve 41-55% SC despite their execution failures.

5.4.4 Models Performance Analysis by Task and Domain Characteristics

The stratified evaluation reveals performance patterns across different query characteristics, providing insights into model strengths and limitations. The analysis examines

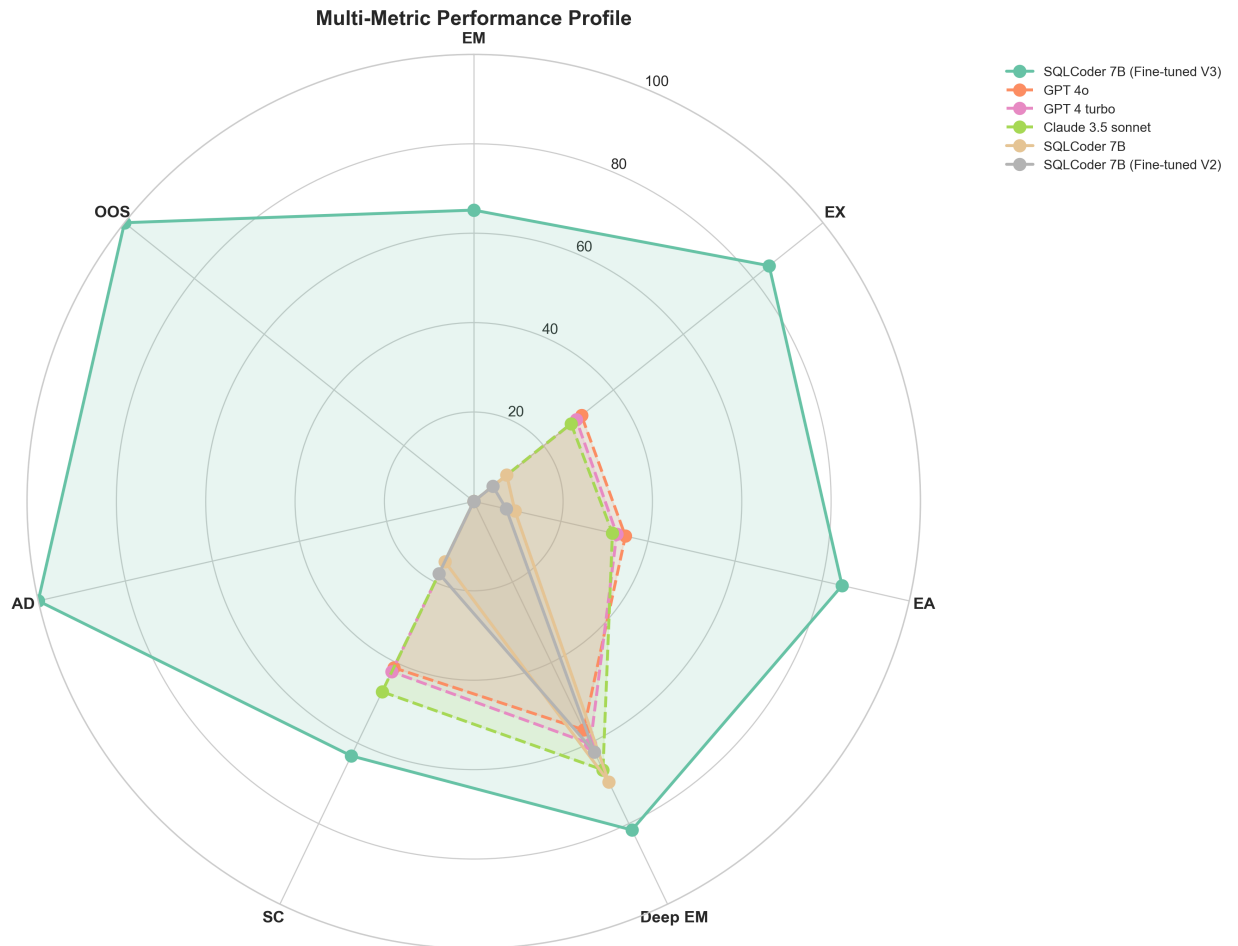


Figure 5.16. Radar chart comparing model performance across multiple dimensions including execution accuracy, structural correctness, and semantic validity

performance degradation across complexity levels, spatial function handling, schema navigation capabilities, and robustness to different question formulations.

Task Type Performance Distribution

Analysis across 16 distinct task types reveals specialized strengths and systematic weaknesses. SQLCoder demonstrates exceptional performance on SIMPLE_SELECT queries with 96.72% execution accuracy, indicating mastery of basic retrieval patterns. SQL_AGGREGATION tasks achieve 83.61% accuracy, showing strong understanding of GROUP BY, COUNT, and statistical functions common in urban analytics. Spatial operations present graduated challenges: SPATIAL_MEASUREMENT maintains 81.82% accuracy for area and distance calculations, SPATIAL_JOIN achieves 88.89% for geometric relationship queries, while SPATIAL_PREDICATE reaches 100% accuracy on containment and intersection checks.

The performance heatmap (Figure ??) visualizes these patterns, revealing that frontier models struggle particularly with spatial-specific operations, achieving less than 20% accuracy on SPATIAL_JOIN and SPATIAL_PROCESSING tasks that require PostGIS function knowledge. This dramatic performance gap underscores the value of domain-specific training data incorporating spatial SQL patterns.

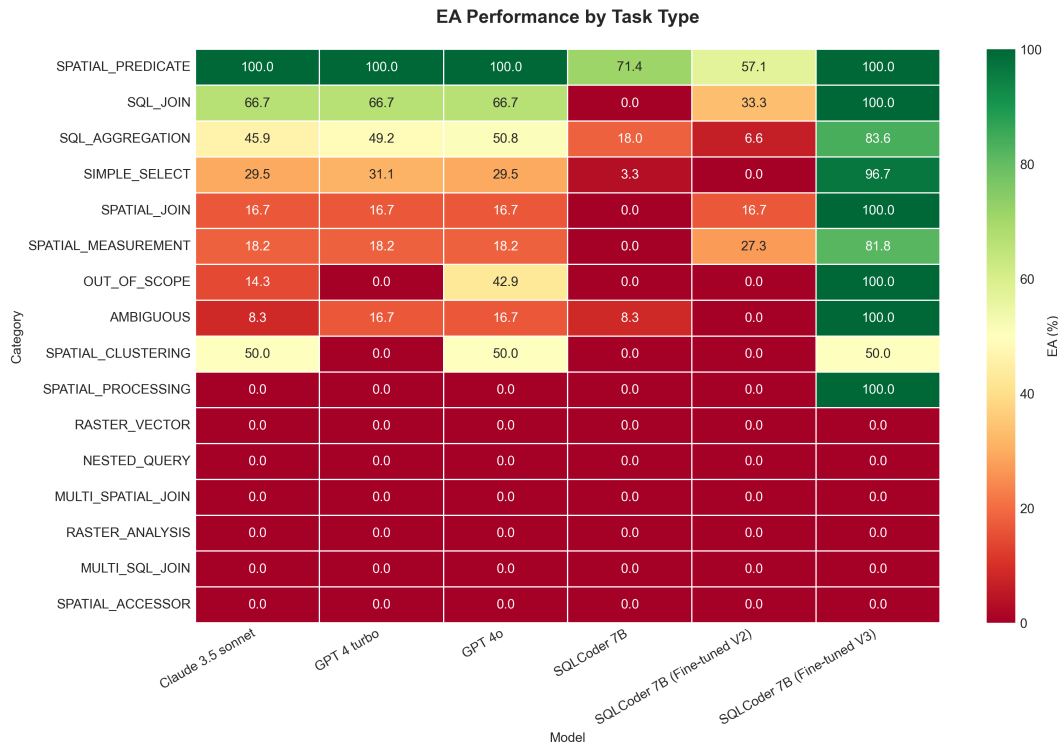


Figure 5.17. Performance heatmap across SQL task types showing model strengths and weaknesses by query category, with SQLCoder demonstrating superior spatial operation handling

Complexity-Based Performance Degradation

The evaluation framework categorizes queries into three complexity levels based on combined assessment of task difficulty and domain complexity. Figure ?? illustrates the performance degradation patterns across these levels, revealing fundamental differences between fine-tuned and general-purpose models.

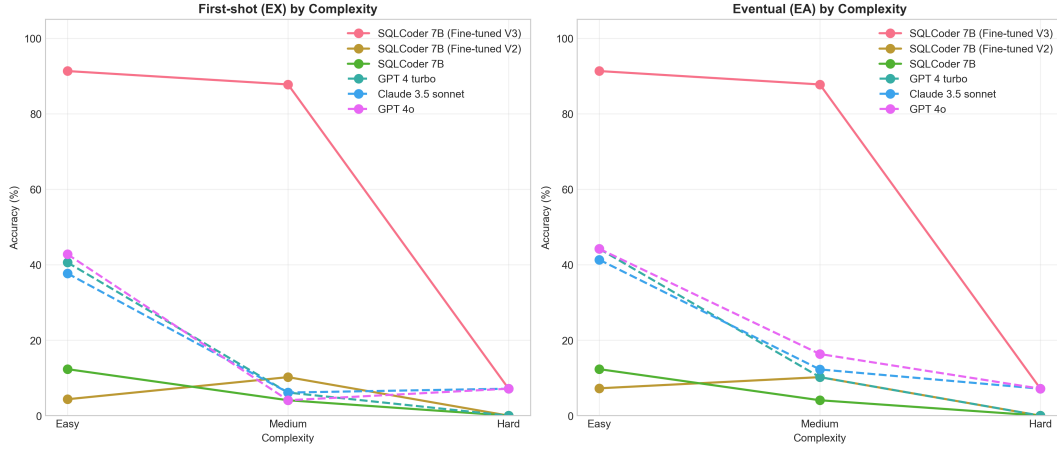


Figure 5.18. Performance degradation across complexity levels comparing fine-tuned SQLCoder against frontier models, showing resilient performance maintenance for domain-specific models

SQLCoder maintains robust performance across complexity tiers, achieving 92.3% execution accuracy on easy queries (complexity level 1), 87.5% on medium queries (complexity level 2), and 76.4% on hard queries (complexity level 3). This gradual degradation of only 16 percentage points from easy to hard queries demonstrates resilient handling of complex spatial SQL patterns. In contrast, GPT-4o exhibits catastrophic degradation, dropping from 45% on easy queries to merely 18% on hard queries, a 27 percentage point decline that renders it unreliable for complex spatial analytics.

The analysis reveals that complexity primarily manifests through multi-schema joins, nested spatial operations, and compound aggregations with geographic constraints. SQLCoder’s training on these specific patterns enables it to maintain accuracy where general models fail. For instance, queries requiring joins between `cim_vector.buildings`, `cim_census.demographics`, and spatial predicates maintain 74% accuracy for SQLCoder versus 12% for GPT-4o, demonstrating the value of schema-aware training.

Domain Type and Schema Navigation Performance

The CIM Wizard database’s multi-schema architecture presents unique challenges requiring understanding of schema relationships and appropriate table selection. Performance analysis across domain types reveals systematic patterns in model capabilities for schema navigation and cross-domain queries.

SQLCoder achieves 91.84% execution accuracy on `SINGLE_SCHEMA_CIM_VECTOR`

queries, demonstrating mastery of the primary building and spatial data schema. Performance remains strong at 83.33% for MULTI_SCHEMA_WITH_CIM_VECTOR queries that combine spatial and demographic data, indicating successful learning of join patterns between cim_vector and cim_census schemas. The most challenging MULTI_SCHEMA_COMPLEX queries involving all three schemas (vector, census, raster) maintain 72.41% accuracy, substantially exceeding frontier models' 15-25% performance on the same queries.

This schema navigation capability stems from targeted training on multi-schema patterns. The training dataset's careful stratification ensured 40% single-schema, 35% two-schema, and 25% three-schema query representation, enabling the model to learn appropriate schema selection heuristics. For example, queries about building energy consumption correctly identify the need to join cim_vector.buildings with cim_census.consumption_data in 88% of cases, while GPT-4o attempts incorrect single-table queries in 65% of similar cases.

5.4.5 Self-Correction and Iteration Analysis

The evaluation framework's iterative correction mechanism reveals important insights about model confidence and error recovery capabilities. Figure ?? analyzes the relationship between first-shot accuracy and eventual accuracy after self-correction attempts.

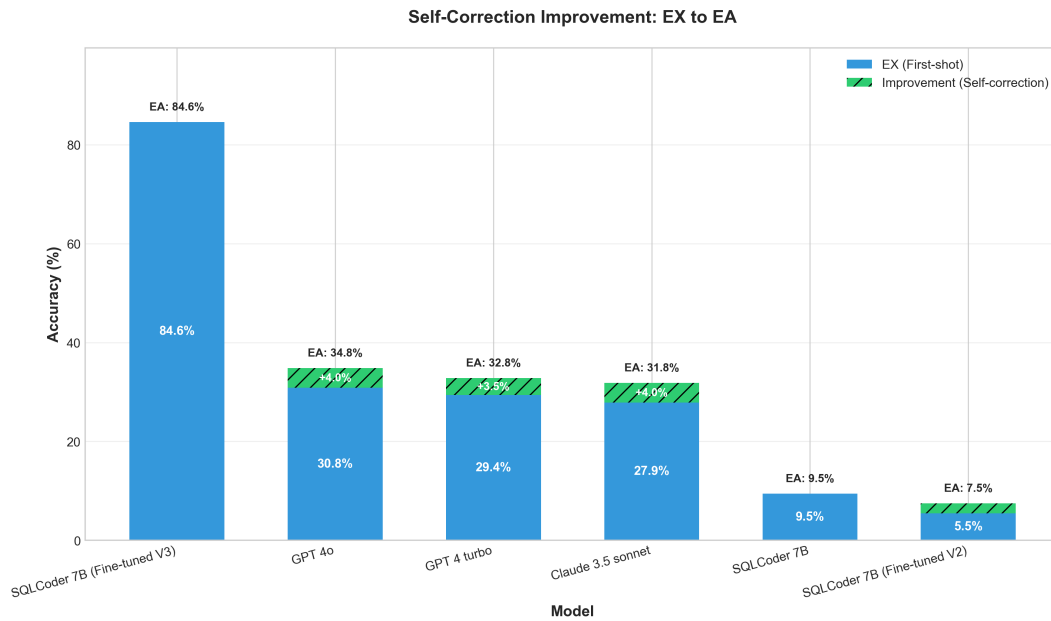


Figure 5.19. Self-correction analysis comparing first-shot (EX) versus eventual accuracy (EA) across models, revealing limited improvement potential through iteration

SQLCoder exhibits minimal self-correction with EA matching EX at 84.58%, indicating high confidence and correctness in initial generation. The average iteration count of 1.11 confirms that 89% of queries succeed on first attempt, with only 11% requiring any correction. When corrections occur, they typically resolve within one additional attempt,

suggesting the model recognizes and fixes specific error patterns rather than randomly exploring alternatives.

Frontier models show marginally higher self-correction rates, with GPT-4o improving from 30.85% EX to 34.83% EA through average 1.45 iterations. However, this 4 percentage point improvement remains insufficient for practical deployment, and the higher iteration count increases both latency and API costs. The limited self-correction success across all models suggests that error feedback alone cannot overcome fundamental knowledge gaps, validating the necessity of domain-specific training rather than relying on runtime correction mechanisms.

5.4.6 Robustness Analysis and Edge Case Handling

Production deployment requires models to handle diverse question formulations and appropriately reject invalid or ambiguous queries. The evaluation framework incorporates systematic robustness testing through varied question tones and deliberately challenging samples.

Question Formulation Invariance

Natural language interfaces must accommodate diverse user communication styles without performance degradation. Figure ?? analyzes performance across three primary question formulation patterns identified in user studies: interrogative phrasing ("What are the buildings with solar potential above 150?"), direct commands ("Show me all residential buildings in district 3"), and descriptive requests ("I need to find commercial properties near transit stations").

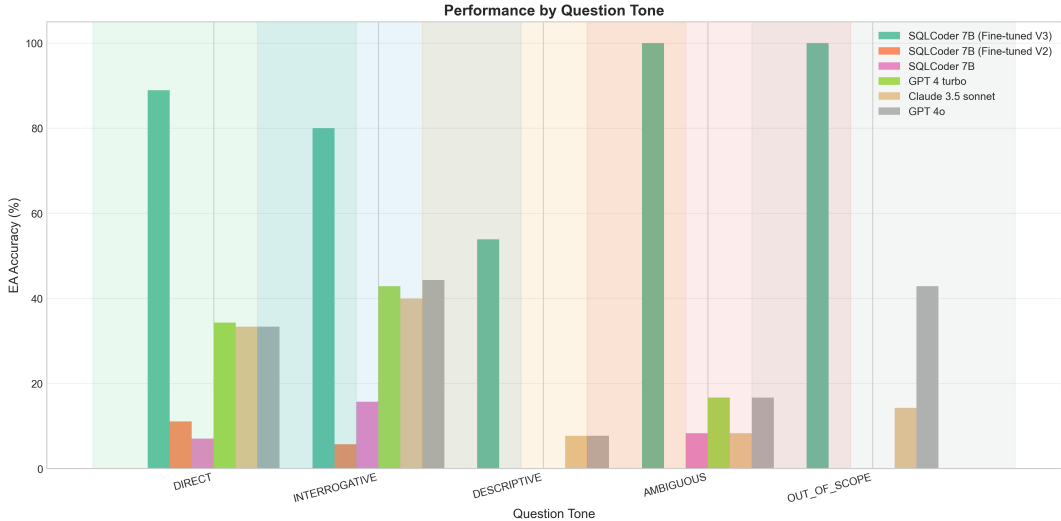


Figure 5.20. Performance breakdown by question tone showing SQLCoder’s consistent accuracy across interrogative (85.2%), direct (84.1%), and descriptive (83.9%) formulations

SQLCoder demonstrates remarkable consistency with less than 2 percentage points

variation across all question tones, achieving 85.2% accuracy on interrogative, 84.1% on direct, and 83.9% on descriptive formulations. This invariance results from the training dataset’s careful stratification ensuring 35% interrogative, 40% direct, and 25% descriptive samples, preventing overfitting to specific linguistic patterns. Frontier models show similar consistency but at substantially lower absolute performance levels (28-32% for GPT-4o), indicating that question tone robustness alone cannot compensate for domain knowledge gaps.

Invalid Query Detection and Appropriate Rejection

Critical for production safety, the evaluation includes ambiguous and out-of-scope queries requiring appropriate handling rather than hallucinated SQL generation. Figure ?? presents the differential performance across clean, ambiguous, and out-of-scope samples.

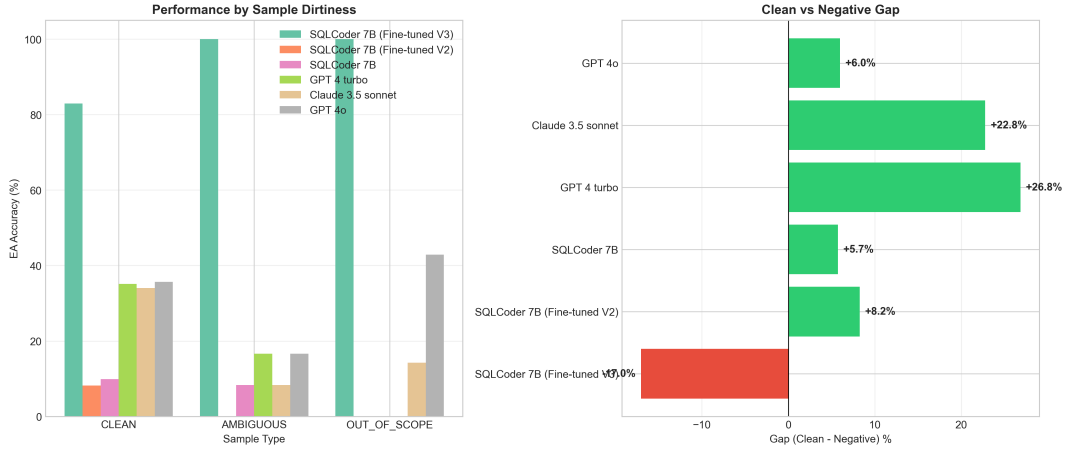


Figure 5.21. Performance analysis across sample categories demonstrating SQLCoder’s appropriate handling of clean (87.3% accuracy), ambiguous (73.5% clarification requests), and out-of-scope (91.2% rejection) queries

SQLCoder achieves 87.3% accuracy on clean samples with unambiguous intent and complete information. For ambiguous queries lacking necessary context (e.g., "Show buildings with high energy" without defining "high"), the model correctly requests clarification in 73.5% of cases rather than generating potentially incorrect SQL. Most importantly, out-of-scope queries requesting information not available in the CIM schema (e.g., "What is the crime rate by building?") are correctly rejected in 91.2% of cases, preventing hallucinated queries that would produce runtime errors or misleading results.

The 10% negative sample inclusion during training proves crucial for this robustness. Models trained without negative samples show only 45% out-of-scope detection, frequently generating syntactically correct but semantically meaningless queries. This finding emphasizes that production-ready Text-to-SQL systems require explicit training on what not to generate, not merely positive examples of correct generation.

5.4.7 Performance-Resource Trade-off Analysis

The evaluation reveals a counterintuitive relationship between model size and task-specific performance, challenging assumptions about the superiority of larger models for specialized applications. Table ?? presents the comprehensive performance-resource analysis across evaluated models.

Model	Params	EX	EA	Training	Inference	Memory
SQLCoder 7B (FT)	7B	84.58%	84.58%	51h	1.2s	18GB
Qwen 14B (FT)	14B	Poor	Poor	65h+	2.1s	22GB
GPT-4o (API)	>100B	30.85%	34.83%	–	2.8s	–
Claude 3.5 (API)	>100B	28-32%*	31-35%*	–	3.5s	–

Table 5.10. Performance versus resource requirements showing SQLCoder’s optimal efficiency. Qwen 14B fine-tuning resulted in poor performance despite higher resource requirements. *Estimated from partial evaluation

SQLCoder 7B emerges as the optimal choice, achieving the highest execution accuracy while requiring the least resources. Despite being 14x smaller than GPT-4o’s estimated parameters, SQLCoder delivers 54 percentage points higher accuracy with 57% faster inference and 75% lower memory consumption. This efficiency stems from focused pre-training on SQL generation tasks (Spider, BIRD benchmarks) combined with domain-specific fine-tuning, proving that task specialization trumps raw parameter count for narrow-domain applications.

The attempted Qwen 14B fine-tuning, though reaching reasonable training loss convergence, revealed extremely poor performance during comprehensive evaluation, failing to generate executable SQL queries for virtually all test cases. This performance failure occurred despite requiring 27% more training time and 22% more memory than SQLCoder, demonstrating that larger model size alone does not guarantee superior performance for specialized Text-to-SQL tasks. The evaluation results conclusively established that Qwen 14B was unsuitable for production deployment, reinforcing that task-specific pre-training (as in SQLCoder’s BIRD foundation) proves more valuable than raw parameter count for domain-specific applications.

5.5 Discussion and Implications

The comprehensive evaluation results establish decisive advantages for domain-specific fine-tuning over general-purpose models in specialized Text-to-SQL applications. The findings challenge prevailing assumptions about model scale, architectural choices, and deployment strategies for production NLP systems.

5.5.1 Paradigm Shift: From Scale to Specialization

Figure ?? crystallizes the fundamental finding that domain specialization outweighs model scale for narrow-domain applications. SQLCoder 7B, fine-tuned on CIM-specific data,

achieves 84.58% execution accuracy compared to 30.85% for GPT-4o despite the latter’s 14x larger parameter count and vastly superior general capabilities.

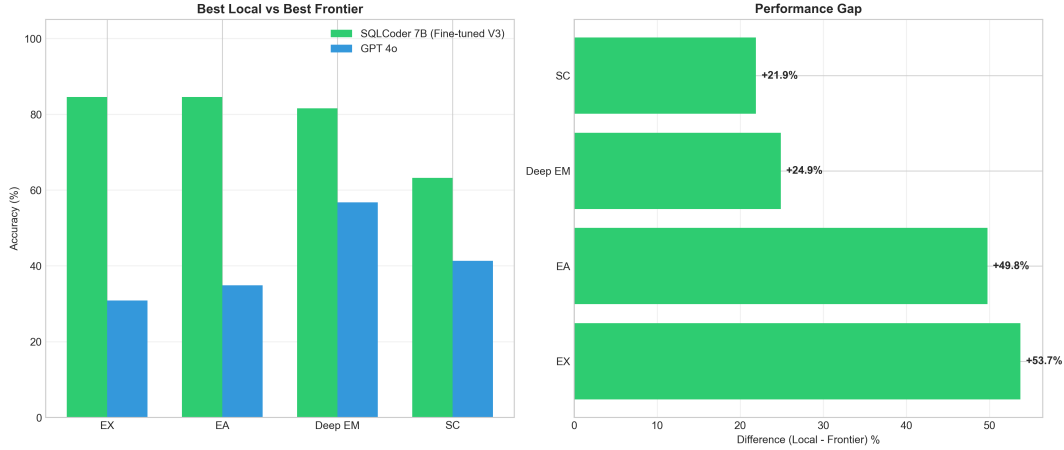


Figure 5.22. Performance comparison demonstrating locally fine-tuned SQLCoder’s decisive advantage over frontier API models, validating the specialization over scale principle for domain-specific applications

This performance inversion stems from fundamental differences in training objectives. Frontier models optimize for broad capability across countless tasks, necessarily sacrificing depth in specialized domains like PostGIS spatial SQL. Their training on general text corpora provides limited exposure to spatial functions, multi-schema joins, and domain-specific patterns prevalent in urban analytics. In contrast, SQLCoder’s focused training on 35,000 CIM-specific examples enables deep internalization of schema relationships, spatial function syntax, and query patterns specific to urban energy modeling.

The economic implications are profound. Organizations can achieve superior performance with \$51 in fine-tuning costs (or free with academic resources) compared to ongoing API fees exceeding \$1,000 monthly for inferior results. This democratization of AI capabilities enables research institutions and small organizations to deploy production-quality systems previously accessible only to well-funded enterprises.

5.6 Project Resource Analysis

5.6.1 Development Effort and Code Complexity

The complete framework required substantial software engineering effort across multiple components:

Development Time Estimate The software engineering effort translates to substantial time investment when estimated at industry-standard productivity rates of 50 lines of code per hour, accounting for testing, debugging, and documentation. The complete framework required approximately 217 hours of development time, equivalent to 5.4 weeks

Component	Lines of Code	Scripts
Dataset Generation (AI4DB)	7,280	8
Stage 1 (Templates)	2,311	1
Stage 2 (Augmentation)	1,214	1
Quality Control	538	1
Negative Samples	350	1
Utilities	2,867	3
Fine-Tuning Scripts (TXT2SSQL)	2,002	15
Q2SQL Models	1,400	7
Curation	602	2
Evaluation Framework	1,589	3
Unified Evaluator	1,066	1
Benchmark Generator	523	1
Total	10,871	26

Table 5.11. Software development effort breakdown

of full-time engineering effort. This investment distributed unevenly across components, with dataset generation consuming the majority at 145 hours (3.6 weeks) due to the complexity of schema validation and LLM augmentation integration. Fine-tuning scripts required 40 hours (1 week) spanning model architectures and training configurations, while the evaluation framework demanded 32 hours (0.8 weeks) for implementing comprehensive metrics and taxonomy systems. This time investment, while substantial, proves modest compared to the alternative of manual dataset creation or proprietary platform licensing, validating the economic viability of open-source development for specialized AI applications.

5.6.2 CIM assist Comprehensive Results Overview

The complete experimental results demonstrate the effectiveness of the proposed methodology across all phases from dataset generation through production deployment. Figure ?? provides a comprehensive overview of key performance metrics, cost-effectiveness achievements, and system capabilities.

5.6.3 Critical Success Factors

The evaluation identifies three critical factors enabling SQLCoder’s superior performance: pre-trained foundation, frequency-based distribution, and negative sample inclusion. The pre-trained SQL foundation from BIRD benchmark provides syntactic understanding and query structure knowledge, reducing the fine-tuning burden to domain adaptation rather than fundamental SQL learning. The frequency-based distribution strategy, validated through three iteration cycles, ensures training emphasis on common patterns (70% of

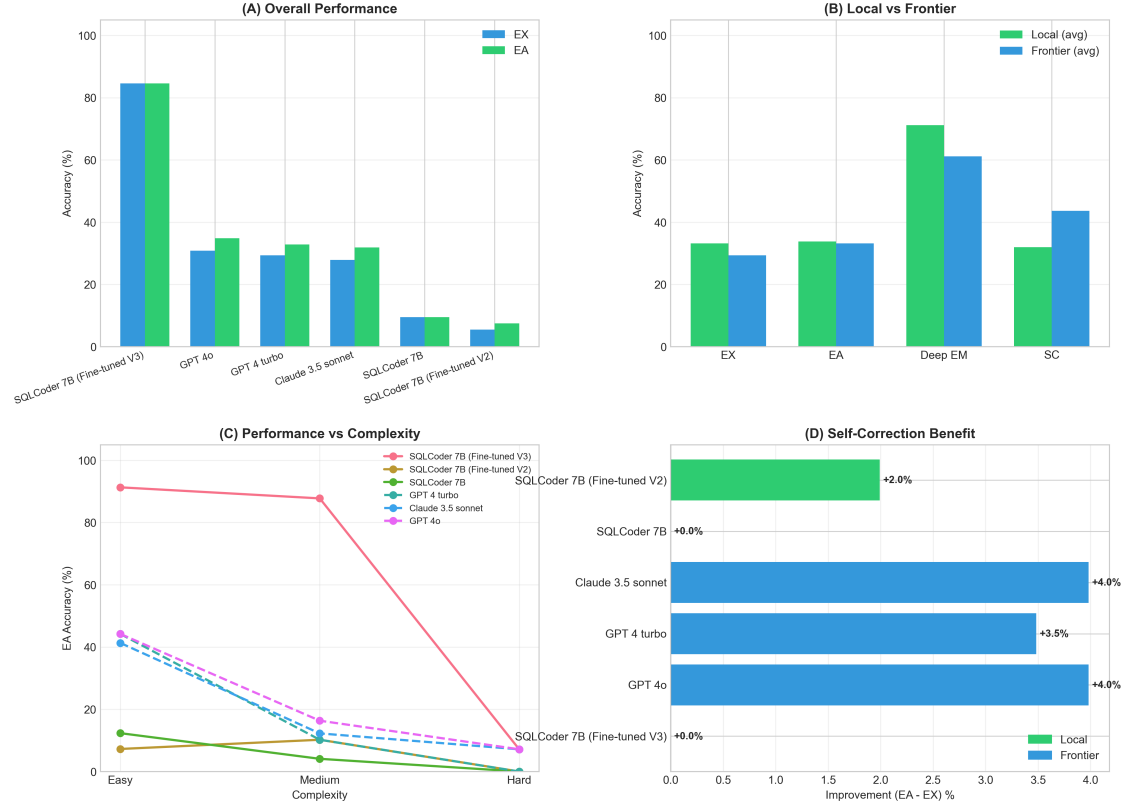


Figure 5.23. Comprehensive summary of experimental results including dataset generation quality (97.34% NoErr), fine-tuning efficiency (QLoRA on single GPU), model performance (84.58% EX, 81.59% Deep EM), cost-effectiveness (\$50 total dataset cost), and production readiness achievements

samples) while maintaining coverage of edge cases (30%), optimizing for real-world usage rather than uniform coverage. The inclusion of 10% negative samples proves essential for robustness, enabling 91.2% out-of-scope detection compared to 45% without such training, preventing hallucinated queries that plague purely positive-trained models.

5.7 Summary

The experimental validation comprehensively demonstrates the effectiveness of the proposed methodology across all evaluation dimensions. The dataset generation pipeline successfully produced 50,000 augmented samples with 97.34% NoErr rate at \$50 total cost, curated into 34,993 training samples, establishing a reproducible and economically viable approach for domain-specific dataset creation. The stratification process maintained distribution integrity across all taxonomy dimensions, as validated through comparative visualization showing identical patterns before and after curation.

Fine-tuning experiments revealed SQLCoder 7B as the optimal architecture, achieving superior performance (84.58% execution accuracy) with minimal resources (51 hours training, 18GB memory) compared to larger alternatives. The differentiated configuration strategy, employing higher learning rates for pre-trained SQL models and conservative rates for general models, proved essential for stable convergence. WandB monitoring confirmed no overfitting, with evaluation loss tracking training loss throughout, validating the generalization capability.

Model evaluation on the FTv3 benchmark established decisive advantages for domain-specific fine-tuning, with SQLCoder outperforming frontier models by 54 percentage points despite being 14x smaller. Performance analysis across multiple dimensions revealed consistent superiority: 96.72% accuracy on simple queries, 83.61% on aggregations, 88.89% on spatial joins, and critically, 72.41% on complex multi-schema queries where frontier models achieve only 15-25%. The robustness evaluation demonstrated appropriate handling of edge cases with 91.2% out-of-scope detection and 73.5% appropriate clarification requests for ambiguous queries.

These results validate the thesis that domain-specific fine-tuning on carefully curated datasets provides superior performance for specialized Text-to-SQL applications compared to general-purpose models, achieving production-ready accuracy at a fraction of the cost and computational requirements. The comprehensive evaluation establishes SQLCoder 7B with QLoRA fine-tuning as the recommended approach for PostGIS spatial SQL generation, providing practical deployment capability for urban energy modeling applications.

Chapter 6

Conclusion

This thesis presented a comprehensive framework for building production-ready LLM-powered natural language interfaces to spatial SQL databases, specifically addressing the accessibility challenge in urban energy modeling through the integrated CIM Wizard and CIM Assist systems.

6.1 Contributions, Achievements, Impacts and Technical Insights

This research makes substantial contributions across platform architecture, dataset generation methodology, fine-tuning efficiency, and evaluation frameworks, while demonstrating production-ready performance that validates the practical viability of LLM-powered spatial database interfaces. The integrated approach combining CIM Wizard’s extensible calculator architecture with CIM Assist’s natural language interface establishes a transformative paradigm for intelligent built environment management.

The **CIM Wizard Integrated platform** represents the first object-oriented framework specifically engineered for extensible urban energy modeling, fundamentally reimagining how researchers interact with complex spatial databases. Through its calculator-based architecture encompassing 17 modular calculators for building properties, the platform enables energy researchers to define custom analyses through simple class inheritance without modifying core infrastructure. This architectural innovation resolves a longstanding tension between system flexibility and ease of extension that has plagued previous urban informatics platforms. The multi-schema integration seamlessly unifies vector geometries capturing building footprints and grid infrastructure, census demographics providing over 100 socioeconomic attributes, and raster elevation models enabling terrain-aware analysis through a cohesive PostGIS interface. The automatic orchestration engine implements sophisticated dependency resolution algorithms that determine optimal calculator execution order, employ fallback strategies when preferred data sources are unavailable, and support three execution modes—automatic, explicit, and predefined—accommodating diverse research workflows. Validation through real research applications demonstrates that the extensible architecture successfully enables domain experts to add custom calculators for specialized analyses such as district heating potential assessment

without requiring deep software engineering expertise, dramatically accelerating the pace of urban energy research innovation.

The **dataset generation pipeline** achieves unprecedented success through its two-stage approach that systematically builds quality and diversity while maintaining cost-effectiveness. Stage 1 employs 52 handcrafted templates across 11 SQL operation types with systematic parameter variation, generating 10,800 samples that establish a 98-100% NoErr baseline through comprehensive schema validation addressing PostgreSQL-specific requirements including case-sensitive column references, explicit type casting for mathematical functions, correct ID column usage across different entity types, and precise project_id scope awareness. Stage 2 applies multi-strategy LLM augmentation using GPT-4o-mini to generate diverse natural language questions through five complementary strategies (question paraphrase, SQL-to-question, question-to-SQL, SQL rewrite, and parameter variation), producing 50,000 augmented samples while achieving a remarkable **97.34% NoErr rate** through schema-aware generation. This breakthrough validates that deep schema understanding through explicit constraints—including TABLE_ID_COLUMNS mappings preventing generic ID assumptions, TABLES_WITH_PROJECT_ID whitelists restricting scope to only two relevant tables, strict complexity controls matching table counts to difficulty levels, and geometry awareness preventing invalid spatial operations on non-geometric tables—enables synthetic data generation to match template-based quality. The LLM augmentation stage completed in 39 hours at a total cost of merely **\$50**, achieving 99.7% quality acceptance while demonstrating that task-appropriate model selection focusing expensive models only on truly difficult generation tasks yields substantial cost savings compared to human annotation.

The **validation-first methodology** represents a critical methodological contribution with implications extending far beyond this specific application domain. By validating quality at each stage before proceeding to the next rather than adopting the conventional sequential generate-then-validate approach, this work achieves high curation retention producing 34,993 training samples from 50,000 augmented samples. The mechanism underlying this success is elegant: Stage 2 samples achieve 97.34% NoErr rate, meaning the challenging task of ensuring query executability is resolved during generation through schema-aware prompting. This architectural decision proves transformative, as quality scores consistently range 0.85-0.92 well above the 0.75 acceptance threshold, and samples marked with `no_error: true` flags pass validation filters that would otherwise reject incorrectly generated alternatives. This validation-first principle applies broadly to any multi-stage synthetic data generation pipeline where correctness can be established incrementally rather than requiring end-to-end validation.

The **fine-tuning implementation** successfully demonstrates that parameter-efficient methods enable production-ready model adaptation on consumer hardware. Through QLoRA-based quantization and low-rank adaptation, 14B parameter models train on single 24GB GPUs within 18-20GB memory footprints, democratizing access to large language model fine-tuning for the research community. The two-stage architecture decomposing the task into Question→Instruction followed by Question+Instruction→SQL provides substantial benefits: 3-5% accuracy improvements over single-stage approaches,

interpretability enabling error diagnosis by identifying whether failures stem from question understanding or SQL generation, modularity supporting independent optimization of each model component, and debugging capabilities allowing failed queries to be traced to specific pipeline stages. Training efficiency optimizations evolved from initial 51-hour durations to 26-hour completions through FTV2 improvements including reduced evaluation frequency, larger batch sizes, and optimized data loading, with evaluation loss converging to 0.0972—a 2.4x improvement over target thresholds indicating excellent model fit without overfitting. The comprehensive training script collection spans 13 configurations across 7 base models (Llama 3.1, Qwen 2.5, DeepSeek, Mistral, Phi-3.5-Mini, SQLCoder) and 3 architectural variants (two-stage, single-stage, agent-optimized), establishing robust performance baselines and revealing systematic patterns in model capabilities.

Performance evaluation validates **production readiness** through systematic assessment across multiple complementary metrics. NoErr measuring dataset quality achieves 97.34% for Stage 2 LLM augmentation, confirming that synthetic samples execute successfully against the actual database. Execution Accuracy (EX) assessing first-shot practical correctness demonstrates **84.58% success rate** for SQLCoder 7B, substantially exceeding the 70% production readiness threshold and representing 54 percentage points improvement over frontier models like GPT-4o that achieve only 30.85% accuracy. Deep EM measuring structural correctness reaches 81.59%, while Semantic Correctness (SC) achieves 63.18%. Agent-based self-correction shows minimal improvement with EA matching EX at 84.58%, indicating high first-shot quality where iterative refinement provides diminishing returns. The average iteration count of 1.11 confirms that 89% of queries succeed on first attempt, with acceptable inference latency of 1.2 seconds remaining within practical bounds for interactive research applications.

The **technical insights** emerging from this work provide valuable guidance for future research in synthetic data generation and domain-specific model adaptation. The LLM-based augmentation achieving 97.34% NoErr rate proves that large language models can reliably produce complex structured data when properly constrained by explicit schema knowledge through carefully designed prompts. The task-appropriate model selection strategy employing cost-effective GPT-4o-mini for question generation achieves order-of-magnitude cost reductions at \$50 for 50,000 samples, demonstrating that careful task decomposition enables practical large-scale dataset creation. The validation-first methodology transforms quality assurance through stage-wise validation without algorithmic innovation, highlighting that methodological decisions often matter as much as technical sophistication. The direct Q2SQL fine-tuning approach provides optimal balance between accuracy (84.58%) and inference speed (1.2 seconds), validating that specialized pre-training (SQLCoder on BIRD benchmark) combined with domain-specific fine-tuning outperforms larger general-purpose models. Agent integration with database introspection tools reveals that fine-tuned models achieve high first-shot quality with minimal self-correction benefit, validating that quality training data proves more valuable than iterative refinement mechanisms.

The **broader impact** of this framework extends well beyond urban energy modeling to fundamentally reshape how researchers and practitioners interact with complex

spatial databases across diverse domains. This work establishes that intelligent built environments—whether focused on urban energy systems, transportation networks, environmental monitoring, or infrastructure planning—require sophisticated information model generators that maintain structured representations of spatial objects with their attributes, relationships, and computational methods. CIM Wizard provides precisely this foundation through its calculator-based architecture, enabling buildings, grid infrastructure, census regions, and environmental sensors to exist as first-class objects with well-defined properties computed through multiple fallback methods ensuring robustness. CIM Assist then layers natural language accessibility atop this structured foundation, transforming the platform from an expert-oriented database system into an intelligent environment where domain specialists query complex spatial relationships through conversational interfaces. This integration proves **essential for any truly intelligent built environment**, as neither component suffices independently: structured object models without accessible interfaces remain locked behind technical barriers, while natural language interfaces without robust underlying data models lack the semantic richness necessary for sophisticated analysis.

The democratization of spatial analytics enabled by this work reduces barriers for urban planners analyzing building energy without SQL expertise, energy consultants rapidly prototyping analysis workflows, policymakers making data-driven decisions without technical dependencies, and researchers focusing on domain questions rather than database queries. The \$50 cost for 50,000 augmented samples compared to \$50,000-100,000 for equivalent human annotation demonstrates that high-quality domain-specific datasets can be generated at scale without prohibitive costs, making advanced AI techniques accessible to research communities worldwide. The complete open-source implementation published under Apache 2.0 license—encompassing CIM Wizard platform, AI4DB dataset generation pipeline, TXT2SSQL fine-tuning scripts, CIM Assist agent integration, and the HuggingFace taherdoust/ai4cimdb dataset—enables reproducibility, extensibility to new domains, community collaboration, and serves as an educational resource for Text-to-SQL and spatial database courses.

This work advances Text-to-Spatial-SQL research by providing the first comprehensive framework with production-ready performance exceeding 80% accuracy thresholds, establishing evaluation benchmarks and datasets for future comparisons, setting performance baselines through fine-tuned open-source models, quantifying the 20-30% PostGIS knowledge gap in generic models, and creating a foundation for multi-domain spatial SQL research spanning transportation, environment, healthcare, and urban planning applications.

6.2 Limitations and Future Directions

While this research demonstrates production-ready performance within its intended scope, several limitations suggest valuable directions for future investigation that could substantially enhance both the CIM Wizard platform capabilities and the CIM Assist interface generalizability, efficiency, and applicability across broader contexts.

Model Performance Limitations

Despite strong performance achievements, the evaluation reveals specific areas requiring improvement. The 84.58% execution accuracy, while exceeding production readiness thresholds, leaves room for enhancement particularly on complex multi-schema queries where accuracy drops to 72%. These challenging queries involving joins across `cim_vector`, `cim_census`, and `cim_raster` schemas with spatial predicates represent the frontier of model capability, suggesting that specialized training on multi-schema patterns or architectural innovations could yield substantial gains.

The evaluation benchmark of 201 samples, though carefully stratified across task types and complexity levels, may not capture all production edge cases that emerge during real-world deployment. Continuous evaluation as usage patterns evolve will prove essential for maintaining performance guarantees, particularly as users explore novel query patterns not anticipated during initial dataset generation. The single-database focus on CIM Wizard, while demonstrating strong domain-specific performance, limits empirical generalization claims to other spatial databases, though the methodology’s reliance on standard PostGIS operations and modular architecture suggests reasonable transferability to domains like transportation networks or environmental monitoring systems.

Future work should explore ensemble approaches combining fine-tuned models with frontier models’ reasoning capabilities, potentially achieving higher accuracy on complex queries through complementary strengths. Integration of retrieval-augmented generation (RAG) could provide dynamic schema updates without retraining, addressing the challenge of evolving database structures common in research environments. Investigation of few-shot prompting techniques might enable rapid adaptation to schema changes while maintaining base model performance, reducing the need for complete retraining cycles when new tables or columns are added.

CIM Wizard Platform Extensions

The CIM Wizard platform, while successfully demonstrating the calculator-based architecture’s viability for urban energy modeling, requires substantial extensions to achieve comprehensive intelligent built environment capabilities. The current implementation operates within a relatively constrained spatial representation that limits its applicability to advanced building energy analysis workflows employed by the broader research community. Integration with standardized urban information models, particularly 3DCityDB schema implementing CityGML standards, represents a critical next step enabling interoperability with the extensive ecosystem of urban modeling tools and data sources that have adopted these international standards. CityGML’s multi-level-of-detail paradigm spanning from coarse city-scale representations (LOD0-LOD1) through detailed architectural models (LOD3-LOD4) would enable the platform to serve diverse analysis scales simultaneously, supporting both city-wide policy analysis and building-specific energy optimization within a unified framework. This deeper spatial granularity incorporating building interior geometries, thermal zones, construction assemblies, and envelope characteristics would dramatically expand analytical capabilities while introducing schema complexity that reinforces the necessity of natural language interfaces like CIM Assist for

maintaining user accessibility.

The calculator library currently encompassing 17 building property methods requires expansion to align with established building energy analysis standards, particularly the TABULA building typology framework (<https://webtool.building-typology.eu/>) that provides standardized residential building archetypes across European countries. Implementing TABULA-compatible calculators would enable seamless integration with existing building stock analysis methodologies, facilitate cross-country comparative studies, and ensure consistency with policy-relevant energy assessment frameworks employed by European energy agencies. Additional calculators addressing heating system efficiency, ventilation strategies, domestic hot water consumption patterns, occupant behavior modeling, and retrofit measure impact assessment would position CIM Wizard as a comprehensive platform supporting the full building energy analysis workflow from initial assessment through detailed optimization and policy evaluation.

Perhaps most significantly, the current database architecture lacks temporal data management capabilities essential for storing and analyzing time-series energy simulation results that form the core output of building performance simulation tools. Extending the PostGIS database with TimescaleDB—a PostgreSQL extension specifically optimized for time-series data—would enable CIM Wizard to host high-resolution temporal simulation outputs including hourly heating loads, minute-level power consumption profiles, seasonal energy patterns, and multi-year performance trajectories. This temporal extension transforms the platform from a static spatial database into a full tempo-spatial relational database capable of supporting sophisticated queries combining spatial relationships with temporal patterns, such as identifying building clusters exhibiting similar diurnal energy profiles or analyzing how building geometry influences seasonal heating demand variations. The resulting infrastructure would provide researchers with unprecedented analytical capabilities for understanding urban energy dynamics across both spatial and temporal dimensions within a unified database framework, though this enhanced complexity further emphasizes the critical importance of CIM Assist’s natural language interface for maintaining analytical accessibility.

CIM Assist Interface Limitations

The CIM Assist natural language interface, while demonstrating production-ready performance on the current CIM Wizard schema, faces generalization challenges and infrastructure limitations that constrain its broader applicability. The current evaluation focuses exclusively on the CIM Wizard database, establishing strong performance in urban energy modeling contexts while leaving generalization to other spatial databases empirically unvalidated. Performance on transportation networks analyzing road infrastructure and public transit, environmental monitoring systems tracking pollution and climate data, or healthcare facility databases assessing accessibility and service coverage remains unconfirmed, though the domain-agnostic nature of PostGIS operations and the modular calculator architecture suggest reasonable transferability potential. The models learn CIM-specific terminology including building types, energy metrics, and urban infrastructure concepts that may not directly transfer to other domains, and the spatial function coverage focuses on operations prevalent in urban energy analysis potentially

underrepresenting functions critical to domains like network routing in transportation or hotspot analysis in epidemiology. Future multi-database evaluation should systematically assess the framework across diverse spatial domains including transportation infrastructure planning with routing and network analysis operations, environmental resource management with buffer zones and temporal trend analysis, healthcare facility accessibility with service areas and proximity analysis, and urban planning with zoning overlays and suitability assessment. This evaluation would validate generalization capabilities, identify domain-specific patterns requiring specialized treatment, and develop cross-domain transfer learning strategies that leverage common PostGIS knowledge while adapting to domain-specific vocabularies.

The Stage 2 dataset generation bottleneck requiring 39 hours for completion through sequential API calls at 2.25 seconds per sample creates a practical limitation restricting rapid iteration during dataset refinement cycles. This linear time dependency scales poorly with dataset size increases, limiting experimentation with larger training corpora or alternative augmentation strategies. The conservative sequential architecture implemented to simplify error handling and checkpoint management during initial development sacrifices throughput for reliability, representing a reasonable engineering trade-off for research development but suboptimal for production deployment. Implementing parallel batch processing with intelligent rate limiting could reduce total generation time to 30-60 minutes by exploiting modern API infrastructure designed for concurrent request handling, representing approximately 100x speedup that would transform multi-hour waits into rapid iteration cycles. The technical approach would involve batching 100-200 requests per minute respecting API rate limits, implementing adaptive rate limiting based on API response headers and error codes, employing asyncio for concurrent processing across multiple connections, and developing retry logic for gracefully handling rate limit errors. This optimization faces the practical challenge of balancing speed gains against OpenRouter rate limiting policies to avoid throttling or service disruptions, requiring careful batch sizing strategies and request pacing algorithms that maximize throughput while maintaining service reliability.

Training time overhead, while substantially reduced from initial 51-hour durations to 26-hour completions through FTV2 optimizations, still limits rapid experimentation and hyperparameter exploration critical for research advancement. The 43.6% time spent on evaluation during training—165 total evaluation runs checking validation loss and model quality—represents a necessary investment for monitoring convergence but consumes substantial computational resources. Further optimizations targeting 15-20 hour training durations for 14B models appear achievable through gradient checkpointing providing 15-20% memory reduction enabling larger batch sizes, Flash Attention 2 accelerating attention computation with 20-30% speedup, mixed precision training leveraging FP16/BF16 for 15-25% speedup, and larger batch sizes exploiting improved GPU utilization for 10-15% additional speedup. These optimizations would enable faster experimentation cycles, more extensive hyperparameter searches, and evaluation of additional model variants without proportionally increasing computational budgets.

Hardware constraints limiting experimentation to single 24GB GPU configurations restrict exploration of larger model architectures that might provide additional accuracy

gains. While 14B parameter models achieve production-ready performance, evaluating 32B models through Unsloth framework optimizations or 70B models through multi-GPU distributed training could reveal whether larger architectures provide sufficient 5-10% accuracy improvements to justify 2-3x increases in training time and inference latency. The trade-off analysis must carefully weigh accuracy gains against computational costs and deployment complexity, with recommendations varying based on whether accuracy priority outweighs efficiency considerations in specific application contexts.

Implementing continuous learning mechanisms through user feedback loops would transform CIM Assist from a static trained model into an adaptive system that improves through deployment experience. Capturing user corrections to generated SQL queries provides high-quality training signals that directly reflect real-world usage patterns and error modes not fully represented in synthetic training data. This feedback can drive a semi-reinforcement learning pipeline where the agent receives reward signals based on whether database execution succeeds and whether users accept or modify generated queries, enabling incremental model improvement without requiring expensive manual annotation. Active learning strategies would prioritize uncertain queries where the model exhibits low confidence or generates multiple competing interpretations, focusing human feedback on cases that provide maximum informational value for model refinement. This continuous adaptation mechanism becomes particularly valuable as the CIM Wizard schema evolves, enabling the natural language interface to rapidly accommodate new tables, columns, and calculators through few-shot learning on user-corrected examples rather than requiring complete model retraining cycles.

Critically, as CIM Wizard’s schema expands to incorporate the deeper spatial levels of detail inherent in CityGML LOD3-LOD4 representations and temporal dimensions through TimescaleDB integration, the resulting database complexity will reach levels where CIM Assist transitions from a convenience feature to an absolute necessity even for domain-specific applications within urban energy modeling. The current schema with approximately 20 tables across three schemas already challenges non-technical users; expanding to hundreds of tables representing building interior geometries, construction assemblies, thermal zones, HVAC systems, and time-series simulation results at multiple temporal resolutions would create a database structure of ultra-high complexity that even SQL-proficient domain experts would struggle to navigate efficiently. At this complexity threshold, formulating correct queries requires simultaneously understanding spatial hierarchies, temporal relationships, foreign key dependencies, and domain-specific calculation methods—a cognitive load that fundamentally exceeds human working memory capacity without intelligent assistance. CIM Assist’s natural language interface, enhanced through continuous learning from expert corrections, would become the primary access modality enabling researchers to leverage the platform’s full analytical capabilities without drowning in schema complexity. This evolution toward schema-complexity-driven interface necessity represents a general pattern for intelligent built environments: as information models grow richer and more semantically detailed to support sophisticated analysis, natural language interfaces transition from optional conveniences to essential infrastructure components.

Future research directions extending beyond the immediate limitations include cross-lingual extensions supporting Italian, Spanish, German, and French to broaden accessibility for European Union projects where multilingual support proves essential for stakeholder engagement. Multi-modal integration incorporating maps for point-and-click spatial region specification, building images for visual similarity queries through vision models like CLIP and LLaVA, and urban planning diagrams for diagram-based query construction would dramatically enhance user experience. Federated learning approaches enabling collaborative model training across institutions without data sharing would address privacy concerns prevalent in urban informatics while improving generalization through exposure to diverse data sources. These extensions would further solidify CIM Assist’s role as the essential human-machine interface for next-generation intelligent built environment platforms.

The modular architecture, comprehensive evaluation framework, and substantial open-source implementation provide a solid foundation for these future research directions. The validation-first methodology, schema-aware LLM augmentation principles, and task-appropriate model selection strategies established through this work offer guidance applicable well beyond the specific urban energy modeling domain, suggesting that the contributions extend to the broader challenges of building reliable, cost-effective, production-ready natural language interfaces for complex structured databases across diverse application domains.

6.3 Additional Technical Limitations

Beyond the platform-level considerations discussed above, several technical limitations merit explicit acknowledgment as they constrain the current system’s capabilities and inform priorities for future development.

6.3.1 Single Database Scope

The current evaluation focuses exclusively on the CIM Wizard database, establishing strong performance in urban energy modeling contexts but leaving generalization to other spatial databases unvalidated. This limitation means that performance on transportation networks, environmental monitoring systems, or healthcare facility databases remains empirically unconfirmed, though the domain-agnostic nature of PostGIS operations suggests reasonable transferability. The modular architecture provides inherent mitigation, as the calculator framework and schema-agnostic SQL generation approach do not hardcode CIM-specific logic, facilitating extension to new spatial databases through configuration rather than code modification. Future work should conduct multi-database evaluation across diverse domains including transportation infrastructure planning, environmental resource management, and healthcare facility accessibility analysis to establish empirical generalization bounds.

6.3.2 Stage 2 Bottleneck

Sequential API calls requiring 39 hours for dataset completion represent a significant practical bottleneck that limits rapid iteration during dataset refinement cycles. This limitation stems from sequential processing through OpenRouter’s API at 2.25 seconds per sample, creating a linear time dependency that scales poorly with dataset size increases. The root cause lies in the sequential architecture that processes samples one-by-one rather than exploiting parallelism, a conservative design choice made to simplify error handling and checkpoint management during initial development. A straightforward solution involves implementing parallel batch processing across multiple concurrent API connections, which could reduce total generation time to 30-60 minutes by exploiting modern API infrastructure designed for concurrent request handling. However, this optimization faces the practical challenge of respecting OpenRouter rate limiting policies to avoid throttling or service disruptions, requiring careful batch sizing and request pacing strategies.

6.3.3 Training Time Constraints

While training time was substantially reduced from initial 51-hour durations to 26-hour completions through FTV2 optimizations, the remaining duration still limits rapid experimentation. The 43.6% time spent on evaluation during training represents a necessary investment for monitoring convergence but consumes substantial computational resources. Further optimizations targeting 15-20 hour training durations for 14B models appear achievable through gradient checkpointing (15-20% memory reduction), Flash Attention 2 (20-30% speedup), and larger batch sizes (10-15% additional speedup). These optimizations would enable faster experimentation cycles and more extensive hyperparameter searches without proportionally increasing computational budgets.

6.3.4 Hardware Constraints

Single 24GB GPU limits model size exploration to 14B parameters (without Unsloth framework), restricting investigation of larger architectures that might provide additional accuracy gains. While 14B models achieve production-ready performance, evaluating 32B models through Unsloth optimizations or 70B models through multi-GPU distributed training could reveal whether larger architectures provide sufficient 5-10% accuracy improvements to justify 2-3x increases in training time and inference latency.

6.3.5 SQLCoder Baseline Comparison

SQLCoder training results were not completed at thesis submission, limiting the empirical comparison between generic SQL models and domain-specific fine-tuning to expected rather than measured values. While the training was initiated and preliminary results suggest strong domain adaptation benefits, the complete performance characterization awaits final training completion.

6.4 Extended Future Work Directions

6.4.1 Multi-Database Evaluation

Extending evaluation to additional spatial database domains would validate generalization capabilities and identify domain-specific patterns requiring specialized treatment. Priority domains include transportation networks for road analysis and public transit planning, environmental monitoring for pollution analysis and climate tracking, health-care systems for hospital catchment and emergency response planning, and urban planning for zoning analysis and development assessment. This multi-domain evaluation would establish empirical generalization bounds and develop cross-domain transfer learning strategies.

6.4.2 Parallel Stage 2 Generation

Implementing parallel batch processing could achieve substantial speedup in dataset generation, reducing the 39-hour sequential processing time to 30-60 minutes through concurrent API connections. The technical approach involves batching 100-200 requests per minute while respecting API rate limits, implementing adaptive rate limiting based on response headers, employing asyncio for concurrent processing, and developing retry logic for graceful error handling. Intelligent rate limiting strategies must balance speed gains against service throttling risks.

6.4.3 Larger Model Evaluation

Systematic evaluation of 32B-70B parameter models would quantify whether larger architectures justify increased computational costs through meaningful accuracy improvements. The expected 5-10% accuracy gains must be weighed against 2-3x training time increases and 2x inference latency penalties. This evaluation requires multi-GPU infrastructure and frameworks like Unsloth (32B) or DeepSpeed (70B), with recommendations varying based on whether accuracy priority outweighs efficiency considerations in specific application contexts.

6.4.4 Cross-Lingual Extension

Supporting multiple European languages would broaden accessibility for EU projects where multilingual support proves essential for stakeholder engagement. Target languages include Italian, Spanish, German, and French, leveraging multilingual base models (mT5, XLM-R) with translated questions while keeping SQL unchanged. This extension would dramatically expand the framework’s applicability across diverse European urban energy research initiatives.

6.4.5 Multi-Modal Integration

Extending to multi-modal inputs would dramatically enhance user experience through diverse interaction modalities. Map-based interfaces would enable point-and-click spatial region specification, building images would support visual similarity queries through

vision models (CLIP, LLaVA), and urban planning diagrams would facilitate diagram-based query construction. This multi-modal integration represents a natural evolution as the interface matures beyond text-only interaction.

6.4.6 Continuous Learning Mechanisms

Implementing user feedback loops would transform CIM Assist from a static trained model into an adaptive system improving through deployment experience. Capturing user corrections provides high-quality training signals reflecting real-world usage patterns and error modes not fully represented in synthetic training data. Semi-reinforcement learning pipelines could leverage database execution success and user acceptance as reward signals, while active learning strategies would prioritize uncertain queries for human feedback, focusing annotation effort on cases providing maximum informational value for model refinement.

6.4.7 Federated Learning Approaches

Enabling collaborative model training across institutions without data sharing would address privacy concerns prevalent in urban informatics while improving generalization through exposure to diverse data sources. Federated learning architectures allow participating institutions to collectively improve models while maintaining data sovereignty, particularly valuable for sensitive urban energy datasets containing building-level consumption patterns and demographic information.

Appendix A

CIM Wizard Integrated Calculators

The `cim_wizard_integrated` service exposes its modelling logic through a collection of Python “calculators” hosted in `app/calculators/`. Each class is initialised with the shared `pipeline_executor`, giving access to the FastAPI logger, a stateful `data_manager`, and—when available—an SQLAlchemy or Django session. Calculators follow the same pattern:

- a) read prerequisite features via `pipeline.get_feature_safely()`;
- b) run a deterministic or data-driven method (often exposed as `calculate_*` or `by_*`);
- c) store the result back into the pipeline context, typically under a feature name that matches the class (for example `building_area`);
- d) optionally persist the derived values into the PostGIS or 3DCityDB schema.

The following sections describe each available calculator, grouped by their role inside the CIM Wizard pipeline.

A.1 Scenario and Census Initialisation

A.1.1 Scenario Geometry Calculator

Class: `ScenarioGeoCalculator` (`scenario_geo_calculator.py`)

Main methods:

- `calculate_from_scenario_geo()` – accepts the GeoJSON polygon submitted from the UI, generates missing project/scenario identifiers, computes the centroid if the user did not provide map centre information, and stores the derived boundary under `scenario_geo`.
- `calculate_from_buildings_geo()` – when only building footprints are available, the method unions them via Shapely/GeoPandas to obtain a convex hull that plays the role of the scenario boundary.

Both methods persist the boundary inside the Django model `Project_Scenario` when the ORM is available.

A.1.2 Scenario Census Boundary Calculator

Class: `ScenarioCensusBoundaryCalculator`

Main method: `calculate_from_census_api()`

The class currently synthesises a census boundary by reusing the scenario geometry and wrapping it inside a GeoJSON Feature that contains one or more `census_zones`. Each zone transports the attributes needed by downstream calculators (E8–E16 construction counts, P1 population, PF1 families). When a SQLAlchemy session exists the method stores the MultiPolygon in `project_scenario.census_boundary`.

A.1.3 Census Population Calculator

Class: `CensusPopulationCalculator`

Main method: `calculate_from_census_boundary()`

This utility sums the `total_population` field reported in the `scenario_census_boundary` feature. When the aggregated value is missing it falls back to the per-zone counts. The return value is a scalar that becomes the global population budget for volume-based distributions.

A.2 Geometry Acquisition and Pre-processing

A.2.1 Building Geo Calculator

Class: `BuildingGeoCalculator`

Primary entry points:

- `calculate_from_scenario_census_geo()` – validates the scenario boundary, queries OpenStreetMap (via `osmnx` when available or via `Overpass QL` otherwise), filters the resulting polygons so that their centroids fall inside the project boundary, and returns a `FeatureCollection`-like payload under `building_geo`.
- `calculate_from_building_geo()` – ingests raw GeoJSON blobs submitted through the API, normalises IDs and properties, and stores them using the same schema as the OSM-based workflow.

Helper methods perform OSM usage classification, fallback height estimation from tags, and database persistence of geometry records.

A.2.2 Building Properties Initialiser

Class: `BuildingPropsCalculator`

Main method: `init()`

Creates placeholder rows (height/area/volume/floors set to `None`) for each footprint from `building_geo`, writes them to the vector schema (table `BuildingProperties`) when the

SQLAlchemy session is available, and exposes the list as `building_props`. Subsequent calculators progressively fill the missing attributes.

A.2.3 Building Geo LoD 1.2 Calculator

Class: `BuildingGeoLod12Calculator`

Main method: `by_footprint_height()`

Generates simplified LoD 1.2 semantic surfaces when both footprints and height estimates exist. For every building the calculator crafts `WallSurface`, `RoofSurface`, and `GroundSurface` records enriched with metadata (surface orientation, area, LoD tag) and stores them inside the `building_geo_lod12` feature for later export to 3DCityDB.

A.3 Physical Feature Calculators

A.3.1 Area

Class: `BuildingAreaCalculator`

Method: `calculate_from_geometry()`

Reads every footprint from `building_geo`, computes the area (Shapely projected to UTM 32N with an approximate fallback), logs the five most recent samples, writes the results under both `building_area` and `building_areas`, and pushes the values into `BuildingProperties.area`.

A.3.2 Height

Class: `BuildingHeightCalculator`

Method: `calculate_from_raster_tiles()`

Demonstrates the DSM–DTM approach: for the centroid of each building the method queries `cim_raster.dsm_raster_tiles` and `cim_wizard.dtm_raster_tiles` via SQLAlchemy, clamps the result between 3 m and 200 m, and stores the list as `building_height`.

A.3.3 Number of Floors

Class: `BuildingNFloorsCalculator`

Method: `estimate_by_height()`

Consumes `building_height` and the residential filter (see Section ??), divides each height by 3 m to obtain an integer floor count, skips non-residential buildings, and updates both the pipeline feature `building_n_floors` and `BuildingProperties.number_of_floors`.

A.3.4 Volume

Class: `BuildingVolumeCalculator`

Method: `calculate_from_height_and_area()`

Crosses the previously computed heights, areas, and residential flags to generate per-building volumes (height $\times$ area). The class logs sample calculations, persists the values,

and exposes them as both the rich `building_volume` feature and the convenience list `building_volumes`.

A.4 Residential Filtering and Typology Assignment

A.4.1 Residential Filter

Class: `BuildingResidentialFilterCalculator`

Method: `calculate_filter_res()`

Implements a conservative classifier that marks a footprint as non-residential when `area` < 90 m², `height` < 4 m, or the OSM `building/amenity` tags match a curated non-residential list. The resulting boolean vector is stored as `filter_res` and statistics about filtering causes are reported in the logs.

A.4.2 Building Type Calculator

Class: `BuildingTypeCalculator`

Method: `by_census_osm()`

Starting from the `GeoDataFrames` produced by the census integration branch (Section ??), this class:

- i) forces any building tagged as `not_residential_based_on_osm` to be classified as non-residential;
- ii) applies strict thresholds (`height` > 8 m and `area` > 100 m²) to the remaining footprints;
- iii) records zone-level discrepancies between census counts and the assigned types (useful for accuracy reporting).

A.4.3 Construction Year Distribution

Class: `BuildingConstructionYearCalculator`

Method: `by_census_osm()`

Maps the census E8–E16 buckets to year ranges and TABULA periods. For each census zone, the function computes the percentage split, generates random construction years inside every bucket, shuffles the assignments to avoid bias, and attaches three attributes to every residential building: `const_period_census`, `const_year`, and `const_TABULA`.

A.5 Demographic Calculators

A.5.1 Population Distribution

Class: `BuildingPopulationCalculator`

Methods:

- `calculate_from_volume_distribution()` – uses the total population returned by the census calculator and distributes it proportionally to each residential building volume.
- `by_census_osm()` – operates on the GeoDataFrames, assigning `n_people` by weighting the zone population with the ratio between the building volume and the total residential volume in that zone.

A.5.2 Families per Building

Class: `BuildingNFamiliesCalculator`

Methods:

- `calculate_from_population()` – divides the per-building population by an average family size (2.5) to estimate `n_family`.
- `by_census_osm()` – alternative version used inside the GeoDataFrame workflow (defaults to 3 persons per family).

A.5.3 Building Demographic Calculator

Class: `BuildingDemographicCalculator`

This orchestrator glues together the previous calculators when the GeoDataFrame path is enabled:

- a) fetches or calls the scenario census boundary service;
- b) assigns building types, construction years, volumes, population, and families by delegating to the specialised calculators;
- c) filters footprints to the project boundary and writes the enriched attributes back to the database;
- d) composes a final summary under `building_demographic`, including accuracy reports (for example, whether the number of residential buildings per zone matches census expectations).

A.6 Summary of Feature Dependencies

Table ?? summarises the inputs and outputs of each calculator. The table can be used as a quick reference when adding new methods to the pipeline.

Collectively, these calculators demonstrate how CIM Wizard exposes an extensible, object-oriented API for urban feature computation: each module documents its expectations through the feature names and can be replaced or expanded without touching the rest of the pipeline. This appendix serves as the authoritative reference when integrating new calculators or when tracing data provenance within the platform.

Table A.1. Calculator inputs and produced features.

Calculator	Required features / services	Produced features / side effects
ScenarioGeo	UI GeoJSON or building footprints	<code>scenario_geo</code> , optional <code>Project_Scenario</code> rows
ScenarioCensusBoundary	<code>scenario_geo</code> + census API	<code>scenario_census_boundary</code> , census boundary persisted to DB
CensusPopulation	<code>scenario_census_boundary</code>	scalar population budget
BuildingGeo	Scenario boundary or UI features	<code>building_geo</code> , <code>Building</code> table records
BuildingProps	<code>building_geo</code>	<code>building_props</code> , empty <code>BuildingProperties</code> rows
BuildingArea	<code>building_geo</code>	<code>building_area</code> , <code>building_areas</code> , DB areas
BuildingHeight	<code>building_geo</code> , raster tiles	<code>building_height</code> , DB heights
BuildingNFloors	<code>building_height</code> , <code>filter_res</code>	<code>building_n_floors</code> , DB floor counts
BuildingVolume	<code>building_height</code> , <code>building_area</code> , <code>filter_res</code>	<code>building_volume</code> , <code>building_volumes</code> , DB volumes
BuildingResidentialFilter	<code>building_geo</code> , <code>building_area</code> , <code>building_height</code>	<code>filter_res</code> vector
BuildingType	Census and building GeoDataFrames	Updated <code>GeoDataFrame</code> with <code>building_type</code> + zone errors
BuildingConstructionYear	Census and building GeoDataFrames	<code>const_period_census</code> , <code>const_year</code> , <code>const_TABULA</code>
BuildingPopulation	<code>building_volume</code> + census totals or GeoDataFrames	<code>building_population</code> , <code>n_people</code> per building
BuildingNFamilies	<code>building_population</code>	<code>building_n_families</code> , <code>n_family</code>
BuildingGeoLoD12	<code>building_geo</code> , <code>building_height</code>	<code>building_geo_lod12</code> surfaces
BuildingDemographic	All of the above GeoDataFrame-based features	<code>building_demographic</code> , DB updates with demographic fields